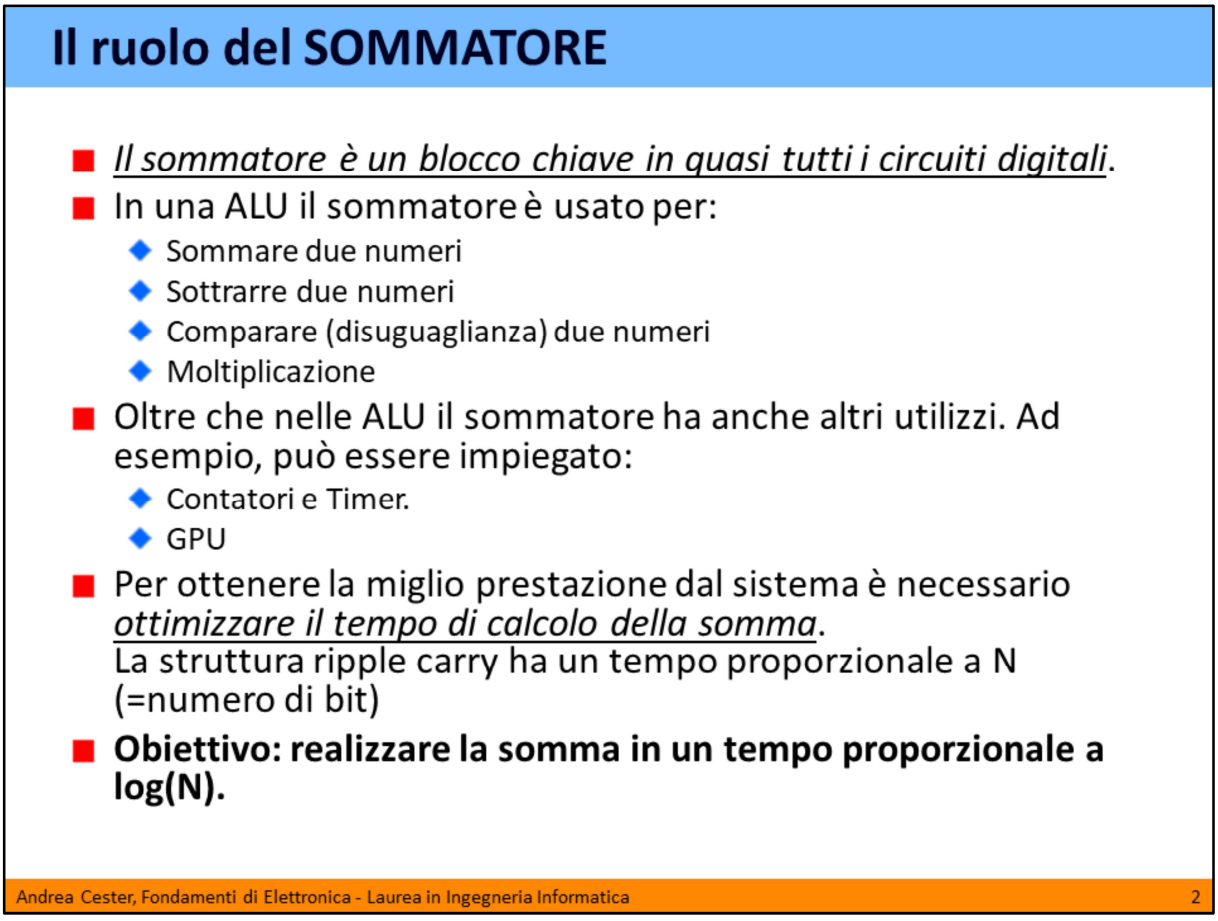
Ottimizzazione del sommatore

Rabaey, Circuiti Integrati Digitali 2°Ed: Cap. 11 p.597-642

- Ottimizzazione del percorso di propagazione del riporto
- Architetture ottimizzate:
  - Carry-Bypass (CBP)
  - Linear Carry Select (LCS)
  - Square Root Select (SRCS)
  - Carry Lookahead (CLA)



Introduzione

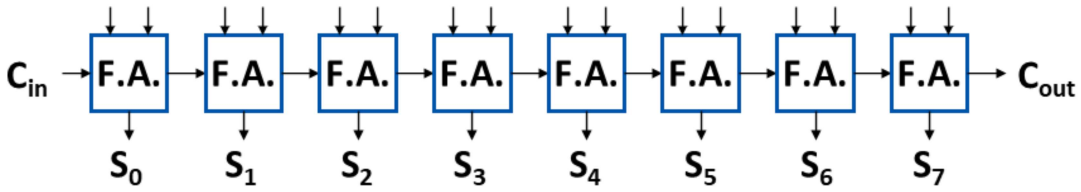
Abbiamo visto che il sommatore è alla base di numerose operazioni logico-aritmetiche: somma, differenza, moltiplicazione, comparazione.
Il sommatore è usato anche per le operazioni in virgola mobile, che si riducono a somme e moltiplicazione di mantissa ed esponente, rappresentati come interi.
Oltre alle operazioni logico aritmetiche il sommatore è impiegato anche in circuiti come timer e contatori, che vedremo in seguito.
Le prestazioni, il consumo e l'occupazione di area di un sommatore sono quindi cruciali per un qualunque sistema digitale.
È naturale voler ottimizzare questa operazione tenendo sotto controllo le tre parole chiave: area, velocità, prestazione.

In generale, la velocità di esecuzione di un'operazione è funzione di due fattori: la tecnologia in cui il circuito viene realizzato e l'algoritmo utilizzato.

Di seguito vengono descritte diverse architetture di sommatori, partendo dalla struttura ripple-carry per proseguire poi con strutture più complesse in grado di ottimizzare le prestazioni in termini di velocità, ma spesso a discapito di area e consumo.
Per ognuno di questi sommatori verrà indicato il percorso critico con il rispettivo tempo di ritardo. Per capire lo scotto da pagare in termini di area e consumo, per ciascuna architettura che studieremo faremo un esempio implementativo stimando il numero di MOS richiesto e il valore complessivo di capacità logica (prodotto tra capacità e fattore di attività) del sistema.

Gli esempi che faremo, come sempre, sono solo a scopo dimostrativo, utili per il confronto tra le diverse architetture.

Esempio: somma di 2 interi a 8bit



Caso peggiore: $T_{CARRY} = 8 \cdot t_C$ $T_{SUM} = 7 \cdot t_C + t_S$

… ma consideriamo un caso specifico

A = 1 1 1 1 1 1 1 1

B = 1 0 0 0 1 0 1 0

LSB MSB

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 3

Esempio.
Consideriamo un sommatore a 8 bit. Calcolare il massimo tempo di propagazione per somma e riporto e i tempi richiesti per sommare i due numeri A = 11111111 e B = 10001010 (LSB a sinistra)

Nel caso peggiore i tempi di ritardo sono:

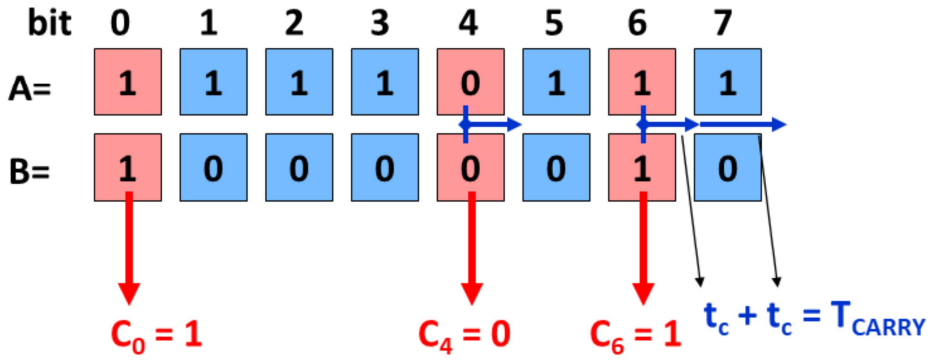
$$T_{SUM} = 7 \cdot t_C + t_S$$
$$T_{CARRY} = 8 \cdot t_C$$

Esempio: somma di 2 interi a 8bit

IMPORTANTE

C_0 e C_6 valgono 1 indipendente dal valore del riporto precedente

C_4 vale 0 indipendente dal valore del riporto precedente



Il riporto C_5 non dipende da nessuno dei riporti antecedenti a C_4

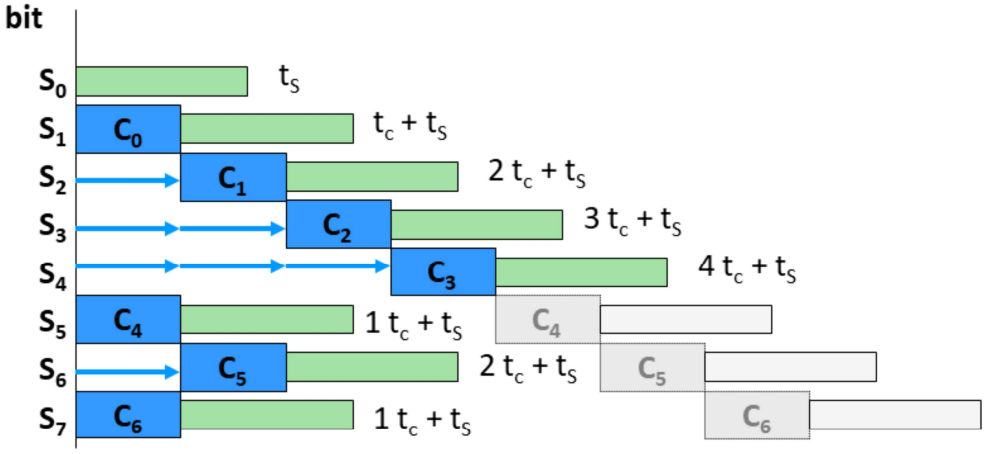
Il riporto C_7 non dipende da nessuno dei riporti antecedenti a C_6

Tuttavia nel caso particolare possiamo osservare che il riporto dei bit 0, 4 e 6 (C_0, C_4 e C_6) sono stabili e costanti fin da subito.
Infatti abbiamo che:
Bit 0: 1+1 + C_{IN} ha riporto 1 indipendentemente dal riporto di ingresso C_{IN}
Bit 4: 0 + 0 + C_3 ha riporto 0 indipendentemente da C_3.
Bit 6: 1 + 1 + C_5 ha riporto 1 indipendentemente da C_5.

Il tempo di calcolo del riporto di uscita sarà solo $2t_C$, poiché C_6 = 1 indipendentemente dai bit precedenti.

Esempio: somma di 2 interi a 8bit

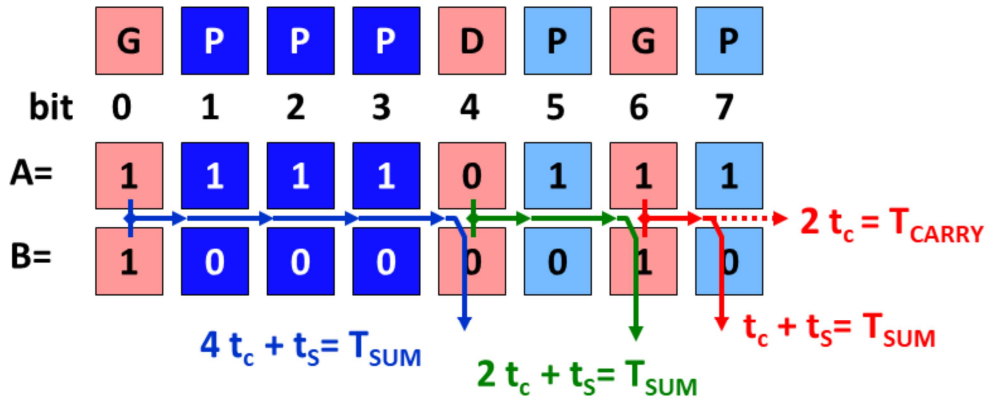
bit	0	1	2	3	4	5	6	7
A=	1	1	1	1	0	1	1	1
B=	1	0	0	0	0	0	1	0



Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 5

Ciò si ripercuote anche nei bit di somma.
Infatti, il bit S_4 di somma richiede il calcolo di C_3, che a sua volta richiede C_2, C_1 e C_0. è quindi necessario aspettare un tempo pari a $4t_C+t_S$.
Il bit S_5, invece, richiede C_4 che è noto subito dopo un tempo t_C. Lo stesso vale per S_7. in entrambi i casi il tempo di ritardo è $t_C + t_S$.

Esempio: somma di 2 interi a 8bit



3 catene di calcolo del riporto che procedono in parallelo

Il tempo di calcolo della somma dipende dalla catena di riporti più lunga
Nel caso peggiore: catena più lunga = (N – 1)

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 6

In generale è possibile identificare tre catene di calcolo del riporto:

- La prima che parte dal bit 0 ed è lunga 4 bit
- La seconda parte dal bit 4 ed è lunga 2 bit
- La terza parte dal bit 6 ed è lunga 2 bit.

Possiamo concludere con questo esempio che il tempo di ritardo per una specifica configurazione di ingresso è dato dalla catena di riporto più lenta.

Il tempo di ritardo di C_{OUT} dipende invece dall'ultima catena di riporti.

Osservazione importante: le posizioni per le quali gli addendi assumono lo stesso valore possano essere considerate come punti in cui è possibile spezzare il sommatore in più tronconi, che, a partire dal momento in cui si presentano i bit di ingresso, operano di fatto in parallelo.

Segnali ausiliari: Propagate & Generate

A_n	B_n	C_{n-1}	S_n	C_n	S_n & C_n	Nome	Condizione
0	0	0	0	0	$S_n = C_{n-1}$ $C_n = 0$	DELETE, CLEAR o GENERATE-0	A = B = 0 $D = \overline{A} \cdot \overline{B} = \overline{A + B}$
0	0	1	1	0			
0	1	0	1	0	$S_n = \overline{C_{n-1}}$ $C_n = C_{n-1}$	PROPAGATE	A = 1, B= 0 o A = 0, B = 1 $P = A \oplus B$
0	1	1	0	1			
1	0	0	1	0			
1	0	1	0	1			
1	1	0	0	1	$S_n = C_{n-1}$ $C_n = 1$	SET o GENERATE-1	A = B = 1 $G = A \cdot B$
1	1	1	1	1			

P, G e D dipendono solo dai bit di ingresso e NON dal riporto.

	P	G	D
$A = B = 0$	0	0	1
$A = B = 1$	0	1	0
$A \neq B$	1	0	0

Per identificare tre condizioni, bastano due segnali:

Se P = 1 allora G = D = 0 sempre → **condizione di propagazione**

Se P = 0 allora uno e solo uno tra G e D è uno → **condizione di generazione**

G = 0 → generazione di riporto 0

G = 1 → generazione di riporto 1

Usiamo solo P e G come segnali ausiliari per prevedere il riporto

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 7

Segnali ausiliari: generazione e propagazione del riporto

Al fine di comprendere quali siano i principi di calcolo del riporto osserviamo il comportamento di un FA che opera sui generici bit di ingresso A_n, B_n e C_{n-1} e genera le uscite S_n e C_{n-1}. Dalla tabella di verità distinguiamo tre casi:

1. Se A = B = 0 allora: $S_n = \overline{C_{n-1}}$ e $C_n = 0$
2. Se A≠B allora: $S_n = \overline{C_{n-1}}$ e $C_n = C_{n-1}$
3. Se A = B = 1 allora: $S_n = C_{n-1}$ e $C_n = 1$

Mentre, per il calcolo di S_n è sempre necessario conoscere il riporto precedente (C_{n-1}), per il calcolo di C_n è possibile, in alcuni casi, conoscerne il valore senza attendere il valore di C_{n-1}. Nel caso 1 C_n = 0 indipendentemente da C_{n-1}. Ciò si veridica se vale la condizione $D = \overline{A} \cdot \overline{B} = \overline{A + B}$. Questa condizione viene detta DELETE, CLEAR o GENERATE-0, per indicare che indipendentemente dal valore di C_{n-1}, C_n viene posto a 0. Il caso 3 è analogo e si verifica se è vera la condizione D = A·B e viene detta SET o GENERATE-1.

Invece nel caso 2 il FA dovrà attendere che quelli precedenti abbiano finito il calcolo del riporto prima di produrre il risultato corretto. Ciò accade se vale la condizione P=A⊕B=1 ed è identificata come PROPAGATE, perché il riporto viene propagato dall'ingresso all'uscita. I segnali D, G e P dipendono SOLO da A_n e B_n e possono essere calcolati senza aspettare i riporti precedenti.

Notiamo inoltre che per qualunque combinazione degli ingressi A e B uno e uno solo tra i segnali D, G e P è a 1. Possiamo quindi identificare i tre casi elencati usando solo due d essi:

- Se P = 1, allora automaticamente G e D sono a 0. Questa situazione rappresenta il caso 2, in cui è necessario attendere il calore C_{n-1}.
- Se P = 0 allora vale sempre la relazione $D = \overline{G}$. Usiamo G per discriminare tra i casi 1 e 3. Se G = 1 (D = 0) si ricade nel caso 3. Se G=0 (D = 1) si ricade nel caso 1. In entrambi i casi il calcolo di C_n non richiede il valore di C_{n-1}. Identifichiamo in modo unificato come GENERATE, cioè generazione di un riporto noto. Questo permette a tutti i FA successivi di iniziare in anticipo il calcolo di somma e riporto.

Obiettivo

- Ottimizzare il sommatore.
- Agiremo su due fronti:
  - A livello di FA: ridurre il percorso da C_{IN} a C_{OUT}
  - A livello di architettura: sfruttare i segnali P e G per "accorciare" la lunghezza dei percorsi di propagazione del riporto
- Confronteremo le diverse architetture in termini di:
  - Velocità (tempo di calcolo)
  - Consumo (capacità logica totale)(*)
  - Area occupata (numero di MOS richiesti)

(*) Ricordiamo che il consumo di potenza di un blocco è dato dalla somma del consumo di tutti i suoi nodi:

$$\sum C_n V_{DD}^2 f \alpha_n = V_{DD}^2 f \sum C_n \alpha_n$$

Il consumo è proporzionale alla somma delle capacità logiche

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 8

Ottimizzazione del sommatore

Per ottimizzazione di un sommatore si intende modificarne l'implementazione in modo da rendere il tempo di ritardo più basso possibile. La semplice architettura ripple-carry è molto semplice, ma ha un tempo di ritardo che dipende linearmente dal numero di bit. L'obiettivo è arrivare a un tempo di ritardo che dipenda da logN. Ciò può essere fatto, ma comporta un compromesso in termini di consumo e occupazione di area.
L'ottimizzazione del sommatore avviene in due fronti:

1. Il primo passo è l'ottimizzazione del FA.
2. Il secondo passo è la modifica architetturale, ovvero la connessione dei blocchi che lo compongono.

In seguito metteremo a confronto tre tipi di architetture stimando per ciascuna:

- Il tempo di ritardo di somma e riporto.
- Il numero di MOS, che è proporzionale all'area occupata
- Il consumo. Poiché il consumo dipende dalla frequenza di funzionamento e dalla tensione di alimentazione, useremo la capacità logica per confrontare le diverse architetture, dato che questo paramento dipende solo dalle capacità dei nodi (e quindi dalla topologia del circuito) e dal fattore di attività (ovvero dalla funzione logica svolta). Si intuisce facilmente che il consumo di due circuiti a parità di alimentazione e frequenza hanno consumi proporzionali alla somma di tutte le capacità effettivamente caricata e scaricata ad ogni operazione:

$$P_{dyn} = \sum C_n V_{DD}^2 f \alpha_n = V_{DD}^2 f \sum C_n \alpha_n$$

C_n e α_n indicano la capacità e il fattore di attività del nodo n-esimo. La sommatoria va estesa a tutti i nodo del circuito. Poiché ciascun transistor porta con se ter capacità parassite (una per nodo), si intuisce che un maggior numero di MOS implica un aumento della capacità logica.

Ottimizzazione logica del full-adder

Riscriviamo le funzioni logiche S e C_{OUT} in funzione dei segnali di propagate e generate

Propagate: $P = A \oplus B$ $\quad C_{OUT} = A \cdot B + (A \oplus B) \cdot C_{IN}$

Generate: $G = A \cdot B$ $\quad S = A \oplus B \oplus C_{IN}$

$$C_{OUT} = G + P \cdot C_{IN} \qquad S = P \oplus C_{IN}$$

C_{OUT} = 1 se:

- viene generato un riporto 1, oppure
- il riporto di ingresso è 1 ed è propagato verso l'uscita

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 9

Ottimizzazione del FA

Abbiamo già visto che l'operazione logica svolta dal full adder è la seguente:

$$S = A \oplus B \oplus C_{IN}$$
$$C_{OUT} = A \cdot B + (A \oplus B) \cdot C_{IN}$$

Le tre condizioni di PROPAGATE e GENERATE sono identificate dai seguenti segnali:

PROPAGATE: P = A⊕B
GENERATE: G = A·B

E dipendono solo dai segnali A e B (non dal riporto)

Possiamo riscrive somma e riporto di uscita nel modo seguente:

$$S = P \oplus C_{IN}$$
$$C_{OUT} = G + P \cdot C_{IN}$$

L'ultima relazione è interpretata a livello logico nel modo seguente: Il riporto in uscita vale 1 se viene generato (A=B=1) oppure se viene propagato (P=1) e il riporto di ingresso è 1.

Ottimizzazione della struttura logica del full-adder

$P = A \oplus B$

$G = A \cdot B$

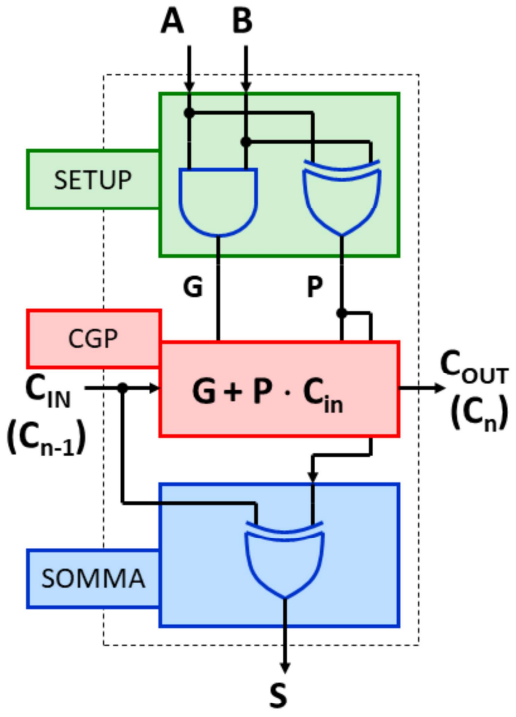
Setup: genera i segnali ausiliari
Dipende solo da A e B
(non serve attendere C_{IN})
Tutti i bit possono procedere in parallelo

$C_{OUT} = G + P \cdot C_{IN}$

Blocco CGP (Carry Generation and Propagation)
Genera o propaga il riporto. Deve attendere il riporto precedente. È critico perché COUT è l'ingresso del blocco successivo

$S = P \oplus C_{IN}$

Somma: *Da P e C_{IN} calcola la somma. Deve attendere il riporto precedente. Non è critico perché S non è un ingresso per il bit successivo*



Struttura di un FA ottimizzato

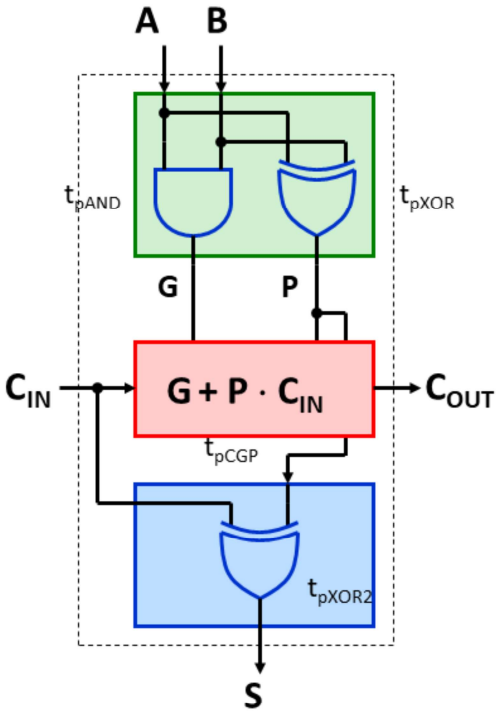
Alla luce di queste osservazioni è possibile adottare la struttura a blocchi del FA illustrata sopra.

- Il blocco in verde è definito blocco di SETUP e genera i segnali ausiliari P e G a partire dai soli bit A e B.
- Il blocco rosso calcola il riporto sulla base di P, G ed eventualmente C_{n-1} (se vale la condizione PROPAGATE). Chiamiamo questo blocco con l'acronimo CGP *(Carry Generation and Propagation)*
- Il blocco blu genera la somma a partire da C_{n-1} e da P.

Tenendo conto che il ritardo deriva dalla propagazione del riporto, l'obiettivo è realizzare il blocco centrale (rosso) più veloce possibile.

Prestazioni del FA ottimizzato

- Definiamo:
  - t_{pAND} e t_{pXOR} i tempi di calcolo di G e P
  - $t_{SETUP} = \max(t_{pAND}, t_{pXOR})$ il tempo di ritardo del blocco di setup
  - t_{pCGP} il tempo di ritardo della funzione logica del blocco CGP
  - t_{pXOR2} il tempo di ritardo della porta XOR che genera la somma
- Se gli ingressi A, B, C_{IN} arrivassero contemporaneamente i tempi di ritardo di S e C_{OUT} sarebbero:
  - $t_S = t_{pXOR} + t_{pXOR2}$
  - $t_C = t_{SETUP} + t_{pCGP}$
- Ma essendo C_{IN} l'ingresso critico (ultimo ad arrivare), G e P sarebbero già pronti:
  - $t_S = t_{pXOR2}$
  - $t_C = t_{pCGP}$



Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 11

Prestazioni del FA ottimizzato

Il blocco di setup svolge le funzioni AND e XOR degli ingressi A e B. Il tempo di ritardo del blocco di setup è determinato dalla porta logica più lenta:

$$t_{SETUP} = \max(t_{pAND}, t_{pXOR})$$

Poiché entrambi i segnali sono indispensabili per il calcolo del riporto, è opportuno che le porte siano ottimizzate per avere lo stesso tempo di ritardo. È inutile, infatti, ottimizzarne una sola a discapito dell'altra.

Il blocco di generazione del riporto (CGP) svolge una funzione logica a 3 ingressi molto semplice e può essere implementata in logica statica complementare, così come a pass-T/TG.

Il blocco di somma è la funzione XOR tra C_{IN} e P e il suo tempo di ritardo sarà quello di una semplice porta logica.

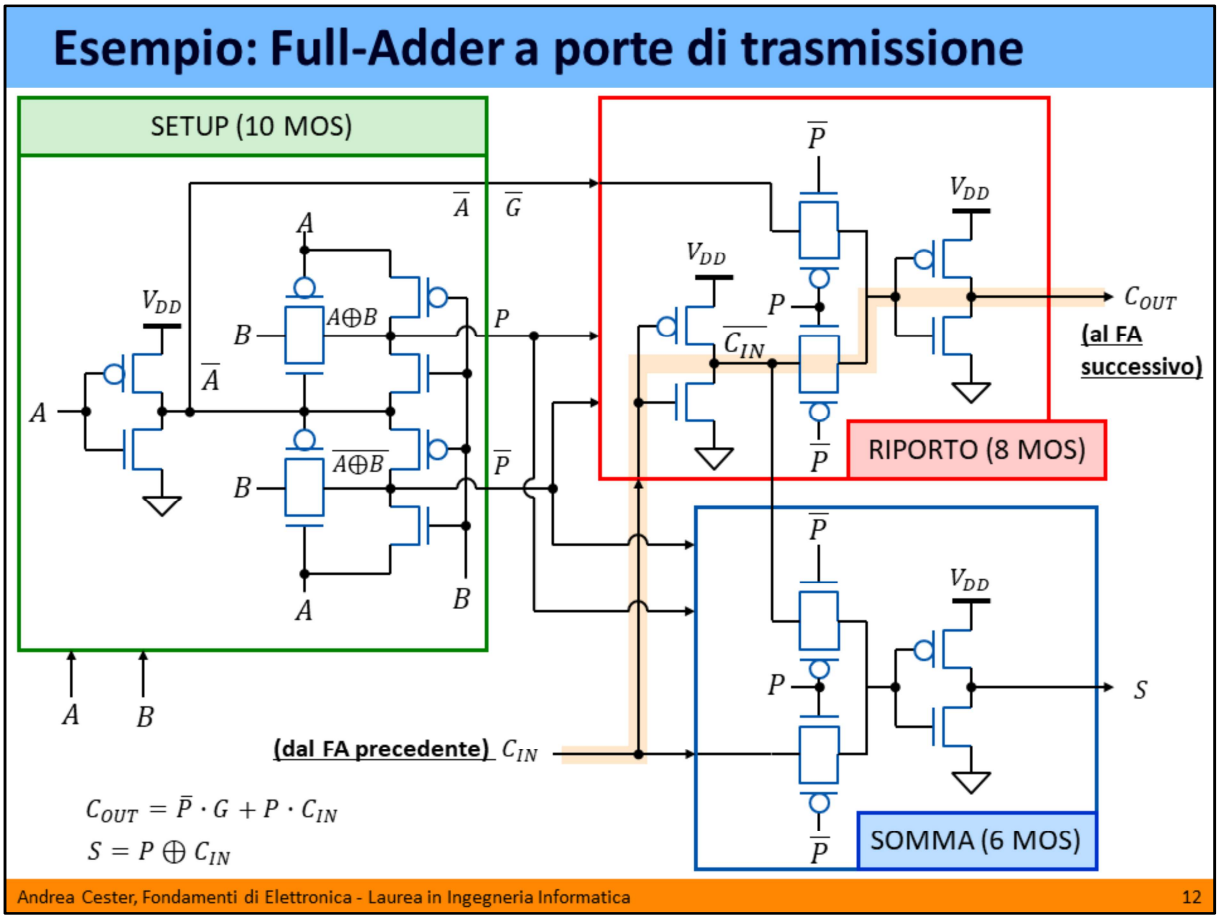
Assumendo che gli ingressi A, B e C_{IN} arrivino tutti contemporaneamente, il tempo di ritardo di somma e riporto sono rispettivamente:

$$t_S = t_{pXOR} + t_{pXOR}$$
$$t_C = t_{SETUP} + t_{pCGP}$$

Tuttavia, se assumiamo che gli ingressi A e B siano arrivati prima del riporto C_{IN}, allora il tempo di ritardo di somma e riporto risulta:

$$t_S = t_{pXOR2}$$
$$t_C = t_{pCGP}$$

Assumendo che ciascuna porta logica abbia lo stesso tempo di ritardo, t_S e t_C risultano dimezzati.



Esempio: FA a porte di trasmissione

Il FA a porte di trasmissione soddisfa questa struttura se definiamo G = A.

Blocco di SETUP

Il blocco di SETUP è costituito dalla coppia di funzioni XOR/XNOR che fornisce P in forma diretta e negata e il segnale G in forma negata che viene ricavato da A.

Blocco CGP

Il riporto è calcolato come:

$$C_{OUT} = \bar{P} \cdot G + P \cdot C_{IN}$$

dove:

$$\bar{P} \cdot G = \bar{A} \cdot \bar{B} \cdot A + A \cdot B \cdot A = A \cdot B$$

Che identifica la definizione originale di GENERATE.

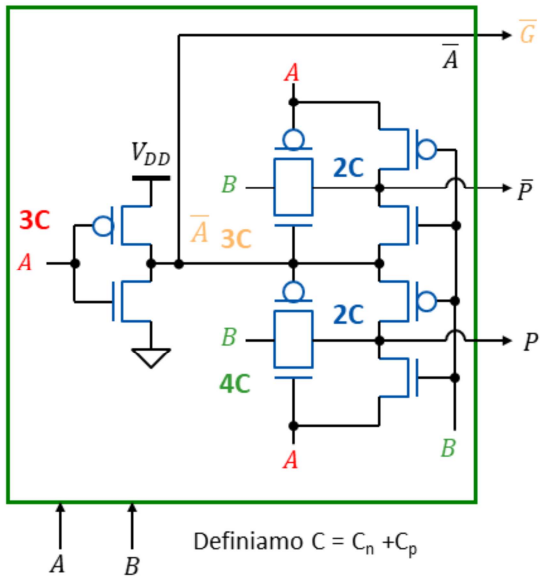
Il motivo per il quale si è deciso di porre G = A nel blocco di SETUP è dovuto alla particolare struttura a MUX del blocco GCP. Infatti, il segnale G passa in uscita solo se P = 0. Essendo P = A⊕B, si evince che P è 0 se e solo se A=B. Poiché G è usato solo dal blocco CGP è inutile calcolare A·B, introducendo ulteriore ritardo e ulteriori MOS inutilmente.

Blocco di somma

La somma viene calcolata dal terzo blocco (XOR):

$$S = P \oplus C_{IN}$$

Esempio: consumo dinamico del FA a TG: setup



Ipotesi:
R_n=R_p=R
C_{gn}=C_{dn}=C_{sn} = C_n C_{gp}=C_{dp}=C_{sp}= C_p

Calcoliamo il consumo dinamico relativo a tutti i nodo del circuito (inclusi gli ingressi).
Assumiamo ragionevolmente che:
p_A = p_B = p_{CIN} = 0.5

Nodo	capacità	segnale	p[%]	α
A	3C	A	0.5	0.25
B	4C	B	0.5	0.25
$\overline{G}$	3C	$\overline{A}$	0.5	0.25
P	2C	$A \oplus B$	0.5	0.25
$\overline{P}$	2C	$\overline{A \oplus B}$	0.5	0.25

$$P_{dyn}^{SETUP} = \sum C_n \cdot V_{DD}^2 f \cdot \alpha_n = V_{DD}^2 f \cdot \frac{14C}{4}$$

Equivale a caricare e scaricare a frequenza f una capacità logica di 14C/4 (C = capacità di un terminale di un NMOS + un PMOS)

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 13

Esempio: consumo dinamico del FA a porte di trasmissione

A titolo di esempio vogliamo calcolare la capacità logica complessiva del FA a porte di trasmissione per usarla nei successivi esempi come esempio di comparazione per le diverse architetture.
Adottando le seguenti ipotesi semplificative:

- NMOS e PMOS sono dimensionati in modo da avere la stessa resistenza. Assumiamo $Z_p = 2Z_n$.
- Supponiamo che ciascun terminale di un NMOS abbia capacità $C_{gn} = C_{sn} = C_{dn} = C_n$ e che ciascun terminale di un PMOS abbia capacità $C_{gp} = C_{sp} = C_{dp} = C_n$.
- Tutti i MOS hanno la stessa lunghezza di canale L, quindi $W_p = 2W_n$ e $C_p = 2C_n$.
- Gli ingressi primari A_n e B_n sono equiprobabili, cioè hanno probabilità di essere a 1 pari a 0.5.

Calcoliamo la capacità logica di ciascun blocco. In ciascun blocco consideriamo le capacità di tutti i MOS che lo compongono e solo degli ingressi che lo compongono. Tra le capacità da caricare e scaricare consideriamo anche le capacità dei nodo di ingresso(*).

Blocco di setup

La tabella elenca i cinque nodi del circuito: $A, B, \overline{G}, P, \overline{P}$. Nella seconda colonna è mostrata la *capacità fisica*. Ad esempio l'ingresso A è collegato a due gate di NMOS, 2 drain di NMOS e a due gate di PMOS e ha capacità: $3C_n + 3C_p = 3C$ (definiamo C = C_n + C_p).
Il nodo B è collegato a 4 terminali di NMOS e 4 di PMOS e ha capacità 4C e così via per gli altri nodi.

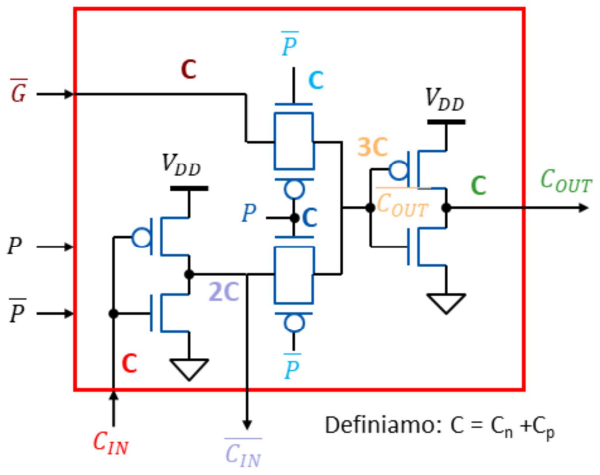
(*) Normalmente nel calcolo del consumo dinamico delle porte logiche non abbiamo mai considerato i nodi di ingresso perché ciò che calcolavamo era l'energia dissipata nei MOS della porta logica studiata. La capacità del nodo di ingresso veniva invece caricata/scaricata dallo stadio precedente. In questo caso dobbiamo tenere conto anche degli ingressi, perché ciascun FA è pilotato da un FA. L'ingresso del secondo è il carico esterno del primo.

Esempio: consumo dinamico del FA a TG: setup

Ipotesi:
$R_n=R_p=R$
$C_{gn}=C_{dn}=C_{sn}=C_n$
$C_{gp}=C_{dp}=C_{sp}=C_p=2C_n$

Calcoliamo il consumo dinamico relativo a tutti i nodo del circuito (inclusi gli ingressi).
Assumiamo ragionevolmente che:
$p_A = p_B = p_{CIN} = 0.5$

Nodo	capacità	segnale	p[%]	α
$\overline{G}$	C	$\overline{A}$	0.5	0.25
P	C	$A \oplus B$	0.5	0.25
$\overline{P}$	C	$\overline{A \oplus B}$	0.5	0.25
C_{IN}	C	C_{IN}	0.5	0.25
$\overline{C_{IN}}$	2C	$\overline{C_{IN}}$	0.5	0.25
$\overline{C_{OUT}}$	3C	$\overline{C_{OUT}}$	0.5	0.25
C_{OUT}	C	C_{OUT}	0.5	0.25



$$P_{dyn}^{CGP} = \sum C_n \cdot V_{DD}^2 f \cdot \alpha_n = V_{DD}^2 f \cdot \frac{10C}{4}$$

Equivale a caricare e scaricare a frequenza f una capacità logica di 10C/4
(C = capacità di un terminale di un NMOS e un PMOS)

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 14

La terza la colonna rappresenta la funzione logica svolta da ciascun nodo (per gli ingressi primari la funzione logica è l'identità). La quarta e la quinta colonna riportano la probabilità che il nodo sia a 1 e il corrispondente fattore di attività. Ovviamente A, B *e* $\overline{G}$ sono a 1 con probabilità 0.5 e hanno fattore di attività 0.25. P *e* $\overline{P}$ svolgono la funzione XOR e XNOR e sono a 1 due casi su quattro. Avendo assunto equiprobabili gli ingressi A e B anche P *e* $\overline{P}$ sono a 1 con probabilità 0.5.
Il consumo dinamico vale:

$$P_{dyn}^{SETUP} = \sum C_i V_{DD}^2 f \alpha_i = V_{DD}^2 f \left(3C\frac{1}{4} + 4C\frac{1}{4} + 3C\frac{1}{4} + 2C\frac{1}{4} + 2C\frac{1}{4}\right) = V_{DD}^2 f \frac{14C}{4}$$

La capacità logica è 14C/4.

Blocco CGP

Il blocco di generazione e propagazione del riporto ha come ingressi P, $\overline{P}$, $\overline{G}$ (dal blocco di SETUP), e C_{IN} (ingresso primario dal FA precedente). Produce due uscite: C_{OUT} in uscita dal FA e $\overline{C_{IN}}$ che entra come ingresso al blocco di somma per realizzare la XOR. Ha inoltre un nodo interno (uscita dei TG, indicato in figura come $\overline{C_{OUT}}$).
La tabella elenca le capacità di questi nodi, le funzioni svolte, la probabilità che siano a 1 e il fattore di attività, usando le stesse notazioni del blocco di setup.
Osserviamo che tutti gli ingressi sono equiprobabili e che la probabilità che C_{OUT} sia a 1 è:

$$p_{C_{OUT}} = p_P \cdot p_{C_{IN}} + (1 - p_P) \cdot (1 - p_G) = 0.5$$

Il consumo dinamico vale:

$$P_{dyn}^{CGP} = \sum C_i V_{DD}^2 f \alpha_i = V_{DD}^2 f \left(C\frac{1}{4} + C\frac{1}{4} + C\frac{1}{4} + C\frac{1}{4} + 2C\frac{1}{4} + +3C\frac{1}{4} + +C\frac{1}{4}\right) = V_{DD}^2 f \frac{10C}{4}$$

La capacità logica è 10C/4.

Esempio: Full-Adder a porte di trasmissione

Nodo	capacità	segnale	p[%]	α
P	C	$A \oplus B$	0.5	0.25
$\overline{P}$	C	$\overline{A \oplus B}$	0.5	0.25
C_{IN}	C	C_{IN}	0.5	0.25
$\overline{C_{IN}}$	C	$\overline{C_{IN}}$	0.5	0.25
$\overline{S}$	3C	$\overline{A \oplus B \oplus C_{IN}}$	0.5	0.25
S	C	$A \oplus B \oplus C_{IN}$	0.5	0.25

Ipotesi:
$R_n=R_p=R$
$C_{gn}=C_{dn}=C_{sn} = C_n$ $C_{gp}=C_{dp}=C_{sp}= C_p = 2C_n$

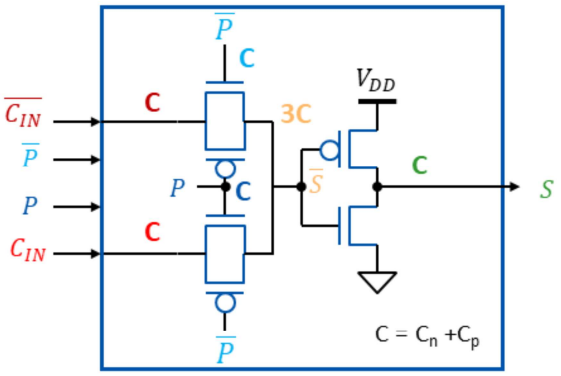
Calcoliamo il consumo dinamico relativo a tutti i nodo del circuito (inclusi gli ingressi).
Assumiamo ragionevolmente che:
$p_A = p_B = p_{CIN} = 0.5$

$$P_{dyn}^{SOMMA} = \sum C_n \cdot V_{DD}^2 f \cdot \alpha_n = V_{DD}^2 f \cdot \frac{8C}{4}$$

Equivale a caricare e scaricare a frequenza f una capacità logica di 8C/4 (C = capacità di un terminale di un NMOS e un PMOS)

Capacità logica complessiva:
($C = C_n + C_p$)

$SETUP \rightarrow 3.5$
$CGP \rightarrow 2.5C$
$SOMMA \rightarrow 2C$
$\boldsymbol{TOTALE \rightarrow 8C}$



Blocco di somma

Il blocco di somma riceve in ingresso $P, \bar{P}$, $C_{IN}, \overline{C_{IN}}$ (tutti equiprobabili), ha un'uscita (S) e un nodo interno che indichiamo con $\bar{S}$.

S risulta la XOR di P e C_{IN} ed è anch'esso un segnale equiprobabile.

In tabella vediamo la consueta sintesi di capacità, probabilità e fattore di attività.

La capacità logica è:

$$C\frac{1}{4} + C\frac{1}{4} + C\frac{1}{4} + C\frac{1}{4} + 3C\frac{1}{4} + C\frac{1}{4} = \frac{8C}{4}$$

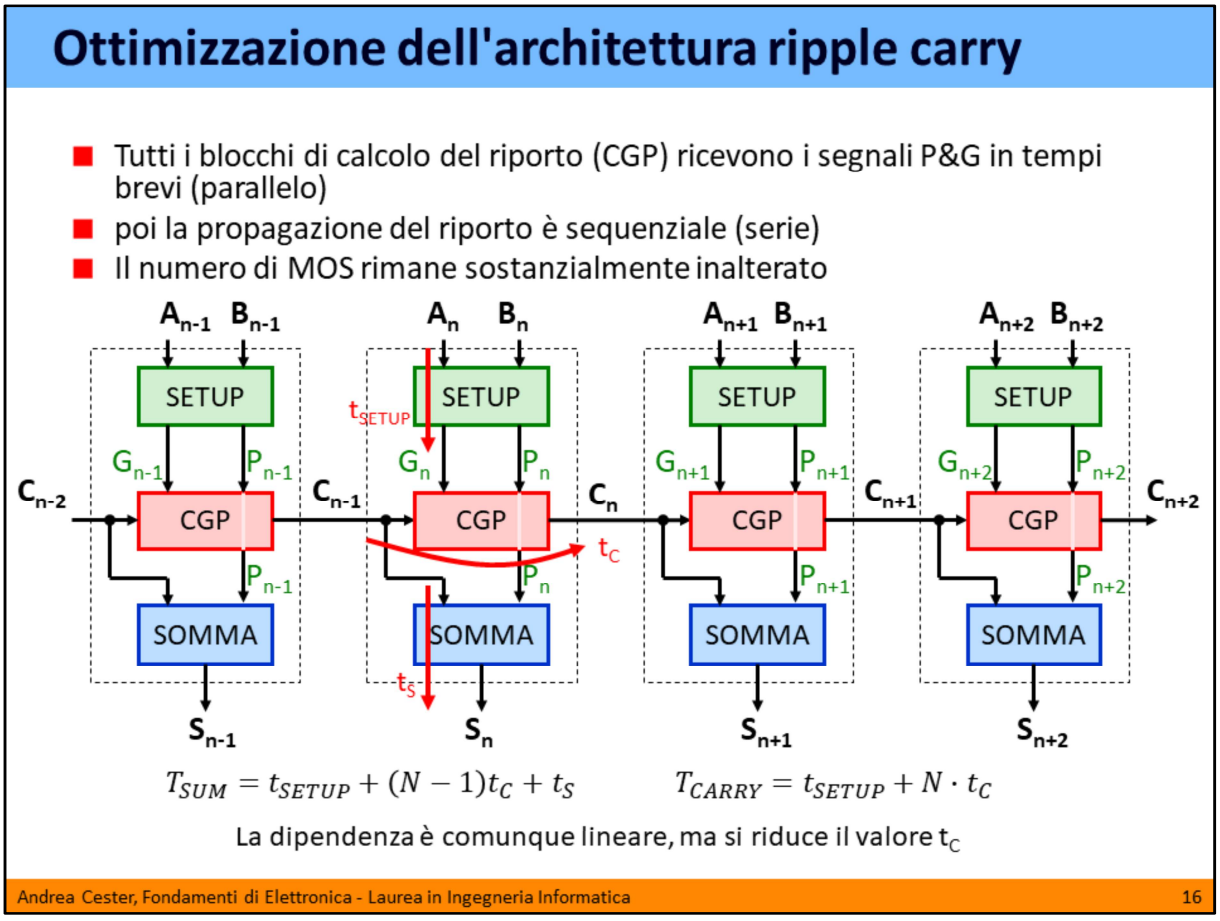
<u>Capacità logica complessiva</u>

In totale un FA a TG con il dimensionamento usato per i MOS ha tre blocchi con capacità logiche:

$$\frac{14C}{4} = 3.5C \qquad setup$$
$$\frac{10C}{4} = 2.5C \qquad CGP\ (riporto)$$
$$\frac{8C}{4} = 2C \qquad somma$$

La capacità logica totale è 8C.

Osserviamo che la somma di tutte le capacità fisiche del circuito è 32C, ma che essendo scaricate e caricate con una frequenza effettiva f/4, il loro contributo ai fini del consumo è solo 8C.



Sommatore ripple-carry ottimizzato

Mettendo insieme i blocchi FA con struttura ottimizzata in una semplice architettura ripple carry si vede facilmente che l'elaborazione procede in due fasi:

1) Prima fase in parallelo (blocchi di SETUP, indipendenti dal riporto)
2) Seconda fase in sequenza (catena del riporto)

Tempo di ritardo

Il tempo di ritardo è la somma delle due fasi. La prima fase si esaurisce in un tempo t_{SETUP} per tutti i bit, procedendo in parallelo. Quella successiva impiega:

$$\begin{matrix} Nt_C & riporto\ finale \\ (N-1)t_C + t_S & somma \end{matrix}$$

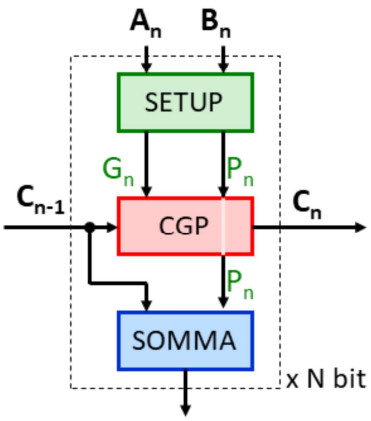
dove con t_C e t_S indichiamo da ora in poi i tempi necessari a calcolare il riporto di un singolo bit (a partire da P_n, G_n e C_{n-1}) e la somma di un singolo bit (a partire da C_{n-1} e P)
In totale abbiamo:

$$\begin{matrix} T_{SU} = t_{SETUP} + Nt_C & riporto\ finale \\ T_{CARRY} = t_{SETU} + (N-1)t_C + t_S & somma \end{matrix}$$

TSUM e TCARRY indicano il tempo di ritardo totale di somma e riporto di uscita
Il tempo di ritardo conserva la dipendenza lineare dal numero dei bit. Il solo miglioramento apportato è la diminuzione del tempo t_C necessario a propagare il riporto, dato che il blocco di SETUP esegue metà del calcolo in parallelo.

Ripple carry: stima del consumo/capacità logica

- Ipotesi semplificative
  - FA a TG
  - Capacità terminale di NMOS e PMOS: $C_n + C_g$
- Capacità logica:
  - SETUP→3.25C
  - CGP→2.5C
  - Somma→2C



Cap. Log.	8 bit	16 bit	32 bit	64 bit	N bit
SETUP	26C	52C	104C	208C	3.5N·C
CGP (FA)	20C	40C	80C	160C	2.5N·C
SOMMA	16C	32C	64C	128C	2N·C
Totale	**62C**	**124C**	**248C**	**486C**	**8N·C**

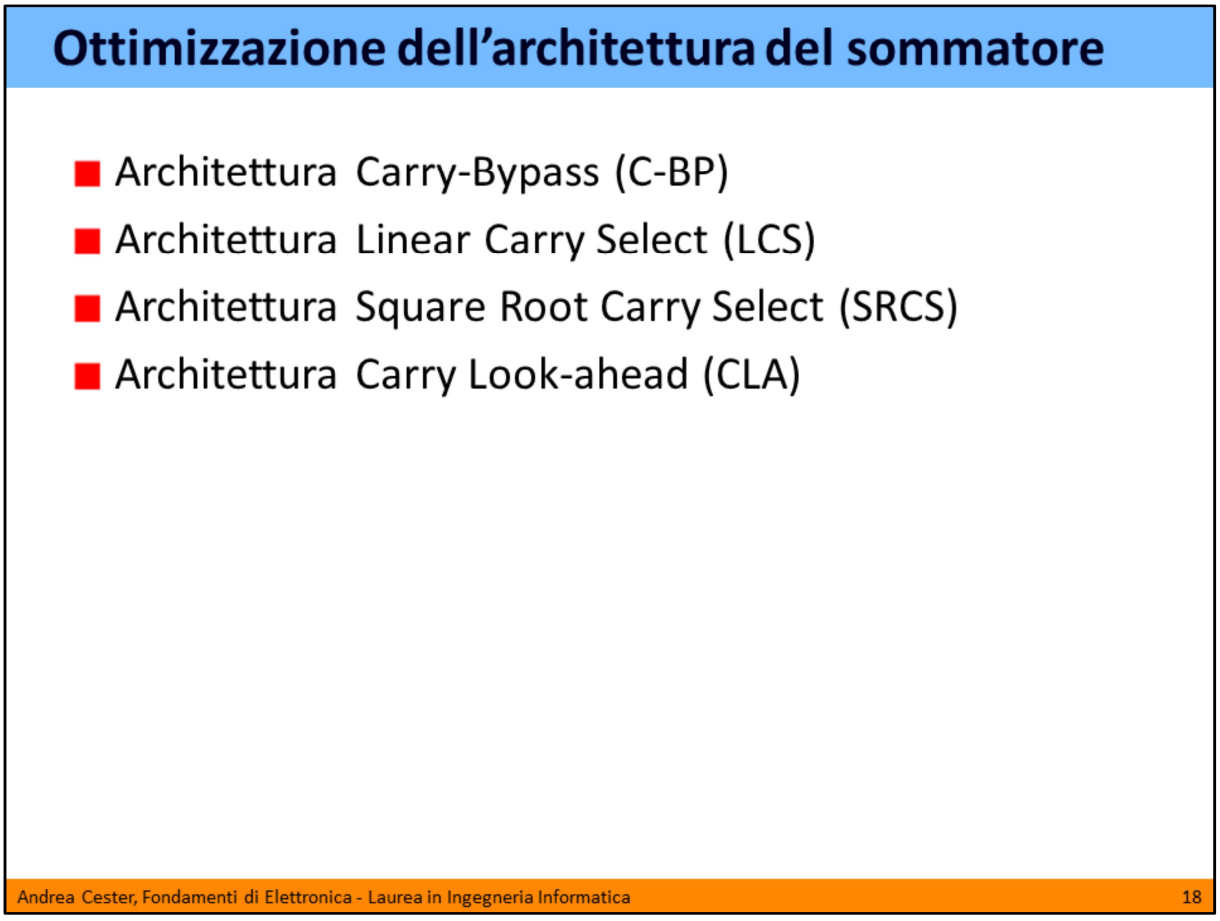
N. MOS	8 bit	16 bit	32 bit	64 bit	N bit
SETUP	80	160	320	640	10N
CGP (FA)	64	128	256	512	8N
SOMMA	48	96	192	384	6N
Totale	**192**	**384**	**768**	**1536**	**8N·C**

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 17

Stima del consumo/capacità logica

In tabella sono mostrati i valori di capacità logica del sommatore ripple-carry appena realizzato con 8, 16, 32, 64 e N bit.
Le prime tre righe riportano le capacità logiche parziali di ciascuno stadio. Osserviamo che:

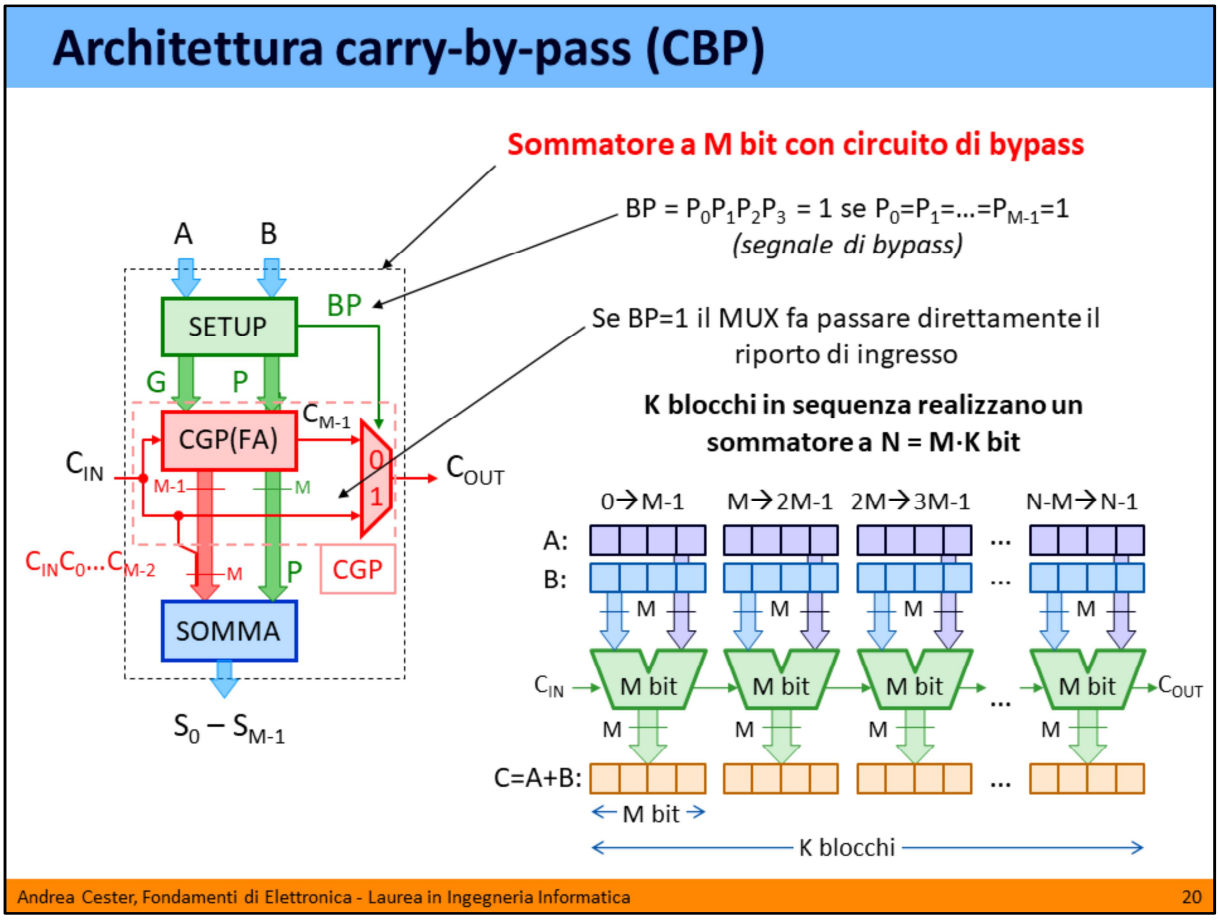
- Il contributo maggiore lo da lo stadio di SETUP con il 44% del consumo.
- Il valore di capacità logica non tiene conto di un eventuale carico collegato all'uscita, ma solo del consumo intrinseco del sommatore.



Il *Ripple Carry*, presenta un limite alla velocità di elaborazione, determinato dal tempo di propagazione del *riporto*.
In seguito vedremo alcune architetture che permettono di diminuire tale ritardo, ottimizzando la propagazione della catena di riporto, sfruttando le condizioni di GENERATE in modo opportuno.
Vedremo, tuttavia, che il miglioramento delle prestazioni avrà come prezzo maggiore occupazione di area e maggior consumo.
Le tecniche principali che permettono di ridurre il tempo necessario al computo della somma sono:

- Architettura Carry-Bypass (CBP)
- Architettura Linear Carry Select (LCS)
- Architettura Square Root Select (SRS)
- Architettura Carry Lookahead (CLA)

ARCHITETTURA CARRY-BY-PASS



Sommatore Carry by-pass (CBP)

L'architettura carry-bypass si basa sulla suddivisione degli N bit da sommare in K gruppi da M bit ciascuno. La somma viene eseguita mediante K sommatori a M bit in cui il riporto di ingresso è connesso al riporto di uscita del blocco precedente, ad eccezione del primo blocco della sequenza che ha in ingresso direttamente C_{IN}. L'ultimo blocco fornisce C_{OUT}.

L'idea alla base del CBP sfrutta il meccanismo di generazione e propagazione del riporto descritto precedentemente per accelerare l'operazione di somma.

Sappiamo che se $A_n \neq B_n$ il riporto si propaga attraverso il FA, che non ne modifica il valore: questo comportamento può essere generalizzato alla cascata di M FA, se tutti hanno come ingressi coppie di bit con valori differenti. Ad esempio, se $P_2 = P_3 = P_4 = 1$ il riporto C_1 si propagherà attraverso tutti tre i bit:

$$C_4 = C_3 = C_2 = C_1$$

Se la condizione $P_2 = P_3 = P_4 = 1$ venisse verificata a priori nel blocco di setup (ricordiamo che dipende solo dai segnali di ingresso), sarebbe possibile collegare direttamente l'uscita C_4 all'ingresso C_1. Questo è esattamente ciò che fa il sommatore CBP.

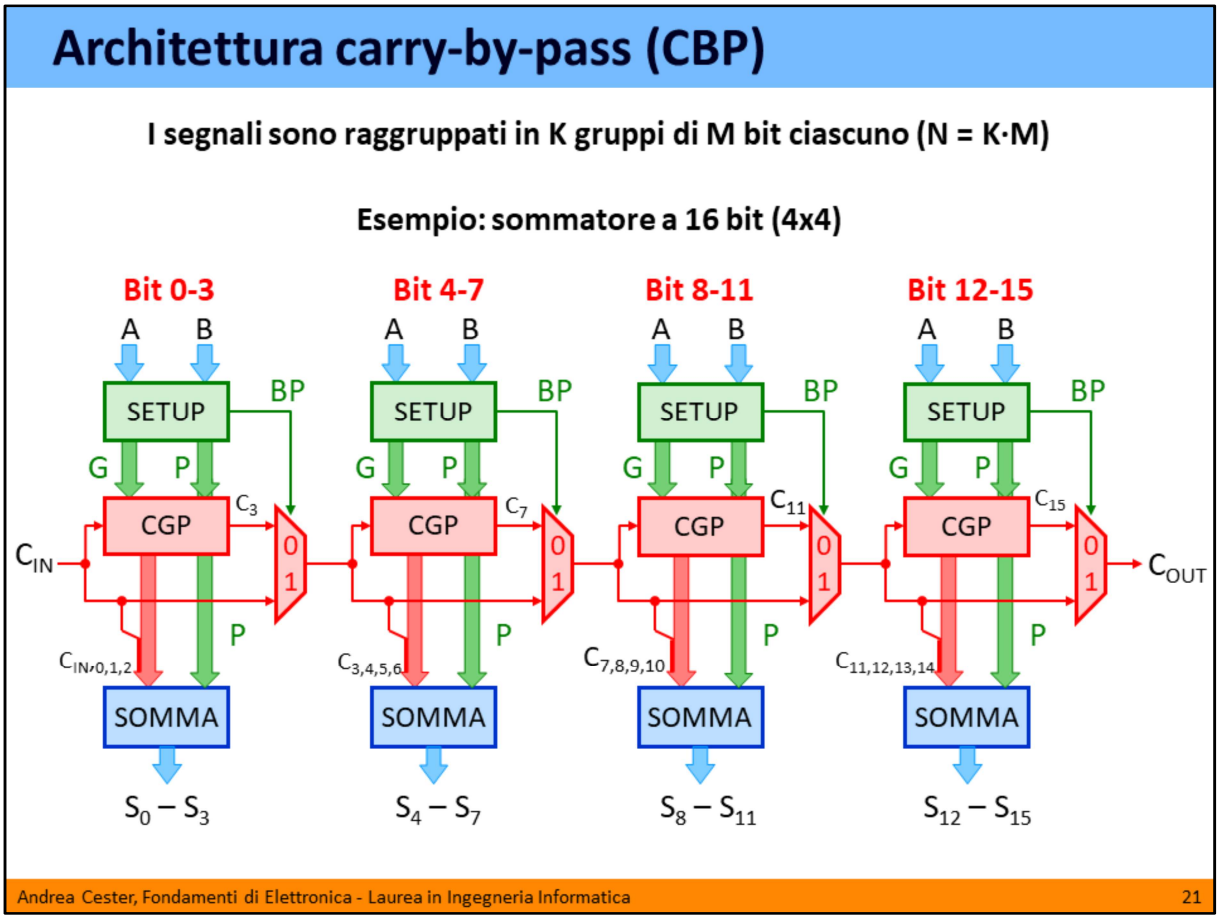
In ciascun blocco di M bit viene aggiunto un circuito in grado di rilevare se tutti i bit da sommare in ingresso al blocco sono diversi a coppie (ovvero se tutti segnali P_n del blocco sono a 1) . Il segnale prodotto da questo circuito viene chiamato Block Propagation o By-Pass (BP).

Il segnale BP non è nient'altro che la AND dei segnali P_n di ciascun bit.

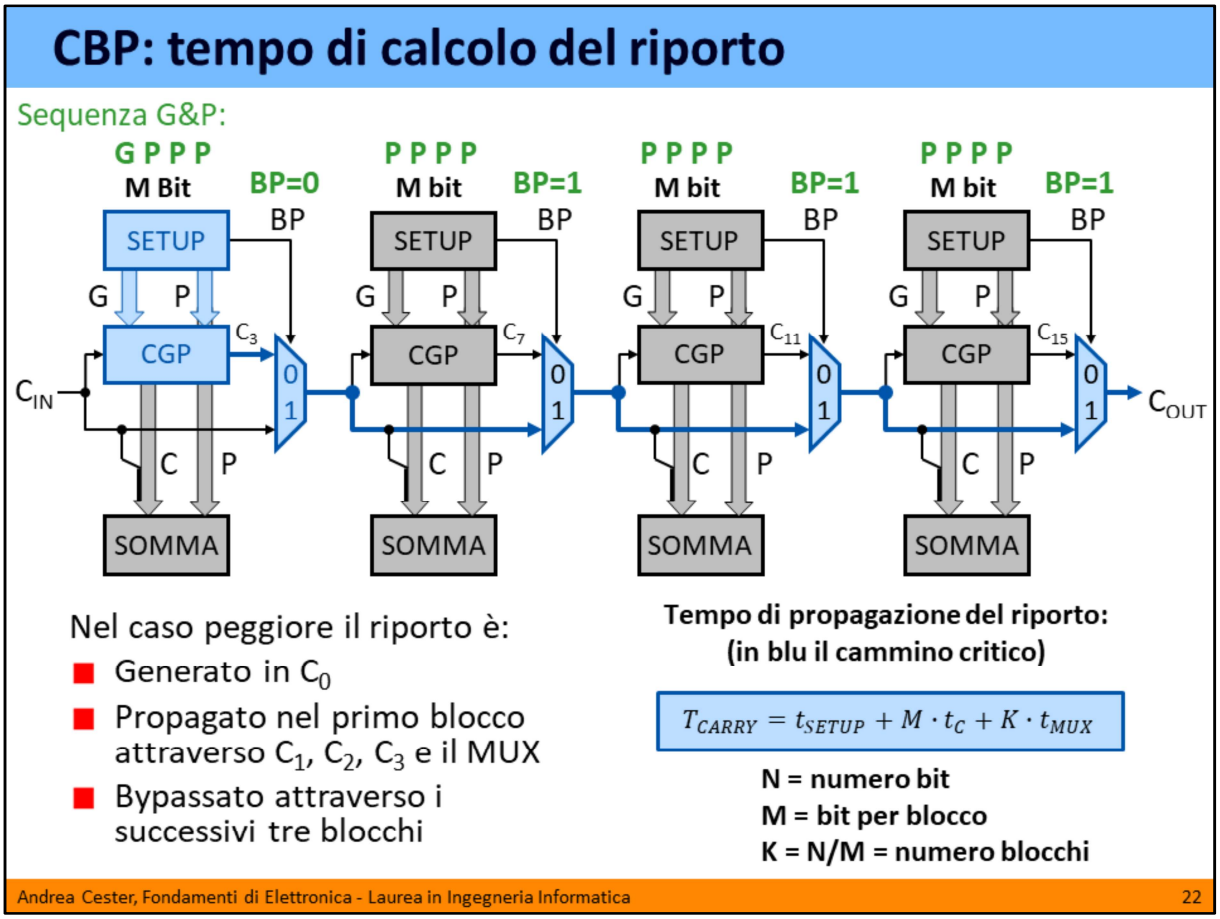
La figura illustra lo schema di un singolo blocco da M bit. Il riporto di uscita viene fornito da un MUX controllato dal segnale BP.

Se BP = 0 significa che almeno un bit è in condizione di generazione (0 o 1). In tal caso il riporto è generato all'interno del blocco è richiede al più $M \cdot t_C$ per essere calcolato. A questo tempo va poi aggiunto il tempo di propagazione del MUX (t_{MUX}).

Se BP = 1 significa che tutti i bit sono in condizioni di propagazione e il riporto di ingresso viene trasmesso direttamente all'uscita in un tempo pari a t_{MUX}.



La figura illustra un CBP che somma numeri a 16 bit suddividendoli in 4 blocchi da 4 bit ciascuno.
Notiamo che il blocco di setup deve generare anche il segnale di BP oltre che i segnali G e P.
I segnali di BP sono tanti quanti i blocchi (4 in questo caso) e richiedono un hardware aggiuntivo che è condiviso da tutti i bit dello stesso blocco.



Tempo di calcolo del riporto

Consideriamo un sommatore CBP suddiviso in K gruppi, ognuno dei quali contenente M celle. Anche in questo caso, come nel caso di un Ripple Carry Adder, è necessario considerare il caso peggiore, cioè quello associato al tempo di ritardo massimo.

Se il riporto fosse generato, ad esempio, sia dal primo bit del GRUPPO 1, sia dal primo bit del GRUPPO 2, si avrà contemporanea propagazione all'interno dei due gruppi di questi riporti.

Entrambi i gruppi calcoleranno in parallelo i riporti C_3 e C_7 che si presenteranno in uscita ai multiplexer nello stesso momento, con un ritardo rispetto agli ingressi pari a:

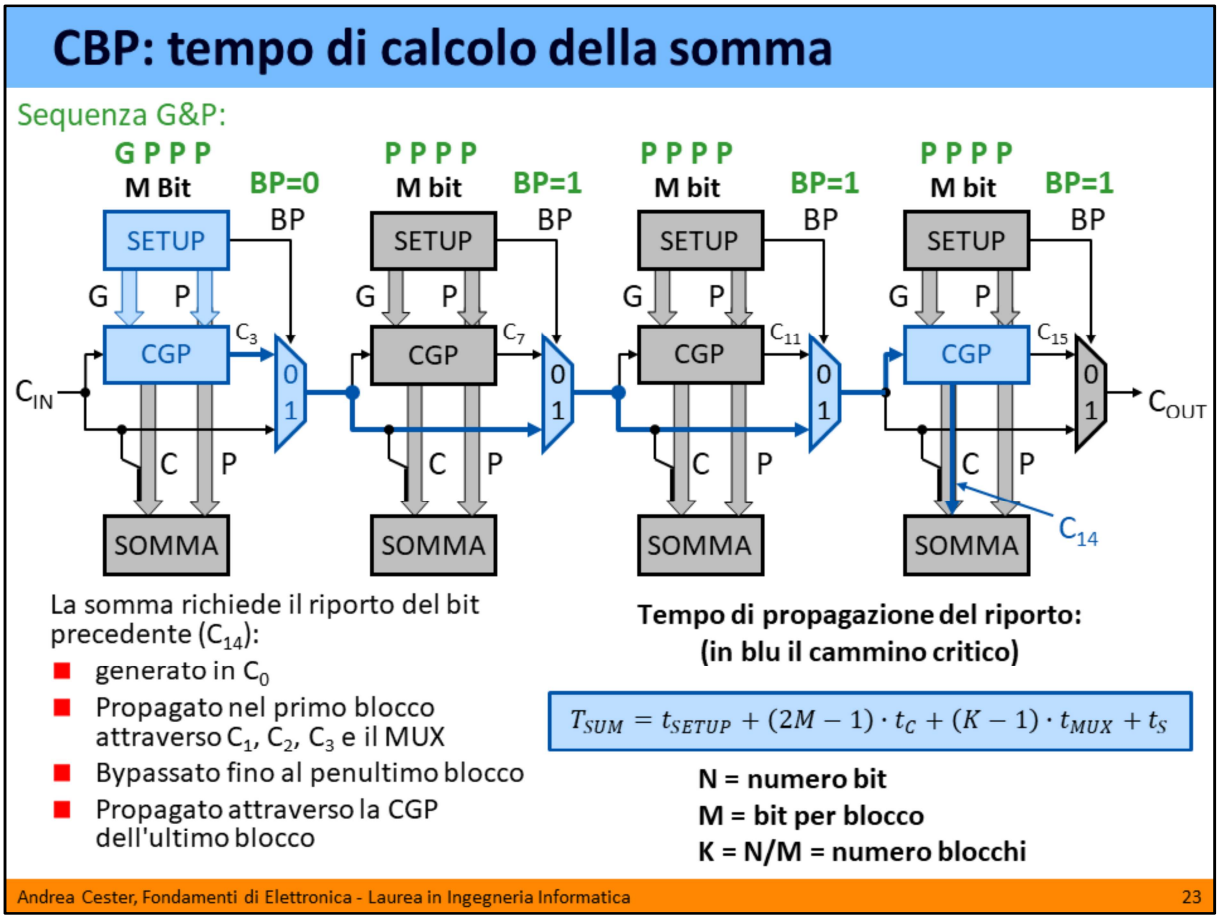
$$t_{SETUP} + M \cdot t_C + t_{MUX}$$

Se invece il riporto viene generato solo dalla prima cella del GRUPPO 1 (in questo caso $BP_1 = 0$ e $BP_2 = 1$), allora il tempo complessivo affinché C_7 sia valido è pari a:

$$t_{SETUP} + M \cdot t_C + 2t_{MUX}$$

Iterando il ragionamento, si deduce che più è lunga la sequenza di gruppi attraverso i quali si ha la propagazione del riporto, più aumenta il ritardo introdotto dal sommatore. Il tempo massimo si ha proprio quando il riporto viene generato dal primo bit del GRUPPO 1, e attraversa il maggior numero di multiplexer. Il caso peggiore di propagazione del riporto è evidenziato in figura, a cui corrisponde un ritardo:

$$T_{CARRY} = t_{SETUP} + M \cdot t_C + K \cdot t_{MUX}$$



Tempo di calcolo della somma

Per il calcolo del tempo di somma è necessario fare un osservazione sul blocco K-esimo. Per il calcolo dell'ultimo bit di somma (S_{N-1}) è necessario il riporto del bit precedente (C_{N-2}). Tale riporto non può essere fornito dal MUX in uscita, ma deve essere necessariamente propagato all'interno dei FA del blocco K-esimo, come evidenziato in figura.
Si ha quindi:

$$T_{SUM} = t_{SETUP} + M\cdot t_C + (K-1)\cdot t_{MUX} + (M-1)\cdot t_C + t_S = t_{SETUP} + (2M-1)\cdot t_C + (K-1)\cdot t_{MUX} + t_S$$

Confrontando i due tempi di ritardo è facile intuire che il tempo critico è quello della somma.

CBP: scelta ottima di M e K

N = numero bit
M = bit per blocco
K = N/M = numero blocchi

$$T_{CARRY} = t_{SETUP} + M \cdot t_C + K \cdot t_{MUX}$$

$$T_{SUM} = t_{SETUP} + (2M-1) \cdot t_C + (K-1) \cdot t_{MUX} + t_S$$ **Tempo critico**

Derivando rispetto a M e uguagliando a 0 si trova:

$$T_{SUM} = t_{SETUP} + (2M-1) \cdot t_C + \left(\frac{N}{M} - 1\right) \cdot t_{MUX} + t_S$$

$$\frac{dT_{SUM}}{dM} = 2t_C - \frac{N}{M^2} t_{MUX} = 0$$

$$M_{OPT} = \sqrt{N \cdot \frac{t_{MUX}}{2t_C}}$$

Se $t_C \cong t_{MUX}$
(come nel FA a TG)

$$M_{OPT} = \sqrt{N/2}$$

N = 8 → M = 2
N = 16 → M = 3 - 4
N = 32 → M = 4
N = 64 → M = 5 - 6

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 24

Scelta del numero di bit per blocco

Il ritardo massimo introdotto dipende sia dalle caratteristiche della tecnologia che dalla suddivisione effettuata.
È necessario a questo punto capire, data la lunghezza N delle parole da sommare, e date le caratteristiche delle porte logiche elementari utilizzate (FA e MUX) in termini di ritardo introdotto, quale sia la partizione migliore, al fine di avere ritardo minimo introdotto.
Essendo t_S e t_{SETUP} costanti la grandezza da minimizzare è:

$$(2M-1)t_C + (K-1)t_{MUX}$$

Fissato il numero M di bit per blocco è possibile calcolare il numero di blocchi:

$$K = \frac{N}{M}$$

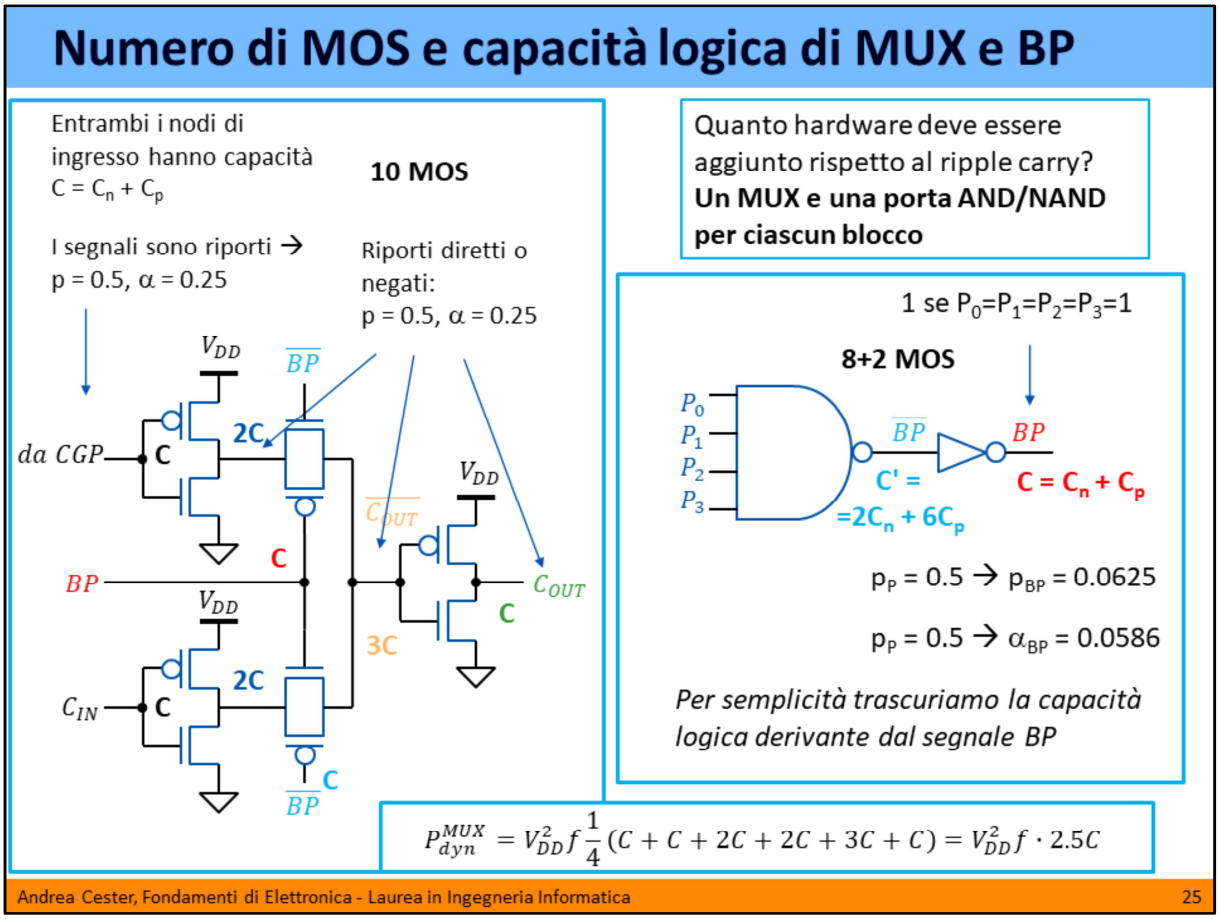
Sostituiamo K, deriviamo rispetto a M e annulliamo la derivata:

$$\frac{\partial}{\partial M}\left[(2M-1)t_C + \frac{N}{M} t_{MUX}\right] = 2t_C - \frac{N}{M^2} t_{MUX} = 0$$

$$M = \sqrt{\frac{N \cdot t_{MUX}}{2t_C}}$$

Questo valore è il valore minimo teorico: poiché *M*, che deve essere necessariamente un numero intero, non potrà assumere (a meno di casi particolari) il valore minimo teorico. Il tempo realmente impiegato dalla struttura sarà leggermente differente da quello ricavato.
Se t_{MUX} e t_C sono circa uguali risulta:

$$t_C \cong \sqrt{\frac{N}{2}}$$



Hardware aggiuntivo per un sommatore CBP

Per ogni blocco è necessario aggiungere un MUX e una porta AND.

MUX

Ad esempio il MUX può essere implementato a TG con 10 MOSFET in totale. Assumendo lo stesso criterio di dimensionamento usato precedentemente per i FA ($Z_p = 2Z_n$) possiamo calcolare la capacità fisica complessiva di ciascun nodo:

- Entrambi gli ingressi hanno una capacità $C = C_n + C_p$
- I nodi di uscita degli invertitori di ingresso hanno una capacità $2C_n + 2C_p = 2C$
- L'uscita dei TG ha una capacità $3C_n + 3C_p = 3C$
- L'uscita del MUX ha una capacità $C = C_n + C_p$

È facile verificare che, essendo i segnali di ingresso riporti da FA precedenti (con probabilità di essere a 1 pari a 0.5), l'uscita del MUX è equiprobabile:

$$C_{OUT} = BP \cdot C_{IN1} + (1 - BP)C_{IN2}$$

$$p_{C_{OUT}} = P_{BP}\frac{1}{2} + (1 - P_{BP})\frac{1}{2} = \frac{1}{2}$$

La capacità logica è pertanto:

$$c\frac{1}{4} + c\frac{1}{4} + 2c\frac{1}{4} + 2c\frac{1}{4} + 3c\frac{1}{4} + c\frac{1}{4} = \frac{10C}{4} = 2.5C$$

Porta AND/NAND

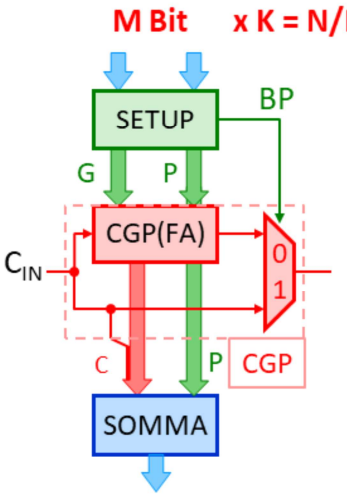
Il segnale di BP può essere realizzato in logica statica complementare, ad esempio, con una NAND e un invertitore che generano BP in forma diretta e negata (entrambi necessari per pilotare il MUX). L'implementazione richiede 2M+2 MOS. In termini di capacità logica la AND/NAND da un contributo trascurabile. Avendo gli ingressi equiprobabili (segnali P_n), la porta AND è a 1 1 caso su 2^M. Ciò si traduce in un fattore di attività bassissimo che trascuriamo per semplicità. La figura a destra mostra un esempio con 4 bit.

CBP: numero di MOS

M = 4

MUX a TG → 10 MOS
BP (NAND-4in + NOT) → 10 MOS

FA a TG:
SETUP → 10 MOS
CPG → 8MOS
SOMMA → 6MOS



	8 bit	16 bit	32 bit	64 bit	N bit
K	2	4	8	16	N/4
SETUP	80	160	320	640	10N
CGP (FA)	64	128	256	512	8N
CGP (BP)	20	40	80	160	2.5N
CGP(MUX)	20	40	80	160	2.5N
SOMMA	48	96	192	384	6N
Totale	**232**	**464**	**928**	**1856**	**29N**

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 26

Numero di MOS

In termini di numero di MOS il CBP richiede hardware aggiuntivo (1 MUX e 1 porta NAND/AND per blocco).
Sia M il numero di bit per blocco, sono richiesti 10 MOS per il MUX e 2M+2 per la porta. In totale abbiamo 2M+12 MOS per blocco.
Per sommare N bit sono necessari N/M blocchi, per un totale di:

$$(2M + 12)\frac{N}{M} = 2N + 12\frac{N}{M}\ MOS$$

Se ad esempio M = 4:

$$2N + 12\frac{N}{M}\ MOS = 5N$$

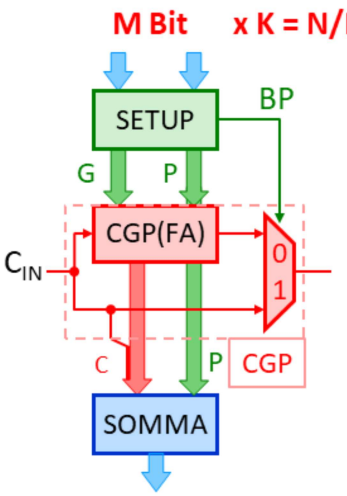
La tabella mostra il numero di MOS assumendo M = 4. In rosso sono mostrati i MOS aggiuntivi rispetto al ripple-carry, suddivisi tra il MUX e la porta AND/NAND del segnale di BP. In totale corrisponde a un aumento del 20% circa.

CBP: stima del consumo (capacità logica)

M = 4

MUX a TG → 2.5C
BP (NAND+ NOT) → trascurabile

FA a TG:
SETUP → 3.25C
CPG → 2.5C
SOMMA → 2C

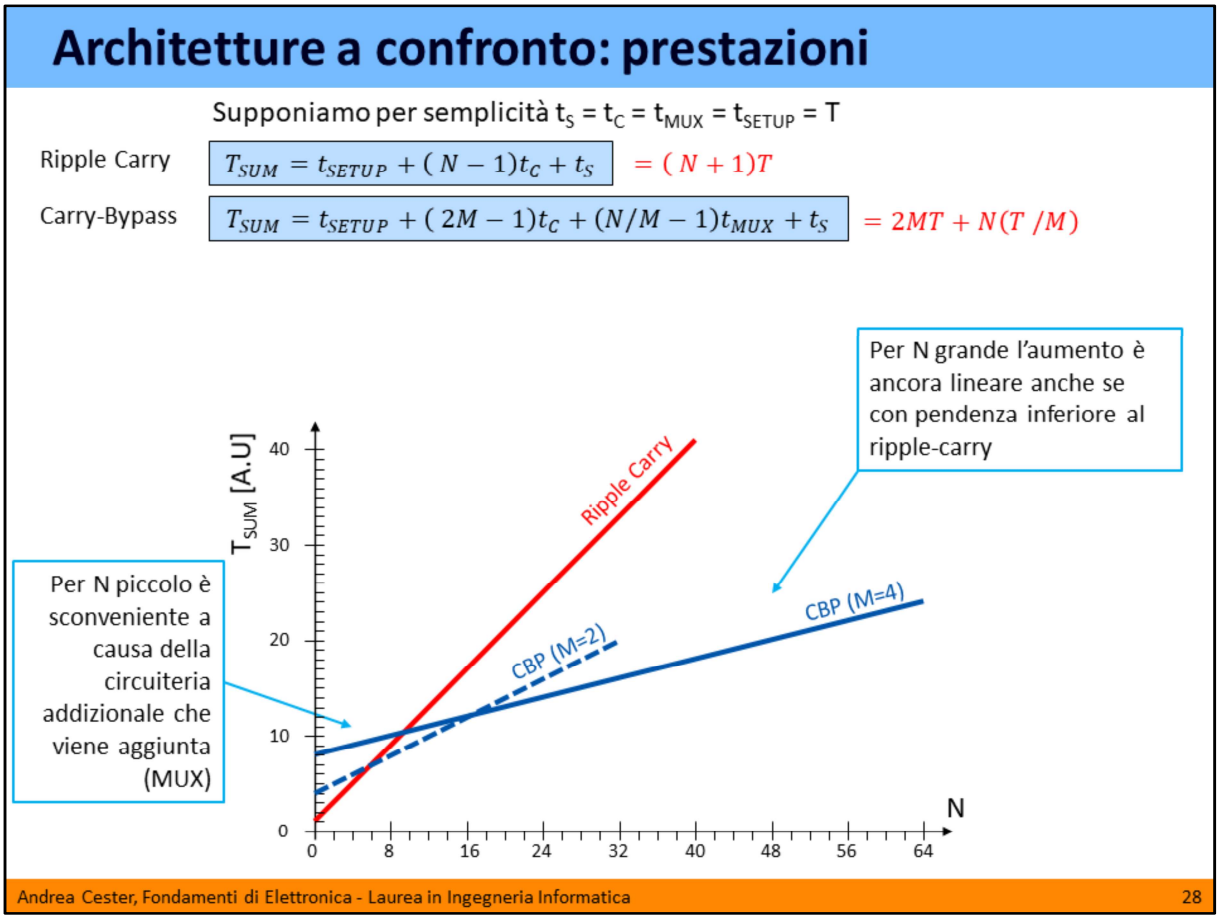


	8 bit	16 bit	32 bit	64 bit	N bit
K	2	4	8	16	N/4
SETUP	26C	52C	104C	208C	3.5N·C
CGP (FA)	20C	40C	80C	160C	2.5N·C
CGP (BP)	~0	~0	~0	~0	~0
CGP(MUX)	5C	10C	20C	40C	0.625N·C
SOMMA	16C	32C	64C	128C	2N·C
Totale	**67C**	**134C**	**268C**	**526C**	**8.625N·C**

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 27

Consumo dinamico / capacità logica

La tabella riporta la capacità logica del sommatore CBP (con M = 4). In rosso sono evidenziati i contributi aggiuntivi rispetto a ripple-carry. Il contributo della porta AND/NAND è stato trascurato per i motivi appena spiegati.



Architetture a confronto: prestazioni

Consideriamo per semplicità tutti i tempi di propagazione dei blocchi elementari pari a T (t_S=t_C=t_{MUX}=t_{SETUP}=T).

Il tempo di somma di un *sommatore ripple-carry* risulta:

$$T_{SUM} = (N-1)t_C + t_S = N \cdot T$$

Il *sommatore CBP* invece ha un tempo di somma pari a:

$$T_{SUM} = t_{SETUP} + (2M-1)t_C + (K-1)t_{MUX} + t_S = T + (2M-1)T + \left(\frac{N}{M} - 1\right)T + T$$

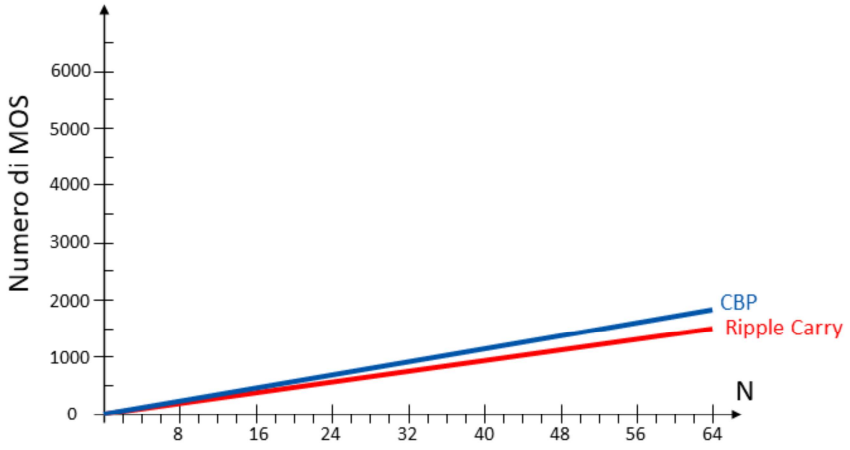
$$= 2MT + \frac{M}{T}N$$

Fissato M è evidente come entrambi i sommatori hanno un tempo di ritardo lineare con N, ma il CBP ha una pendenza M volte inferiore al ripple-carry.

Ciò significa che per N abbastanza grande il CBP diventa sicuramente conveniente rispetto al ripple-carry. Se, però, N è piccolo il CBP introduce un ritardo addizionale.

Architetture a confronto: numero di MOS

Sommatore	8 bit	16 bit	32bit	64bit	N bit
Ripple carry	192	384	768	1536	24N
Carry by-pass	232	464	928	1856	29N



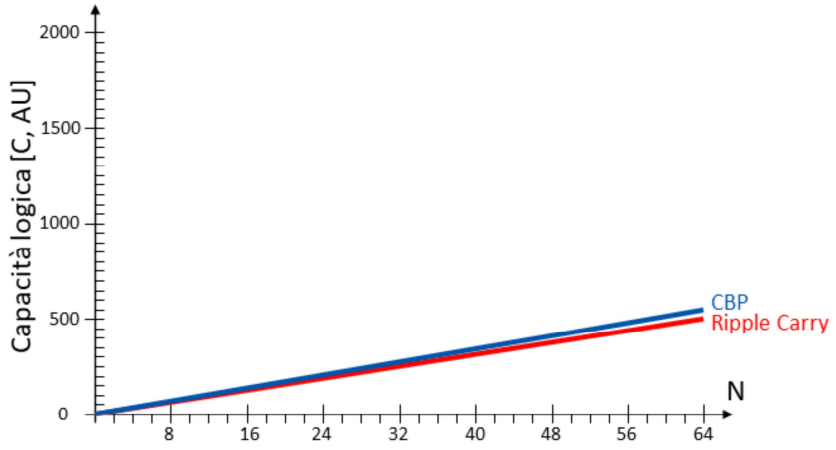
Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 29

Architetture a confronto: numero di MOS

La tabella e il grafico confrontano il numero di MOS richiesti per realizzare un sommatore in architettura ripple-carry e carry-by-pass (con M = 4). L'aumento del numero di MOS è circa il 20%

Architetture a confronto: consumo/capacità logica

Sommatore	8 bit	16 bit	32bit	64bit	N bit
Ripple carry	62C	124C	248C	486C	8N·C
Carry by-pass	67C	134C	268C	526C	8.625N·C

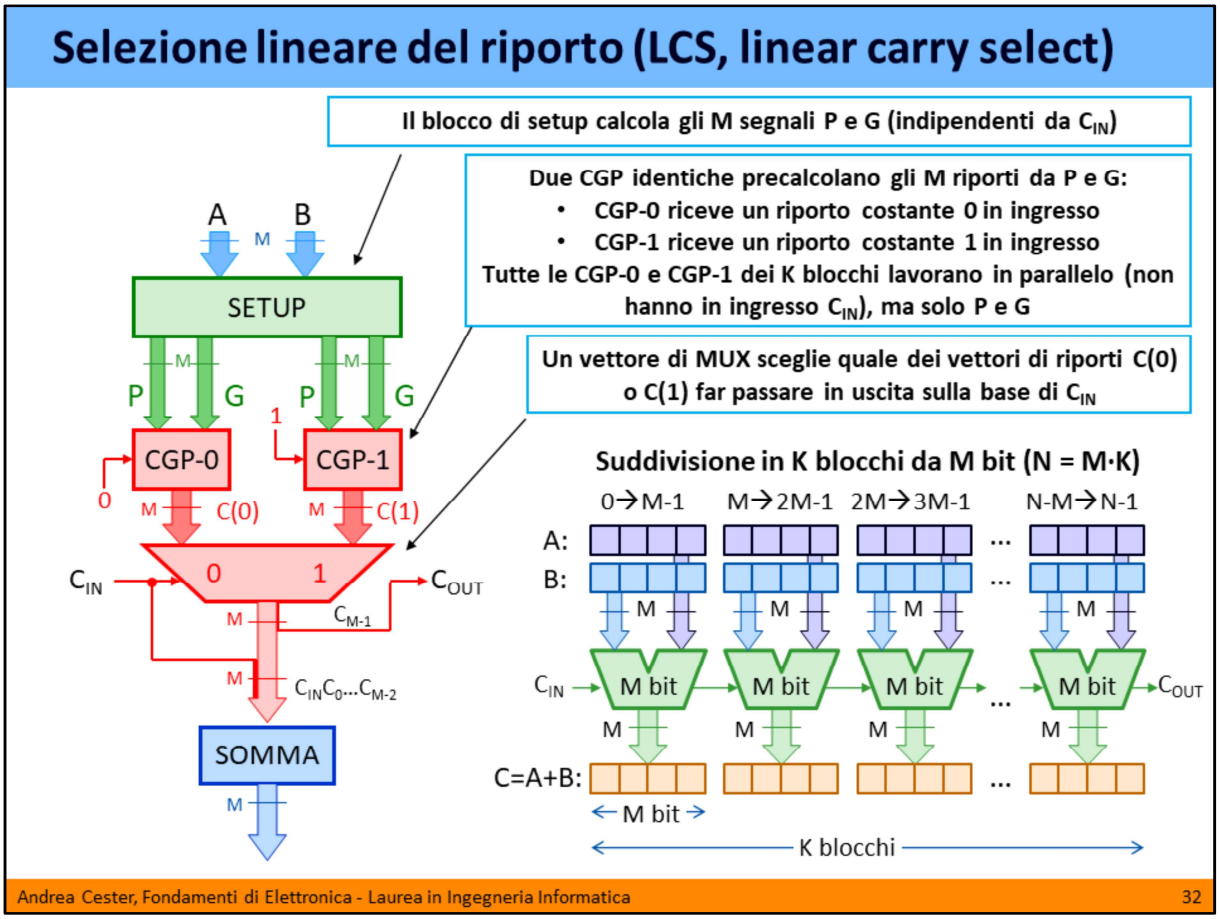


Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 30

Architetture a confronto: consumo dinamico / capacità logica

La tabella e il grafico confrontano la capacità logica di un sommatore in architettura ripple-carry e carry-by-pass (con M = 4). Poiché la porta AND/NAND non aumenta sensibilmente la capacità logica, l'aumento è contenuto a circa 8%.

ARCHITETTURA A SELEZIONE LINEARE DEL RIPORTO



Architettura a selezione linere del riporto (Linear Carry Select, LCS)

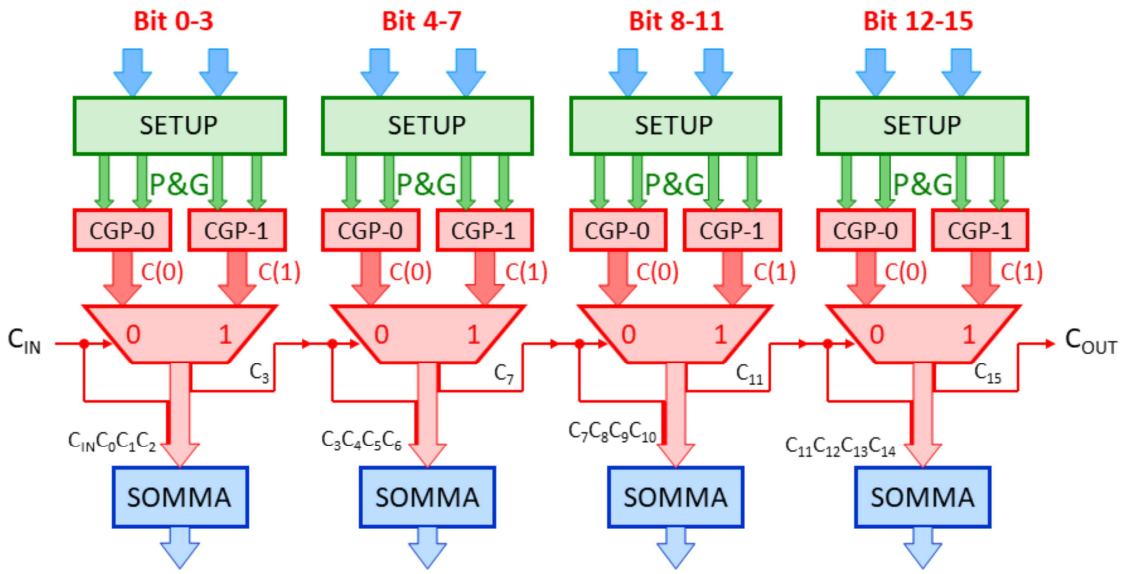
Questo tipo di sommatore affronta il problema di propagazione del riporto utilizzando una quantità di *hardware* che è quasi il doppio, a parità di lunghezza delle parole in ingresso, rispetto a quello utilizzato per un *ripple carry*. L'approccio seguito di fatto è quello di utilizzare sommatori, opportunamente connessi tramite una logica combinatoria, che operano in parallelo. Analogamente al carry-by-pass, i numeri sono divisi in K blocchi di M (=N/K) bit ciascuno.

Il blocco fondamentale di un LCS è costituito da:

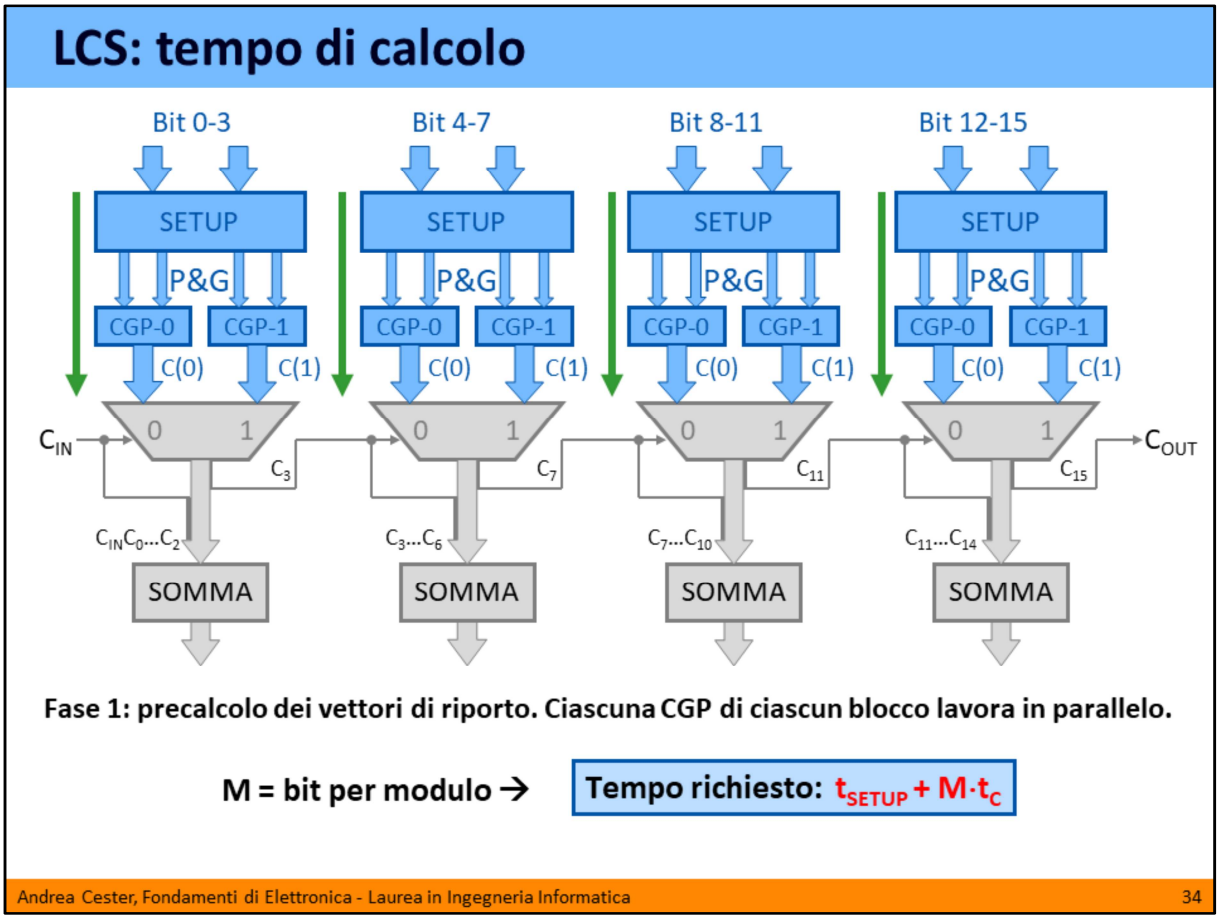
- un circuito di setup che genera i segnali G e P
- Due blocchi di calcolo dei riporti. Il blocco CGP-0 calcola l'intera catena di riporto assumendo un riporto 0 in ingresso. Il blocco CGP-1 calcola l'intera catena di riporto assumendo un riporto 1 in ingresso. Entrambi i blocchi operano in parallelo.
- Un array di MUX (1 MUX per ciascun bit) tutti pilotati da C_{IN} come segnale di controllo, che seleziona quali dei due vettori di riporti trasmettere all'uscita, una volta noto il valore di C_{IN}.
- Un blocco di somma che a partire dai riporti e dai segnali P calcola la somma.

Selezione lineare del riporto (LCS, linear carry select)

Esempio N = 16 bit divisi in K = 4 blocchi da M = 4 bit ciascuno



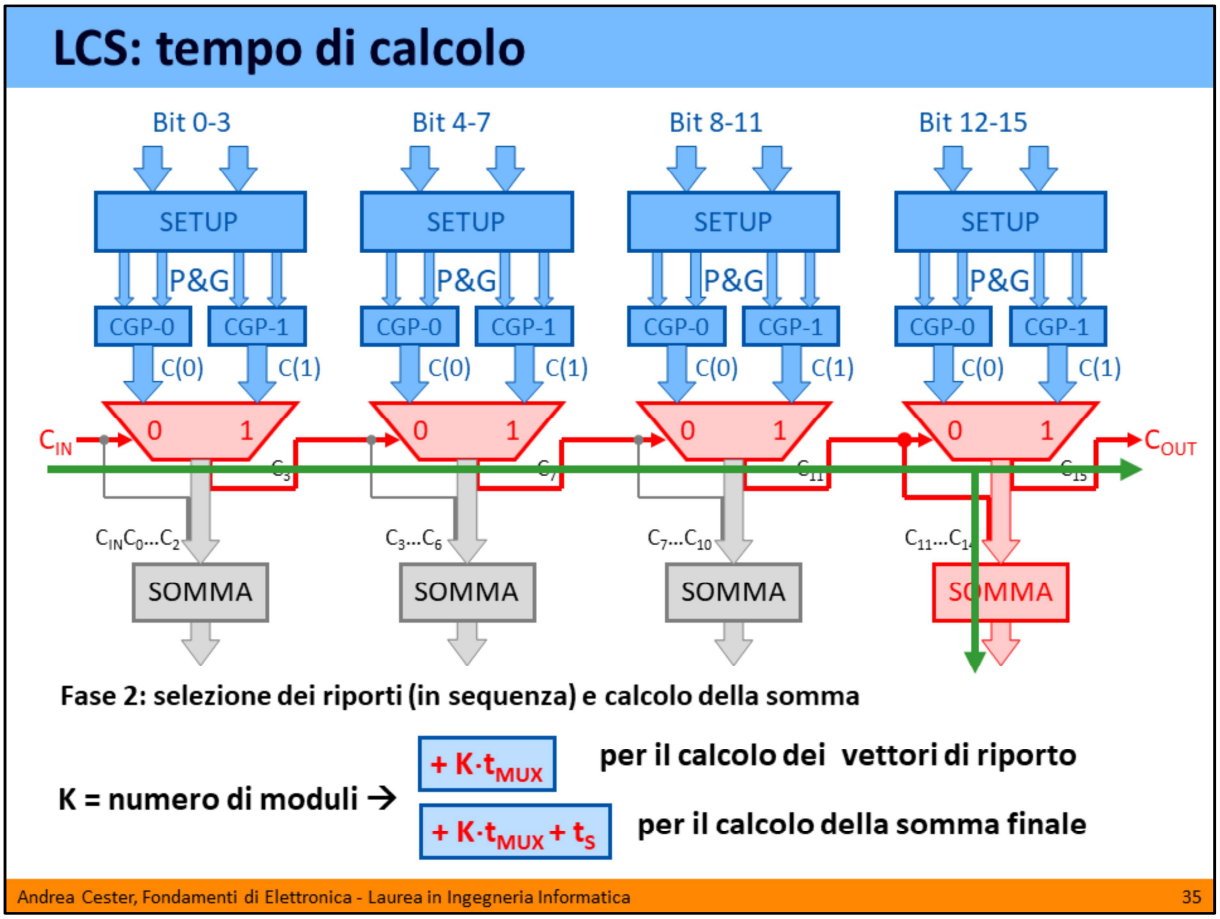
Un *LCS* è suddiviso in *K* blocchi di lunghezza *M* bit, ciascuno dei quali esegue due somme in parallelo, come illustrato nella slide precedente.
In figura vediamo lo schema completo di un sommatore composto da *K* = 4 gruppi con *M* = 4 *bit* ciascuno. Il circuito prevede in ingresso un riporto C_{IN} che possa assumere valore 0 o 1 a seconda che la struttura debba essere utilizzata come sommatore o sottrattore.



Percorso critico e tempo di calcolo

Il calcolo di somma e riporto è suddiviso in due parti.
La prima parte procede in parallelo su ciascun blocco attraverso il circuito di setup e poi lungo le due catene di calcolo del riporto.
Il tempo richiesto per questo calcolo (fino agli ingressi dei MUX) è:

$$t_{SETUP} + M \cdot t_C$$



La seconda parte del percorso di calcolo procede in sequenza blocco dopo blocco a partire dal primo, come per il ripple carry.
Nel primo blocco il segnale C_{IN} seleziona quale delle due catene di riporto trasmettere all'uscita.
Il bit più significativo del primo blocco (cioè il riporto della somma del bit M-esimo) viene usato come segnale di controllo per il MUX del blocco successivo.
Quindi il secondo blocco dovrà aspettare la propagazione attraverso il primo MUX per poter selezionare il vettore di riporto corretto. Ciò accade per tutti i K blocchi fino alla fine.
Questa seconda fase richiede un tempo:

$$k \cdot t_{MUX}$$

Per il calcolo della somma è richiesto un ulteriore tempo t_S del blocco di somma.

$$k \cdot t_{MUX} + t_S$$

La differenza di un LCS rispetto a un ripple carry è che la propagazione in sequenza avviene per blocco e coinvolge K stadi invece di N stadi.

Linear Carry Select: scelta ottima di M e K

N = numero bit
M = bit per blocco
K = N/M = numero blocchi

$$T_{CARRY} = t_{SETUP} + M \cdot t_C + K \cdot t_{MUX}$$

$$T_{SUM} = t_{SETUP} + M \cdot t_C + K \cdot t_{MUX} + t_S$$ **Tempo critico**

Derivando rispetto a M e uguagliando a 0 si trova:

$$T_{SUM} = t_{SETUP} + M \cdot t_C + \frac{N}{M} \cdot t_{MUX} + t_S$$

$$\frac{dT_{SUM}}{dM} = t_C - \frac{N}{M^2} t_{MUX} = 0$$

$$M_{OPT} = \sqrt{N \cdot \frac{t_{MUX}}{t_C}}$$

Se $t_C \cong t_{MUX}$
(come nel FA a TG)

$$M_{OPT} = \sqrt{N}$$

N = 8 → M = 2-3
N = 16 → M = 4
N = 32 → M = 5-6
N = 64 → M = 8

Scelta del numero di bit per blocco

Riassumendo, un sommatore con architettura LCS in cui i bit sono raggruppati in K blocchi di M bit ciascuno, i tempi richiesti per il calcolo di somma e riporto nel caso peggiore sono:

$$T_{SUM} = t_{SETUP} + M \cdot t_C + K \cdot t_{MUX} + t_S$$
$$T_{CARRY} = t_{SETUP} + M \cdot t_C + K \cdot t_{MUX}$$

dove t_{SETUP} è il tempo di propagazione del blocco di setup, t_C è tempo richiesto al calcolo del riporto di un singolo bit, t_{MUX} è il tempo di propagazione dei MUX (supponendo che sia uguale per tutti) e t_S è il tempo di calcolo della somma di un singolo bit.
È necessario a questo punto capire, data la lunghezza *N* delle parole da sommare, e date le caratteristiche delle celle elementari utilizzate (*full adder* e multiplexer) in termini di ritardo introdotto, quale sia la partizione migliore (numero *K* e dimensione *M* dei gruppi in cui viene suddiviso il sommatore), al fine di avere ritardo minimo introdotto.
L'obiettivo è quello di minimizzare la somma:

$$M \cdot t_C + K \cdot t_{MUX}$$

Essendo K = N/M e derivando l'espressione, otteniamo:

$$\frac{\partial}{\partial M}\left(M \cdot t_C + \frac{N}{M} \cdot t_{MUX}\right) = t_C - \frac{N}{M^2} \cdot t_{MUX} = 0$$

$$M = \sqrt{\frac{t_{MUX}}{t_C} N}$$

Se t_{MUX} e t_C sono circa uguali risulta:

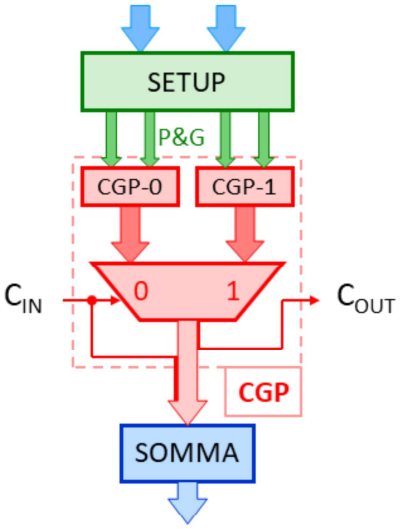
$$t_C \cong \sqrt{N}$$

LCS: numero di MOS

MUX a TG → 10 MOS

FA a TG:
SETUP → 10 MOS
CPG → 8MOS
SOMMA → 6MOS

M Bit x K = N/M blocchi



	8 bit	16 bit	32 bit	64 bit	N bit
SETUP	80	160	320	640	10N
CGP (FA)	2x64	2x128	2x256	2x512	2x8N
CGP(MUX)	80	160	320	640	10N
SOMMA	48	96	192	384	6N
Totale	**336**	**672**	**1344**	**2688**	**42N**

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 37

Numero di MOS

L'approccio LCS richiede molto più hardware aggiuntivo. In particolare, sono richiesti:

- Due moduli CGP per ciascun bit. Che porta da 8N a 16N il numero di MOS per il calcolo dei possibili riporti.
- Un MUX per ciascun bit che richiede ad esempio 10 MOS nella realizzazione a TG (trascurando l'invertitore per il segnale di controllo, che è condiviso per ciascun blocco).

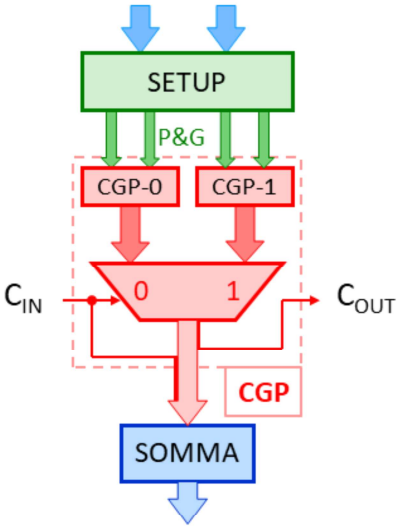
Sommando i due contributi otteniamo circa 18N MOS.

LCS: stima del consumo (capacità logica)

MUX a TG → 2.5C

FA a TG:
SETUP → 3.25C
CPG → 2.5C
SOMMA → 2C

M Bit x K = N/M blocchi



	8 bit	16 bit	32 bit	64 bit	N bit
SETUP	26C	52C	104C	208C	3.5N·C
CGP (FA)	2x20C	2x40C	2x80C	2x160C	2x2.5N·C
CGP(MUX)	20C	40C	80C	160C	2.5N·C
SOMMA	16C	32C	64C	128C	2N·C
Totale	**102C**	**204C**	**408C**	**816C**	**13N·C**

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 38

Consumo dinamico / capacità logica

La tabella riporta la capacità logica del sommatore CBP (con M = 4). In rosso sono evidenziati i contributi aggiuntivi rispetto a ripple-carry.
Il sostanzioso aumento di Hardware si traduce in un cospicuo aumento della capacità logica.
L'incremento complessivo è di 5N·C che si somma a 8N·C del ripple-carry.

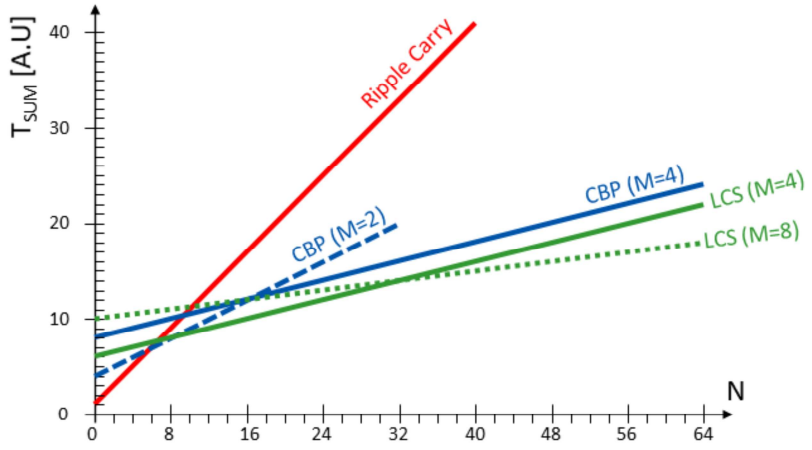
Architetture a confronto

Supponiamo per semplicità $t_S = t_C = t_{MUX} = t_{SETUP} = T$

Ripple Carry $T_{SUM} = t_{SETUP} + (N-1)t_C + t_S = (N+1)T$

Carry-Bypass $T_{SUM} = t_{SETUP} + (2M-1)t_C + (N/M-1)t_{MUX} + t_S = 2MT + N(T/M)$

Lin. Carry. Sel. $T_{SUM} = t_{SETUP} + Mt_C + (N/M)t_{MUX} + t_S = T(M+2) + N(T/M)$



Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 39

Architetture a confronto: prestazioni

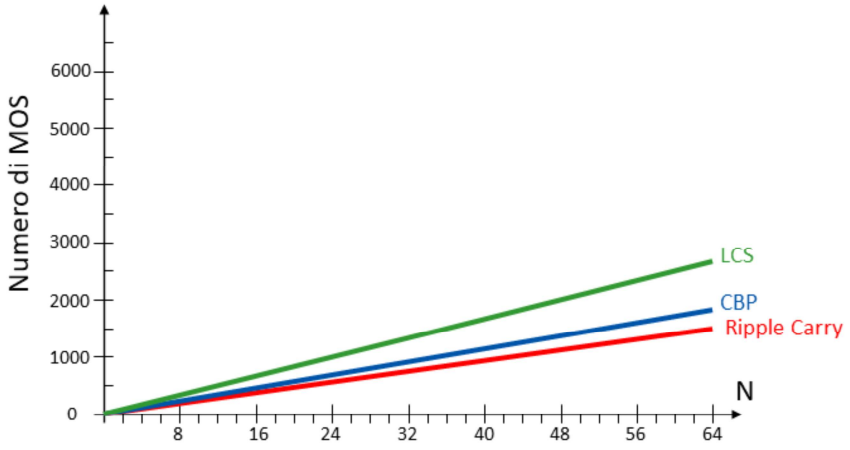
Aggiungiamo nel grafico di confronto il sommatore LCS. Osserviamo che il tempo di ritardo rimane proporzionale a N anche se (nelle ipotesi semplificative fatte) inferiore al sommatore CBP.

Le due curve in verde si riferiscono a M = 4 e M = 8. Vediamo che il valore di M dipende molto dal numero di bit. Se N < 32 è preferibile usare blocchi più piccoli . Tuttavia per N troppo grande l'impiego di blocchi piccoli fa aumentare troppo il numero K di blocchi comportando un eccessivo ritardo sella seconda fase. Conviene incrementare M per mantenere bilanciati i ritardi della prima fase (calcolo parallelo) e seconda fase (calcolo sequenziale).

Confrontando le architetture CBP e LCS con M= 4 bit, osserviamo che entrambe forniscono la stessa pendenza, ma il sommatore LCS ha un offset più basso.

Architetture a confronto: numero di MOS

Sommatore	8 bit	16 bit	32bit	64bit	N bit
Ripple carry	192	384	768	1536	24N
Carry by-pass	232	464	928	1856	29N
Linear Carry Select	336	672	1344	2688	42N



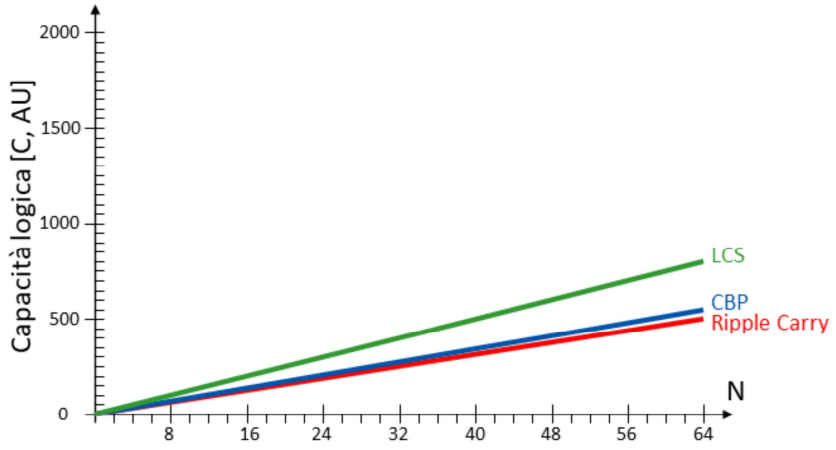
Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 40

Architetture a confronto: numero di MOS

L'aumento delle prestazioni viene pagato ovviamente dal raddoppio dell'hardware, con un incremento del 75% sul numero di MOS rispetto al sommatore ripple carry e del 45% rispetto al sommatore carry by-pass.

Architetture a confronto: consumo/capacità logica

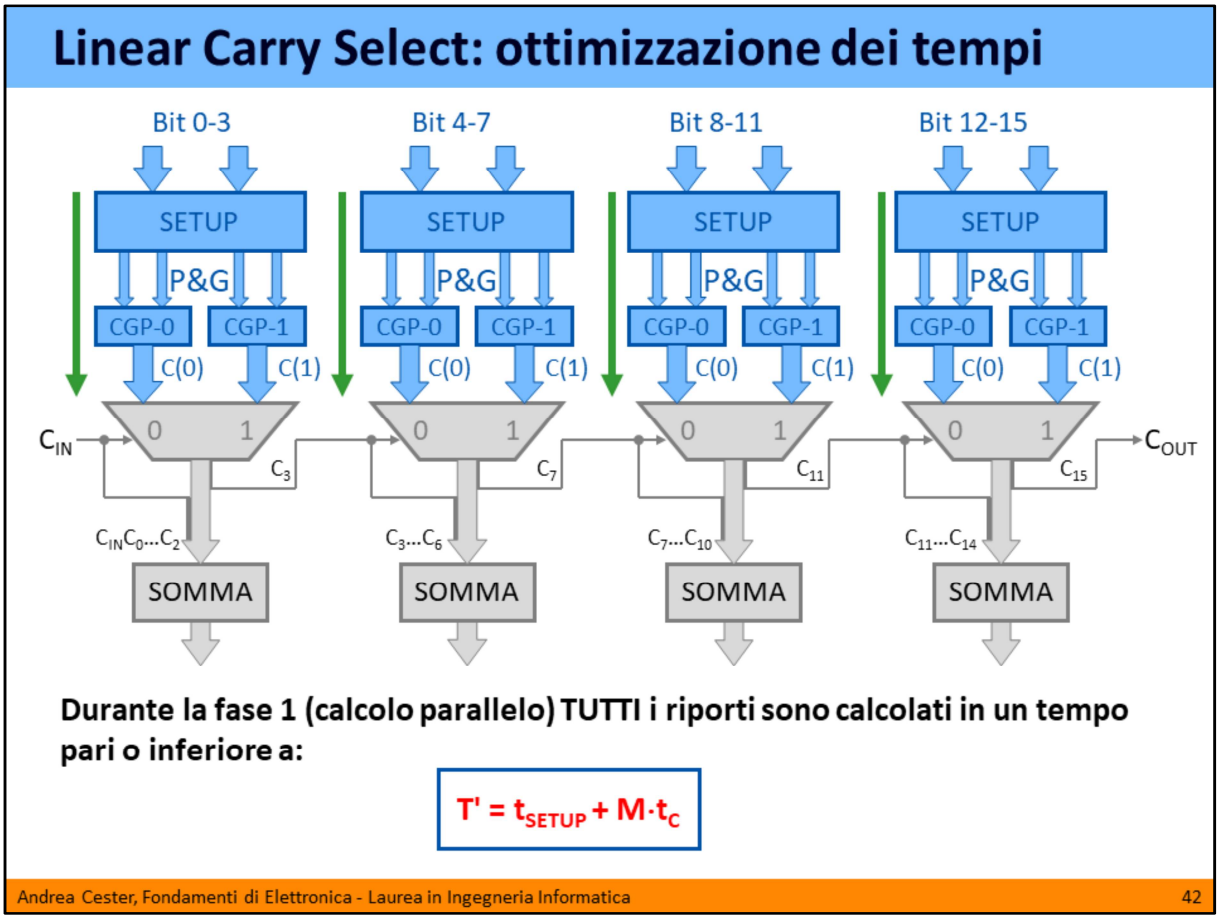
Sommatore	8 bit	16 bit	32bit	64bit	N bit
Ripple carry	62C	124C	248C	486C	8N·C
Carry by-pass	67C	134C	268C	526C	8.625N·C
Linear Carry Select	102C	204C	408C	816C	13N·C



Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 41

Architetture a confronto: consumo dinamico / capacità logica

La tabella e il grafico confrontano la capacità logica del sommatore LCS con il sommatore ripple-carry e carry-by-pass (M = 4). Parallelamente al numero di MOS cresce il consumo (più del 60% rispetto al ripple-carry.



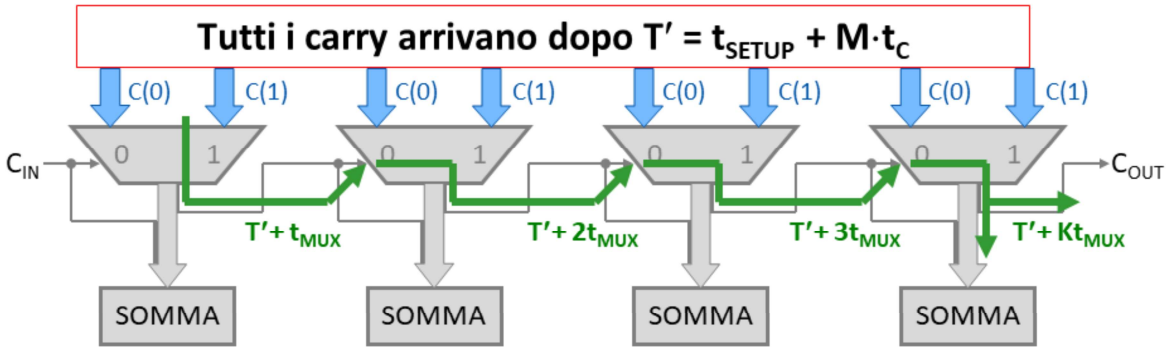
Ottimizzazione del ritardo

Nell'analisi appena svolta si è supposto di dividere i bit in gruppi di dimensione identica *M*.
In questo modo, dopo un tempo paria a:

$$T' = t_{SETUP} + N \cdot t_C$$

tutti i riporti sono pronti all'ingresso dei MUX.
Fino a questo punto il calcolo avviene in parallelo. Da questo punto in poi la propagazione del riporto avviene in sequenza.

LCS: ottimizzazione dei tempi



C_{OUT} (blocco 1) = C_{IN} (blocco 2) = selettore MUX blocco 2: disponibile dopo T′ + t_{MUX}

C_{OUT} (blocco 2) = C_{IN} (blocco 3) = selettore MUX blocco 3: disponibile dopo T′ + $2t_{MUX}$

C_{OUT} (blocco 3) = C_{IN} (blocco 4) = selettore MUX blocco 4: disponibile dopo T′ + $3t_{MUX}$

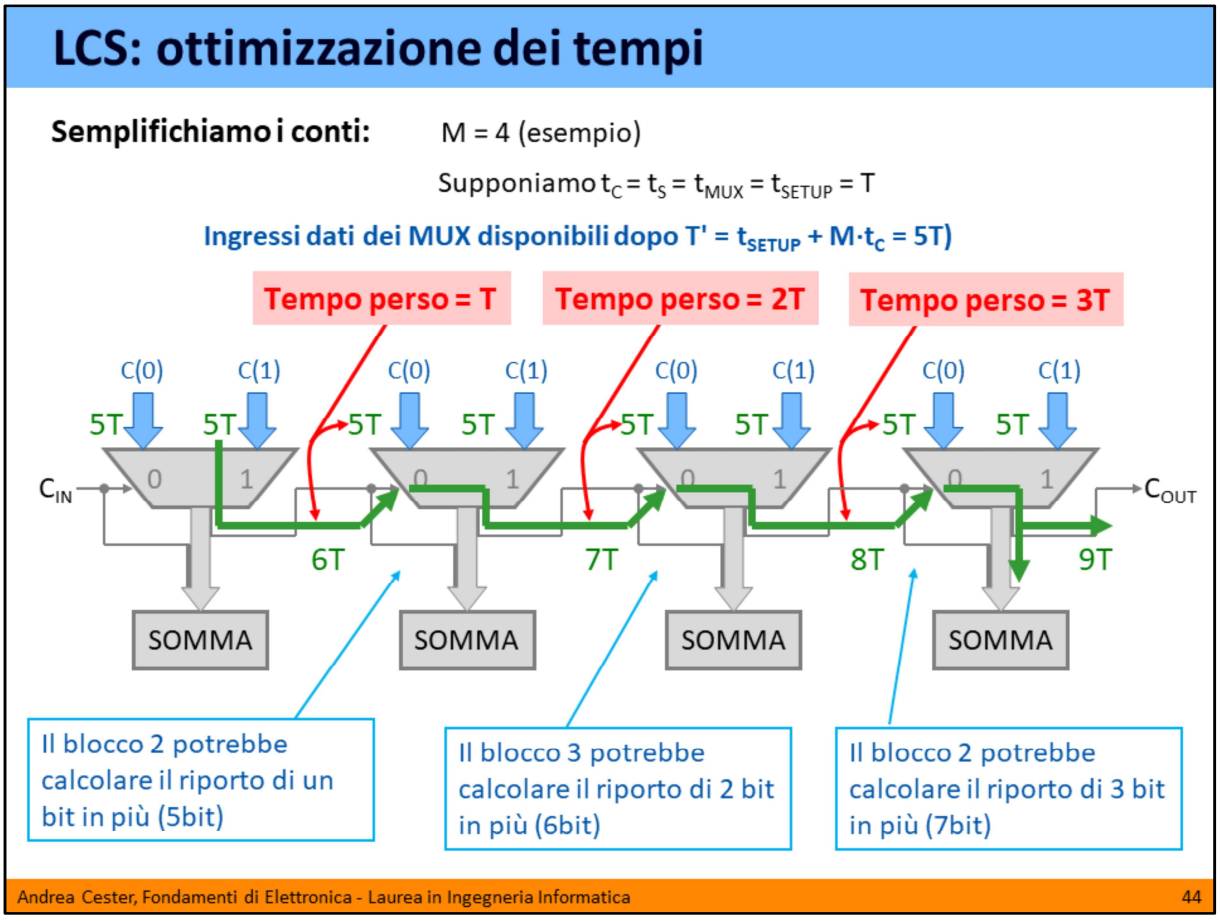
⋮

C_{OUT} (blocco K-1) = C_{IN} (blocco K) = selettore MUX blocco K: disponibile dopo T′ + Kt_{MUX}

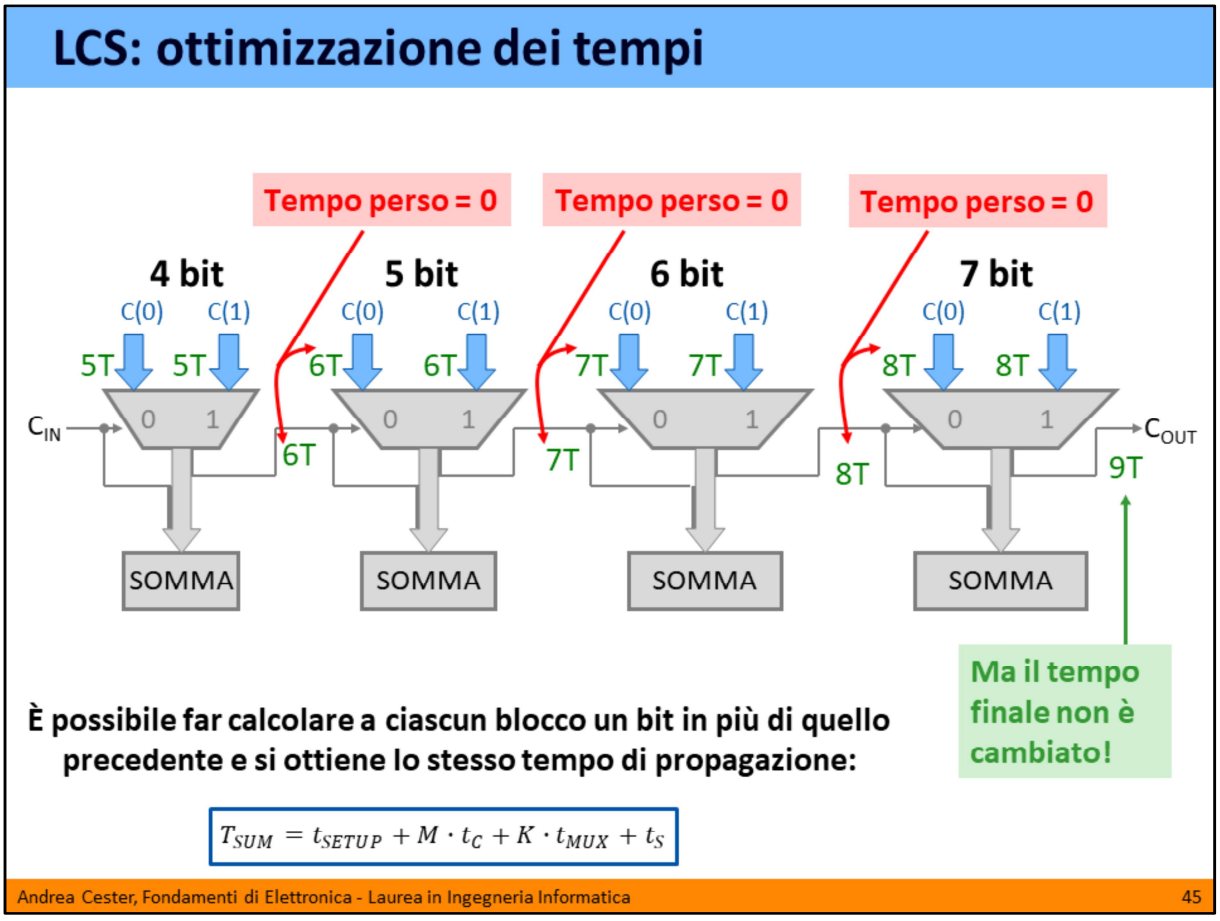
… ma tutti gli ingressi dati dei MUX sono pronti entro T'… stiamo perdendo tempo!

Dopo il tempo T' tutti gli ingressi dati dei MUX sono pronti. Osserviamo però i tempi con cui arrivano gli ingressi di controllo.

- Il segnale di selezione del primo blocco è C_{IN} e si assume pronto insieme agli ingressi A e B. Per il primo blocco il segnale critico è la coppia di riporti relativi all'ultimo bit che arriva dopo un tempo T'. Il riporto di uscita del primo blocco (MSB del blocco) sarà pronta dopo T' + t_{MUX}. Questa rappresenta l'ingresso di selezione di tutti i MUX del secondo blocco.
- Il secondo blocco riceve gli ingressi dati dopo un tempo T' (come il primo blocco), ma l'ingresso critico, cioè l'ultimo a determinare il valore delle uscite è quello di selezione (che arriva dopo T' + t_{MUX}). I MUX del secondo blocco devono attendere un tempo aggiuntivo t_{MUX}! La sua uscita sarà pronta dopo T'+$2t_{MUX}$.
- Il terzo blocco riceve il segnale di selezione dal secondo (dopo T'+$2t_{MUX}$) con un ritardo di 2 t_{MUX} rispetto agli ingressi dati.
- Procedendo in questo modo nel blocco k-esimo c'è un ritardo di k·t_{MUX} tra quando sono pronti i dati a quando arriva l'ingresso di selezione.



Esempio: Supponiamo per semplicità che $t_{MUX} = t_C = T$, N=16, M=4.
Tutti i riporti sono disponibili dopo 5T.
Il primo dei quattro blocchi calcola il riporto di uscita in 6T.
Osserviamo i MUX del secondo blocco. Gli ingressi dati arrivano dopo 5T, ma l'ingresso di selezione arriva dopo 6T. Il secondo blocco avrebbe tempo di calcolare un bit addizionale (5 bit invece di 4) senza perdere ulteriormente tempo.
Lo stesso ragionamento si applica al terzo blocco, in cui gli ingressi dati sono disponibili dopo 5T, ma il segnale di selezione è pronto dopo 7T. In questo caso il terzo blocco potrebbe calcolare fino a 6 bit.
Analogamente il quarto blocco potrebbe calcolare 7bit



Nulla vieta di operare una suddivisione differente, dimensionando il numero di *bit* dei vari blocchi in modo crescente, tale da *"riempire tutti i tempi morti"* dei MUX.

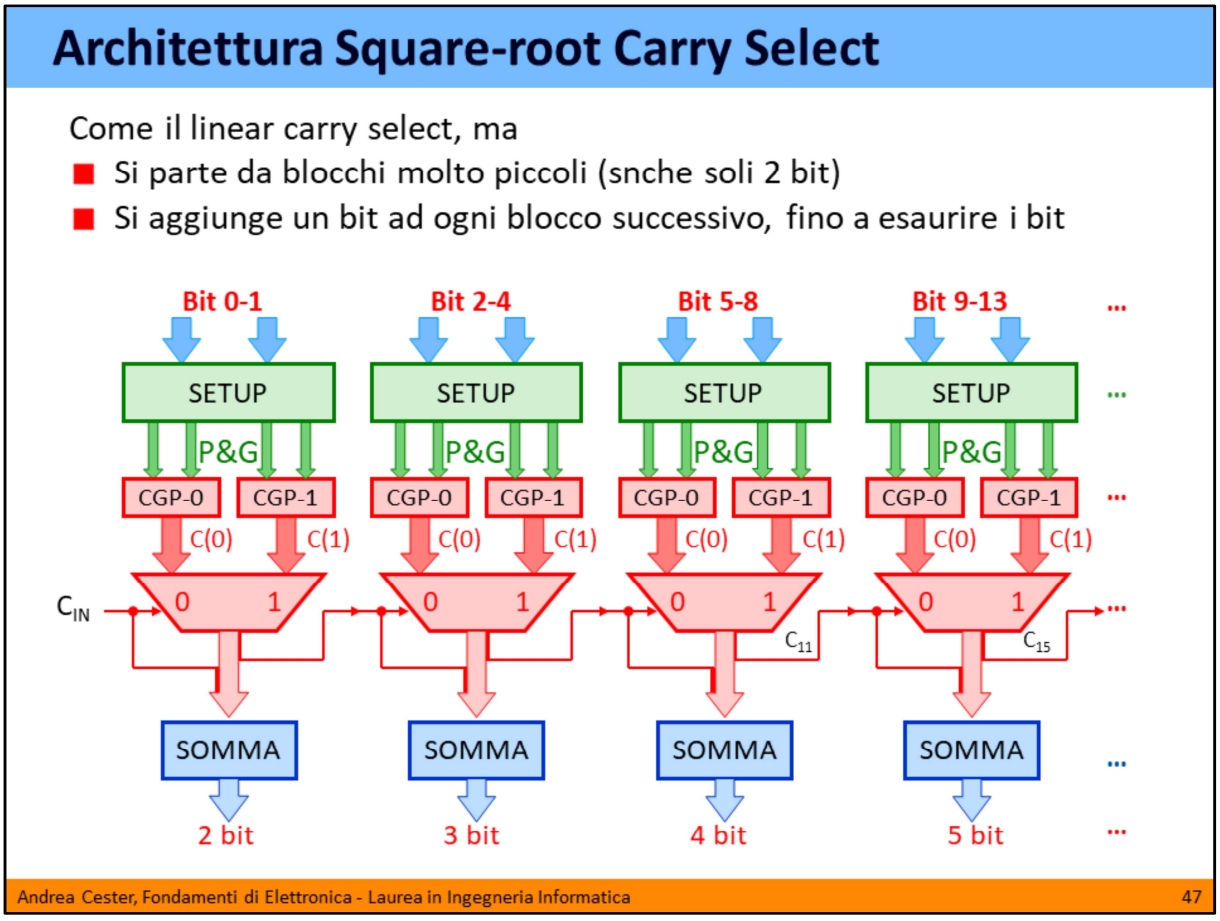
Di quanto devono essere incrementati i bit da un blocco al successivo? Dipende ovviamente dai tempi t_C e t_{MUX}. *L'incremento del numero di bit deve essere tale che il tempo addizionale per il calcolo dei riporti non superi il tempo di attesa.*

Nell'esempio considerato, dato che ciascun MUX introduce un tempo di attesa pari $t_{MUX} = t_C$, possiamo aumentare progressivamente di un bit le dimensioni di ciascun blocco. In questo modo il numero totale di bit calcolati sarebbe:

$$M + M+1 + M+2 + M+3 = 4M+6 = 22$$

Il tempo di calcolo però non cambia.

ARCHITETTURA SQUARE-ROOT CARRY SELECT



Architettura Square-root Carry Select (SRCS)

La considerazione appena fatta può essere usata per ottimizzare ulteriormente il sommatore, partendo da un numero M molto piccolo (ad esempio 2) e aumentare progressivamente il numero di bit per blocco in modo tale che la somma dei bit rimanga N (come mostrato in figura).

Criterio di scelta dell'incremento

Come abbiamo già detto l'incremento del numero di bit per blocco deve essere fatto in modo tale da non aggiungere tempo di calcolo. Durante la prima fase, tutti i blocchi procedono in parallelo. Il primo blocco impiega un tempo:

$$T' = t_{SETUP} + M \cdot t_C$$

I tempi di calcolo della prima fase nei successivi stadi deve essere minore o uguale al tempo in cui arriva il segnale di selezione. Dal secondo blocco in poi, il segnale di selezione arriva al blocco k-esimo in un tempo:

$$T_K = t_{SETUP} + M \cdot t_C + (k-1)t_{MUX}$$

Detto M_k il numero di bit complessivo del blocco k, il tempo impiegato a calcolare le due stringhe di riporti (fase 1) è:

$$T'_K = t_{SETUP} + M_k \cdot t_C$$

Affinché l'incremento di bit non produce ritardi aggiuntivi è necessario che:

$$T'_k < T_K \quad \rightarrow \quad (M_k - M) < \frac{(k-1)t_{MUX}}{t_C}$$

Esempio 1: Assumiamo t_C = 0.25ns, t_{MUX} = 0.35 e M = 2 nel primo blocco.

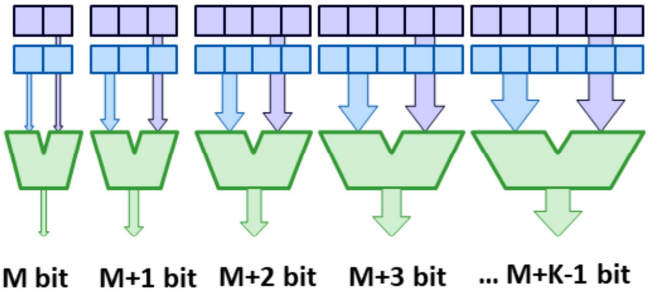
$$M_2 - M < 1.4 \quad , \quad M_3 - M < 2.8 \quad , \quad M_4 - M < 4.2 \quad , \quad M_5 - M < 5.6 \quad , \ldots$$

Possiamo adottare la seguente suddivisione dei bit:

$$2, 3, 4, 6, 7, \ldots$$

Il numero di bit è incrementato di 1 o 2 bit per blocco.

Square-root Carry Select: tempo di calcolo



N = numero bit
M = bit del primo blocco
K $\cong \sqrt{N}$ = numero blocchi

Numero bit totali

Somma dei primi K-1 numeri naturali

$$N = \underbrace{M + (M+1) + (M+2) + \cdots + (M+K-1)}_{\text{K termini}} = K \cdot M + \overbrace{1 + 2 + \cdots (K-1)}$$

$$N = K \cdot M + \frac{K(K-1)}{2} = \frac{K^2}{2} + K\left(M - \frac{1}{2}\right)$$

Se K>>M $\quad K \cong \sqrt{2N}$

$$T_{CARRY} \cong t_{SETUP} + M \cdot t_C + \sqrt{2N} \cdot t_{MUX}$$

$$T_{SUM} \cong t_{SETUP} + M \cdot t_C + \sqrt{2N} \cdot t_{MUX} + t_S$$

Tempo critico

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 48

Esempio 1: Assumiamo t_C = 0.5ns, t_{MUX} = 0.3 e M = 2 nel primo blocco.

$$M_2 - M < 0.6 \quad , \quad M_3 - M < 1.2 \quad , \quad M_4 - M < 1.8 \quad , \quad M_5 - M < 2.4 \quad , \ldots$$

Possiamo adottare la seguente suddivisione dei bit:

$$2, 2, 3, 3, 4, \ldots$$

Il numero di bit non può essere incrementato ad ogni blocco altrimenti si aggiungerebbe tempo di ritardo alla prima fase.

Tempo di calcolo

Definiamo M il numero iniziale di bit e K il numero di blocchi. Assumiamo di aumentare di un bit le dimensioni di ciascun blocco rispetto al precedente. Il numero N di bit vale:

$$N = M + (M+1) + (M+2) \ldots + (M+K-1) = K \cdot M + 1 + 2 + \cdots + (K-1) =$$

$$= K \cdot M + \frac{K \cdot (K-1)}{2}$$

Se M è abbastanza piccolo, possiamo assumere che il termine dominante sia quello quadratico:

$$N \cong \frac{K^2}{2}$$

Da cui ricaviamo:

$$K = \sqrt{2N}$$

I tempi di calcolo di somma e riporto sono:

$$T_{CARRY} = t_{SETUP} + M \cdot t_C + \sqrt{2N} \cdot t_{MUX}$$

$$T_{SUM} = t_{SETUP} + M \cdot t_C + \sqrt{2N} \cdot t_{MUX} + t_S$$

Osserviamo che il valore di M va scelto indipendentemente da N e può essere anche di poche unità (2-3).

SRCS: esempio di distribuzione dei bit sui blocchi

- I bit aumentano circa progressivamente per blocco
- È necessario fare aggiustamenti per tenere conto del fatto che la somma deve fare N
- La tabella mostra un esempio di distribuzione di bit e relativi temi di ritardo nell'ipotesi che $t_{SETUP} = t_C = t_{MUX} = t_S = T$.

	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	K	T_{SUM}
8 bit	2	3	3								**3**	T + 2T + **3T** + T = 7T
12 bit	2	3	3	4							**4**	T + 2T + **4T** + T = 8T
16 bit	2	3	3	4	4						**4**	T + 2T + **5T** + T = 9T
20 bit	2	3	4	5	6						**5**	T + 2T + **5T** + T = 9T
24 bit	2	3	4	5	5	5					**6**	T + 2T + **6T** + T = 10T
32 bit	2	3	4	5	6	6	6				**7**	T + 2T + **7T** + T = 11T
48 bit	2	3	3	4	5	6	7	8	9		**9**	T + 2T + **9T** + T = 13T
54 bit	2	3	4	5	6	7	8	9	10		**9**	T + 2T + **9T** + T = 13T
64 bit	2	3	4	5	6	7	8	9	10	10	**10**	T + 2T + **10T** + T = 14T

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 49

Numero di stadi

Supponiamo per semplicità $t_{MUX} = t_C$, e N = 20. Con un sommatore LCS servirebbero ad esempio 5 blocchi da 4 bit. Il tempo di ritardo sarebbe:

$$T_{SUM} = t_{SETUP} + 4t_C + 5t_{MUX} + t_S$$

L'aumento di 1bit per blocco comporta di sommare più bit nello stesso tempo. Se mantenessimo M = 4 nel primo blocco e K = 5 blocchi, otteniamo:

$$4 + 5 + 6 + 7 + 8 = 30$$

Da dove deriva la riduzione del tempo di calcolo? La risposta sta nel fatto che i bit da sommare sono sempre 20 (nel nostro esempio). Ci sono due possibilità:

1) Eliminare uno stadio mantenendo M=4. In questo caso con 4 stadi il numero di bit che sommiamo è 4 + 5 + 6 +7 = 22. Ciò riduce T_{SUM} di una quantità pari a t_{MUX}:

$$T_{SUM} = t_{SETUP} + 4t_C + 4t_{MUX} + t_S$$

2) Ridurre M es usare il minimo numero di stadi. Prendendo M = 2, abbiamo:

$$2 + 3 + 4 + 5 + 6 = 20$$

Servono sempre 5 stadi, ma il tempo complessivo è diminuito di $2t_C$.

$$T_{SUM} = t_{SETUP} + 2t_C + 5t_{MUX} + t_S$$

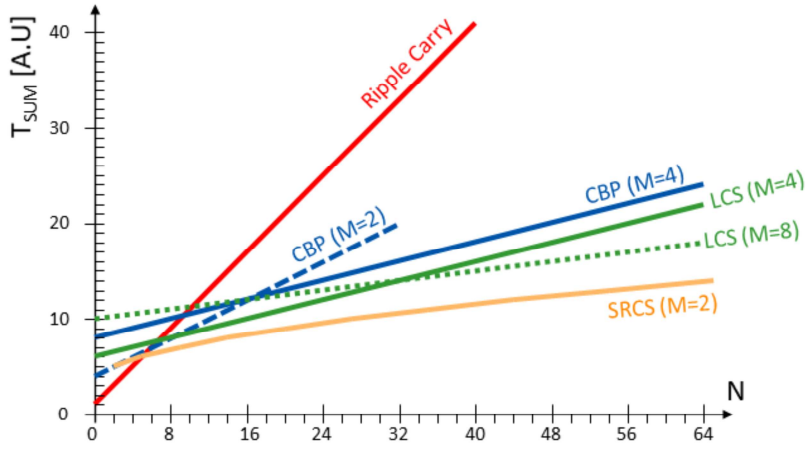
Questa seconda alternativa di dimostra la più efficiente.

La tabella mostra alcuni esempi di ripartizione crescente dei bit assumendo $t_C = t_{MUX}$.

Architetture a confronto

Supponiamo per semplicità $t_S = t_C = t_{MUX} = t_{SETUP} = T$

Ripple Carry	$T_{SUM} = t_{SETUP} + (N-1)t_C + t_S$	$= (N+1)T$
Carry-Bypass	$T_{SUM} = t_{SETUP} + (2M-1)t_C + (N/M-1)t_{MUX} + t_S$	$= 2MT + N(T/M)$
Lin. Carry. Sel.	$T_{SUM} = t_{SETUP} + Mt_C + (N/M)t_{MUX} + t_S$	$= T(M+2) + N(T/M)$
Sqr. Carry Sel.	$T_{SUM} = t_{SETUP} + Mt_C + \sqrt{2N}t_{MUX} + t_S$	$= T(M+2) + T\sqrt{2N}$



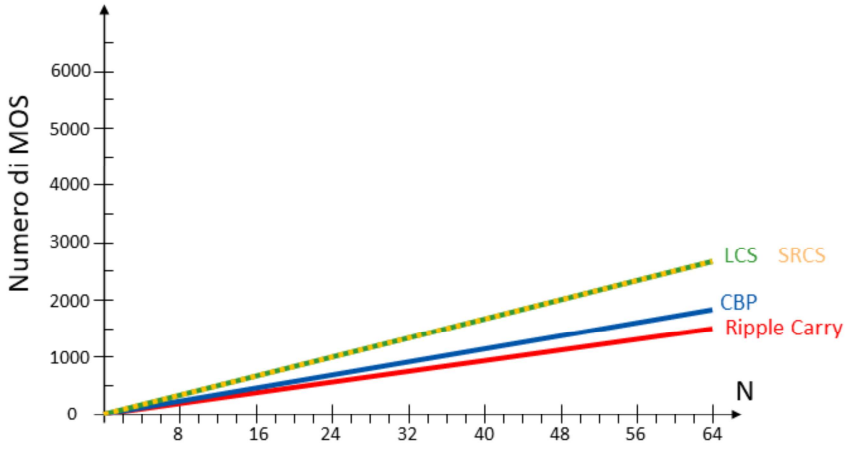
Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 50

Architetture a confronto: prestazioni

Aggiungiamo al grafico delle prestazioni il sommatore SRCS. Notiamo un netto miglioramento rispetto alla configurazione a selezione lineare. Le prestazioni migliorano per qualunque valore di N, anche se per N < 4 l'hardware aggiuntivo introduce un ritardo addizionale rispetto al semplice ripple carry.
Complessivamente, il valore critico di N al di sotto del quale è conveniente l'uso di un ripple carry varia da 4 a 8 bit a seconda dell'architettura con cui lo si confronta.

Architetture a confronto: numero di MOS

Sommatore	8 bit	16 bit	32bit	64bit	N bit
Ripple carry	192	384	768	1536	24N
Carry by-pass	232	464	928	1856	29N
Linear Carry Select	336	672	1344	2688	42N
Square Root Carry Sel.	336	672	1344	2688	42N



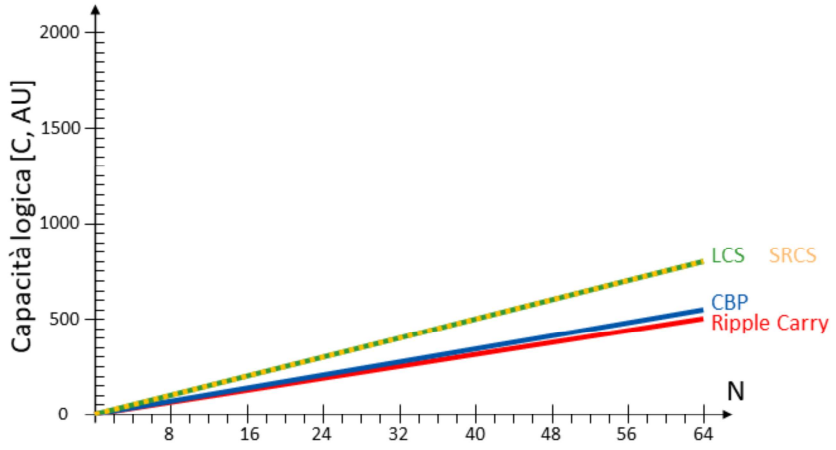
Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 51

Architetture a confronto: numero di MOS

In termini di numero di MOS il sommatore SRCS è identico all'architettura LCS. Infatti, l'hardware aggiuntivo e la struttura sono identiche. L'unica variazione è il modo con cui si raggruppano i bit.

Architetture a confronto: consumo/capacità logica

Sommatore	8 bit	16 bit	32bit	64bit	N bit
Ripple carry	62C	124C	248C	486C	8N·C
Carry by-pass	67C	134C	268C	526C	8.625N·C
Linear Carry Select	102C	204C	408C	816C	13N·C
Square Root Carry Sel.	102C	204C	408C	816C	13N·C



Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 52

Architetture a confronto: consumo dinamico / capacità logica

Anche in termini di capacità logica, ovviamente, la situazione è identica all'architettura LCS.

SOMMATORE CARRY LOOKAHEAD

Concetto base

- I segnali P_n & G_n relativi al bit n possono essere calcolati da A_n e B_n mediante funzioni logiche elementari AND e XOR entro il tempo di propagazione di una porta logica.
- In generale, il riporto C_n è funzione di tutti i riporti precedenti: C_{IN} C_0, C_1, … C_{n-1}.
- Quindi, C_n è funzione di C_{IN}, P_0, P_1, …P_n, G_0, G_1, …G_n.
- È possibile calcolare C_n a partire da C_{IN} P_0-P_n, G_0-G_n, mediante un singolo stadio logico, senza dover attendere il calcolo in sequenza di tutti i riporti precedenti?

Sì, ma serve una porta logica a 2n+1 ingressi!

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 54

Sommatore Carry Look Ahead (CLA)

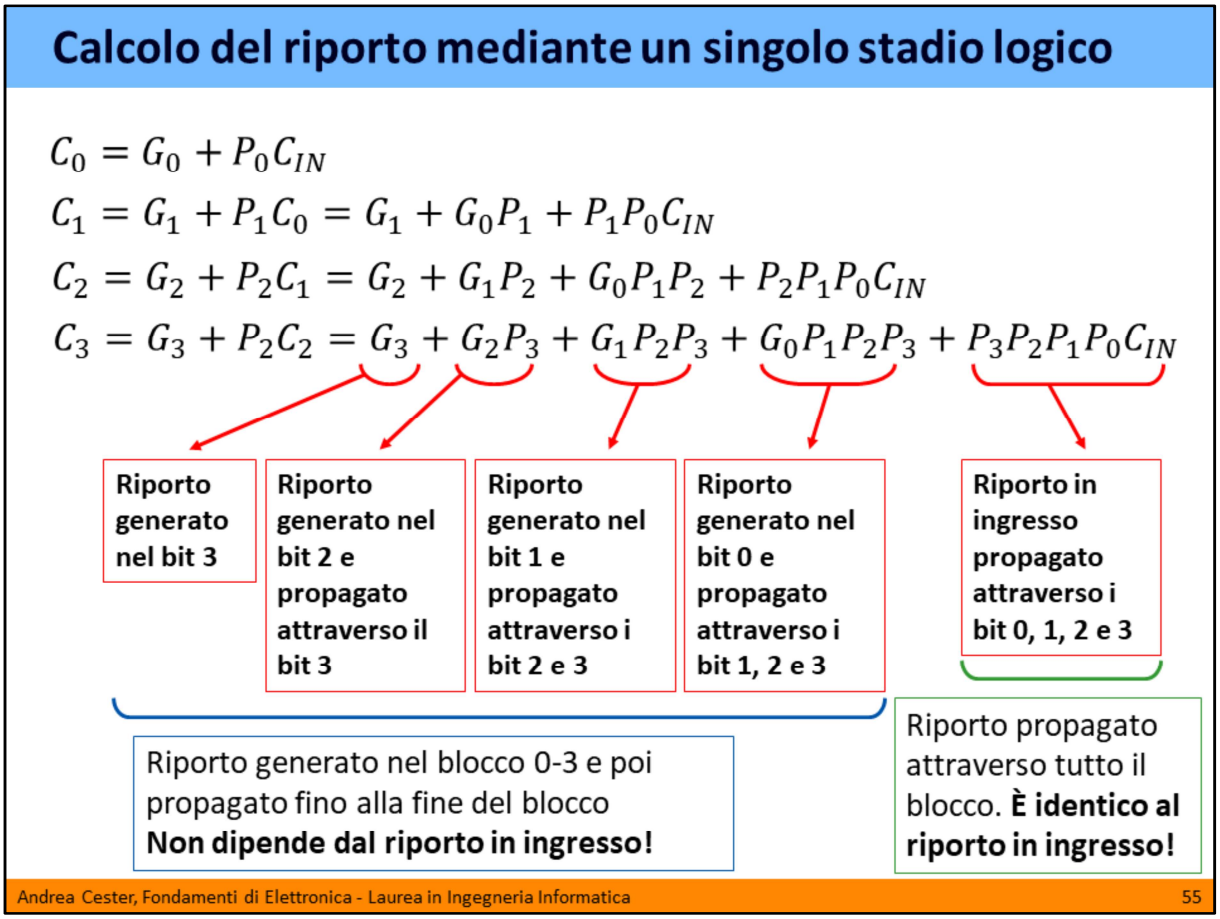
Concetto base

Abbiamo visto che la somma di N bit può essere divisa in due fasi.
1) Il calcolo dei segnali ausiliari P e G
2) Il calcolo dei riporti e delle somme a partire da P e G.

Nella prima fase, i segnali P_n e G_n relativi al bit n possono essere calcolati da A_n e B_n mediante funzioni logiche elementari AND e XOR entro il tempo di propagazione di una porta logica e non necessitano di conoscere il riporto di ingresso. Ciò permette di essere calcolati tutti in parallelo una volta disponibili le due stringhe di bit A e B.

Nella seconda fase, il riporto è calcolato in modo sequenziale. In generale, il riporto C_n è funzione di tutti i riporti precedenti: C_{IN} C_0, C_1, … C_{n-1}.
Quindi, C_n è funzione di C_{IN}, P_0, P_1, …P_n, G_0, G_1, …G_n.

La domanda che ci poniamo ora è: è possibile calcolare C_n a partire da C_{IN} P_0-P_n, G_0-G_n, mediante un singolo stadio logico, senza dover attendere il calcolo in sequenza di tutti i riporti precedenti?



Calcolo di C_{OUT} mediante un singolo stadio

Consideriamo per semplicità un sommatore a 4 bit e calcoliamo i quattro riporti C_0, C_1, C_2, e C_3 in funzione dei corrispondenti segnali P e G e del riporto di ingresso C_{IN}.

$$C_0 = G_0 + P_0 C_{IN}$$
$$C_1 = G_1 + P_1 C_0 = G_1 + G_0 P_1 C_{IN} + P_0 P_1 C_{IN}$$
$$C_0 = G_2 + P_2 C_1 = G_2 + G_1 P_2 C_{IN} + G_0 P_1 P_2 C_{IN} + P_0 P_1 P_2 C_{IN}$$
$$C_3 = G_3 + P_3 C_2 = G_3 + G_2 P_3 C_{IN} + G_1 P_2 P_3 C_{IN} + G_0 P_1 P_2 P_3 C_{IN} + P_0 P_1 P_2 P_3 C_{IN}$$

Il riporto C_{OUT} coincide con C_3 ed è dato dalla OR di 5 contributi:

G_3: il riporto viene generato a 1 nel bit 3 (il più significativo)
G_2P_3: il riporto viene generato a 1 nel bit 2 e propagato attraverso in bit 3
$G_1P_2P_3$: il riporto viene generato a 1 nel bit 1 e propagato attraverso i bit 2 e 3
$G_0P_1P_2P_3$: il riporto viene generato a 1 nel bit 0 e propagato attraverso i bit 1, 2 e 3
$P_0P_1P_2P_3C_{IN}$: il riporto viene propagato dal bit 0 al bit 3.

Osserviamo che nei primi quattro casi il riporto è generato in uno dei quattro bit e propagato fino all'uscita, indipendentemente dal valore di C_{IN}. Nell'ultimo caso tutti i bit sono in condizione di propagazione e il riporto di uscita è uguale a quello di ingresso.

Segnali GENERATE e PROPAGATE di blocco

$$C_0 = G_0 + P_0 C_{IN}$$
$$C_1 = G_1 + P_1 C_0 = G_1 + G_0 P_1 + P_1 P_0 C_{IN}$$
$$C_2 = G_2 + P_2 C_1 = G_2 + G_1 P_2 + G_0 P_1 P_2 + P_2 P_1 P_0 C_{IN}$$
$$C_3 = G_3 + P_2 C_2 = G_3 + G_2 P_3 + G_1 P_2 P_3 + G_0 P_1 P_2 P_3 + P_3 P_2 P_1 P_0 C_{IN}$$

Segnale **generate di blocco**

$$G_{0:3} = G_3 + G_2 P_3 + G_1 P_2 P_3 + G_0 P_1 P_2 P_3$$

N.B. $G_{0:3}$ e $P_{0:3}$ dipendono solo da: $A_0, A_1, A_2, A_3, B_0, B_1, B_2, B_3$ come I segnali G_n e P_n del bit n-esimo

Segnale **propagate di blocco**

$$P_{0:3} = P_3 P_2 P_1 P_0$$

$$C_{OUT} = G_{0:3} + P_{0:3} C_{IN}$$

Formula identica alla generazione e propagazione del bit n-esimo:

$$C_{n+1} = G_n + P_n C_n$$

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 56

Segnali GENERATE e PROPAGATE di blocco

Definiamo i seguenti segnali:

$$G_{0:3} = G_3 + G_2 P_3 + G_1 P_2 P_3 + G_0 P_1 P_2 P_3$$

e

$$P_{0:3} = P_3 P_2 P_1 P_0$$

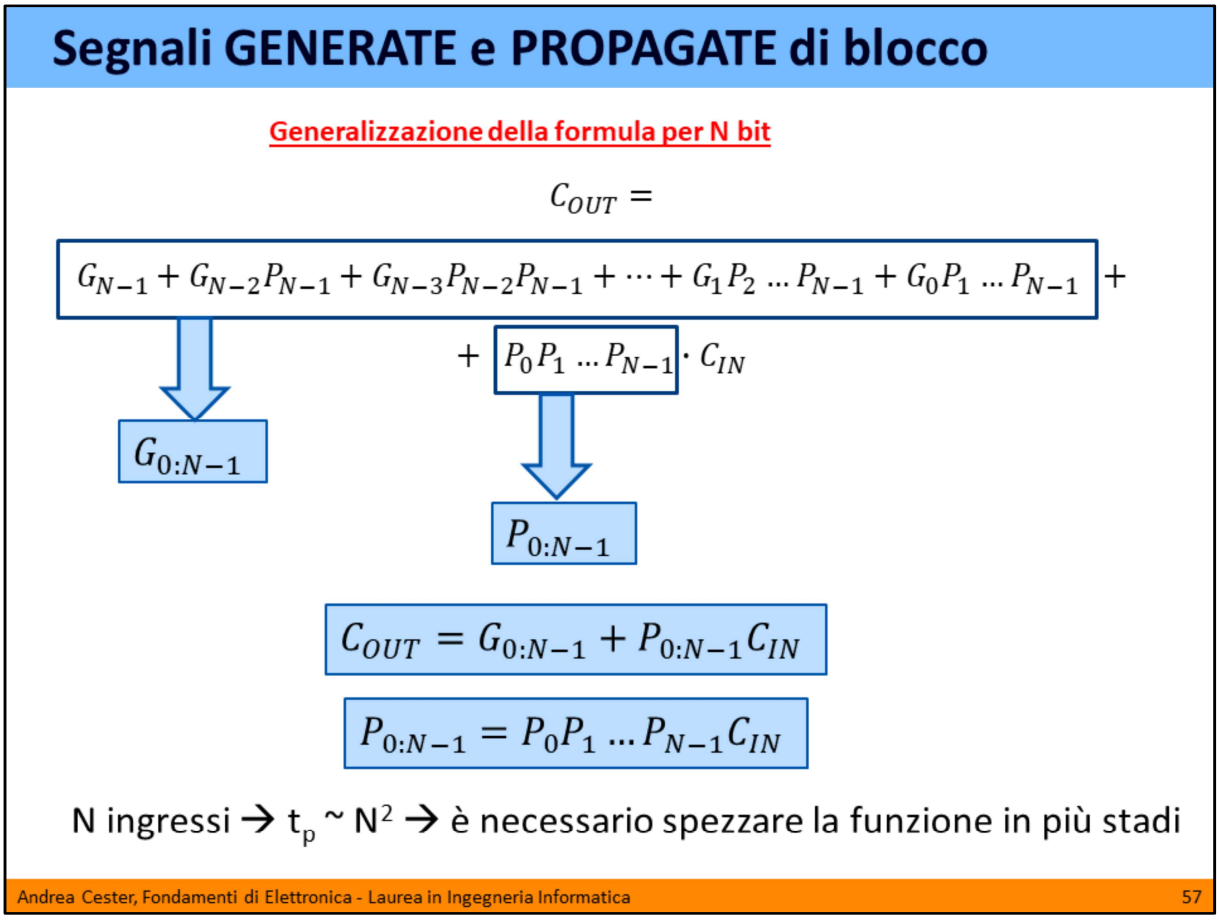
Il riporto di uscita può essere riscritto come:

$$C_{OUT} = G_{0:3} + P_{0:3} C_{IN}$$

$G_{0:3}$ rappresenta la OR dei quattro casi in cui il riporto viene generato in uno dei quattro bit e propagato fino alla fine. Questa condizione equivale a un segnale GENERATE relativo al blocco, in cui il riporto è generato a 1 in un qualunque bit e propagato fino alla fine. C_{OUT} sarà 1 indipendentemente da C_{IN}.

$P_{0:3}$ rappresenta la AND di tutti i segnali di propagate, ed equivale alla condizione di bypass del sommatore CBP. $P_{0:3}$ identifica una condizione di PROPAGATE relativa all'intero blocco, in cui $C_{OUT} = C_{IN}$.

Osserviamo che $P_{0:3}$ e $G_{0:3}$ dipendono solo dai segnali di ingresso A e B e non dai riporti.



Estendiamo il ragionamento al caso di N bit. Il riporto di uscita risulta:

$$C_{OUT} = G_{N-1} + G_{N-2}P_{N-1} + G_{N-3}P_{N-2}P_{N-1} + \cdots + G_1P_2 \ldots P_{N-1} + G_0P_1 \ldots P_{N-1} + \\ + P_0P_1 \ldots P_{N-1}C_{IN}$$

Se definiamo i segnali:

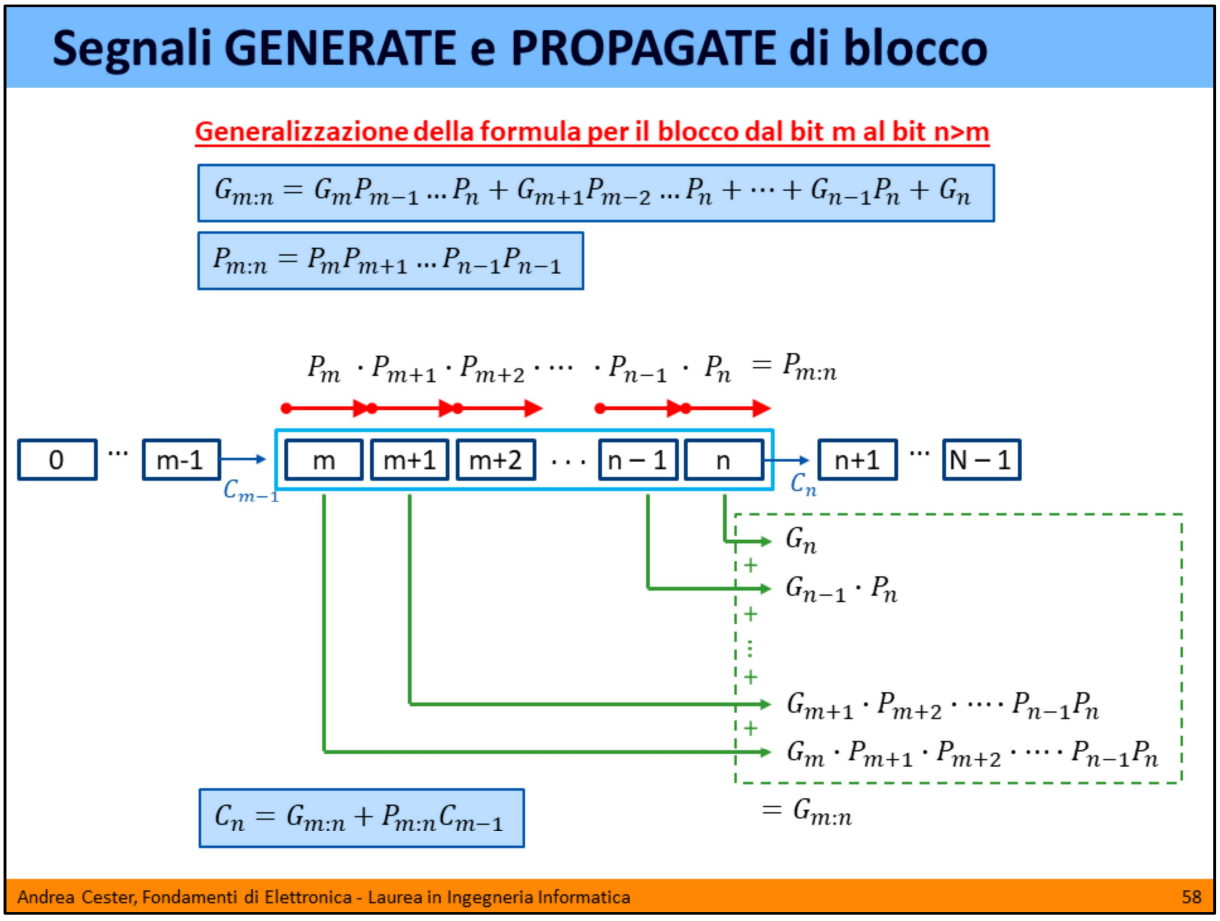
$$G_{0:N-1} = G_{N-1} + G_{N-2}P_{N-1} + G_{N-3}P_{N-2}P_{N-1} + \cdots + G_1P_2 \ldots P_{N-1} + G_0P_1 \ldots P_{N-1}$$

$$P_{0:N-1} = P_0P_1 \ldots P_{N-1}$$

Il riporto finale risulta:

$$C_{OUT} = G_{0:N-1} + P_{0:N}C_{IN}$$

In linea di principio è possibile calcolare i segnali P e G di blocco con un singolo stadio, ma la funzione richiede 2N ingressi. È facile dimostrare che la realizzazione della funzione in logica statica complementare richiede circa N transistor in serie, quindi il tempo di propagazione aumenta con N^2, che è peggiore del semplice ripple carry.



La relazione che ci fornisce i segnali $G_{0:N-1}$ e $P_{0:N-1}$ può essere ulteriormente generalizzata per un blocco di bit che va dal bit *m* al bit *n*. Noto il riporto C_{m-1}, scriviamo i riporti precedenti come:

$$C_m = G_m + P_m C_{m-1}$$
$$C_{m+1} = G_{m+1} + P_{m+1} C_m = G_{m+1} + G_m P_{m+1} + P_m P_{m+1} C_{m-1}$$
$$C_{m+2} = G_{m+2} + P_{m+2} C_{m+1} = G_{m+2} + G_{m+1} P_{m+2} + G_m P_{m+1} P_{m+2} + P_m P_{m+1} P_{m+2} C_{m-1}$$
$$\dots$$
$$C_n = G_n + P_n C_{n-1} = G_n + G_{n-1} P_n + \dots + G_m P_{m+1} \dots P_n + P_m P_{m+1} \dots P_n C_{m-1}$$

Indichiamo i segnali di PROPAGATE e GENERATE del blocco m:n come:

$$G_{m:n} = G_n + G_{n-1} P_n + G_{n-2} P_{n-1} P_n + \dots + G_m P_{m+1} \dots P_n$$
$$P_{n:m} = P_m P_{m+1} \dots P_{n-1} P_n$$

Il segnale di PROPAGATE del blocco m:n è dato dalla OR (somma) di tutti i segnali di GENERATE dei singoli bit (G_k, con k = m,…,n) ciascuno messo in AND con tutti i segnali di PROPAGATE dei bit successivi (P_i con i = k+1,…,n).
Il segnale di PROPAGATE dei blocco è la AND di tutti i segnali di PROPAGATE dei singoli bit.

Il riporto di uscita del blocco è ricavato dal riporto di ingresso da una forma analoga a quella usata per un singolo bit.

$$C_n = G_{m:n} + P_{m:n} C_{m-1}$$

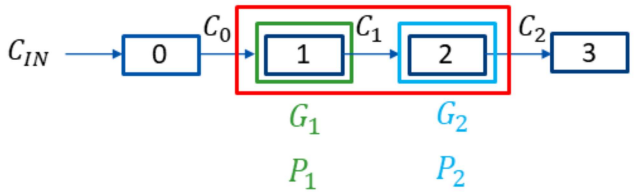
Con questa generalizzazione, i segnali di PROPAGATE e GENERATE di un singolo bit possono essere visti come segnali di PROPAGATE e GENERATE di un blocco costituito da 1 bit:

$$G_{n:n} = G_n \quad e \quad P_{n:n} = P_n$$

Combinazione dei segnali P&G di bit

- Esempio 1: supponiamo noti:
  - I segnali P e G relativi al bit 1 (P_1 e G_1)
  - I segnali P e G relativi al bit 2 (P_2 e G_2)
- Calcoliamo i segnali P e G di blocco $P_{1:2}$ e $G_{1:2}$ tali che, noto C_0, scriviamo:

$$C_2 = G_{1:2} + P_{0:1}C_0$$



Riporto $C_2 = G_2 + P_2C_1 = G_2 + P_2(G_1 + P_1C_0)$

$$C_2 = \underbrace{G_2 + G_1P_2}_{G_{1:2} = G_2 + G_1P_2} + \underbrace{P_2P_1}_{P_{1:2} = P_1P_2}C_0$$

$$G_{1:2} = G_1 + G_2P_1$$
$$P_{1:2} = P_1P_2$$

Calcolo dei segnali P&G di blocco a partire dai segnali P&G di bit

I segnali P e G di bit consecutivi possono essere assemblati per costruire la coppia P e G di blocco. Ad esempio, noti P_n, C_n, P_{n+1} e G_{n+1}, scriviamo:

$$C_{n+1} = G_{n+1} + P_{n+1}C_n = G_{n+1} + P_{n+1}(G_n + P_nC_{n-1}) = (G_{n+1} + G_nP_{n+1}) + (P_nP_{n+1})C_{n-1}$$

I segnali relativi al blocco n:n+1 si ricavano da quelli dei singoli bit:

$$P_{n:n+1} = G_{n+1} + G_nP_{n+1}$$
$$C_{n:n+1} = P_nP_{n+1}$$

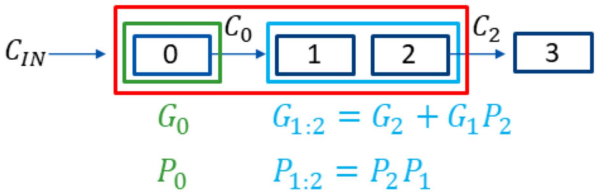
Ricordando che il singolo bit può essere visto come un blocco lungo 1 bit, possiamo riscrivere le due relazioni come segue:

$$P_{n:n+1} = G_{n+1:n+1} + G_{n:n}P_{n+1:n+1}$$
$$P_{n:n+1} = P_{n:n}P_{n+1:n+1}$$

Combinazione dei segnali P&G di bit + blocco

- Esempio 2: supponiamo noti:
  - I segnali P e G relativi al bit 0 (P_0 e G_0)
  - I segnali P e G relativi al blocco (contiguo al bit 0) 1:2 ($P_{1:2}$, $G_{1:2}$)
- Calcoliamo i segnali P e G di blocco $P_{0:2}$ e $G_{0:2}$ tali che, noto C_{IN}, scriviamo:

$$C_2 = G_{0:2} + P_{0:2}C_{IN}$$



G_0 $\quad G_{1:2} = G_2 + G_1P_2$

P_0 $\quad P_{1:2} = P_2P_1$

Riporto $C_2 = G_{1:2} + P_{1:2}C_0 = G_2 + G_1P_2 + P_2P_1(G_0 + P_0C_{IN})$

$$C_2 = \underbrace{G_2 + G_1P_2 + G_0P_2P_1}_{G_{0:2} = G_{1:2} + G_0 \cdot P_{1:2}} + \underbrace{P_2P_1P_0}_{P_{0:2} = P_0 \cdot P_{1:2}}C_{IN}$$

$G_{0:2} = G_{1:2} + G_0P_{1:2}$

$P_{0:2} = P_0P_{1:2}$

Estensione di un blocco mediante aggiunta di un singolo bit

Dato il blocco m:n vogliamo estenderlo aggiungendo al blocco il bit m-1 (cioè quello precedente).
Il riporto di uscita dal blocco m:n è:

$$C_n = G_{m:n} + P_{m:n}C_{m-1}$$

Il riporto di ingresso al blocco m:n coincide con quello di uscita del bit m-1. Noti P_{m-1} e G_{m-1}, scriviamo dunque:

$$C_{m-1} = G_{m-1} + P_{m-1}C_{m-2}$$

Unendo le due relazioni:

$$C_n = G_{m:n} + P_{m:n}(G_{m-1} + P_{m-1}C_{m-2}) = G_{m:n} + G_{m-1}P_{m:n} + P_{m-1}P_{m:n}C_{m-2}$$

Introducendo i segnali G e P del blocco m-1:n, otteniamo:

$$C_n = G_{m-1:n} + P_{m-1:n}C_{m-2}$$

con:

$$G_{m-1:n} = G_{m:n} + G_{m-1}P_{m:n}$$
$$P_{m-1:n} = P_{m-1}P_{m:n}$$

I segnali P e G di blocco possono essere estesi aggiungendo un bit adiacente. Ricordando ancora una volta che il singolo bit può essere visto come un blocco lungo 1 bit, possiamo riscrivere le due relazioni come segue:

$$P_{m-1:n} = G_{m:n} + G_{m-1:m-1}P_{m:n}$$
$$P_{m-1:n} = P_{m-1:m-1}P_{m:n}$$

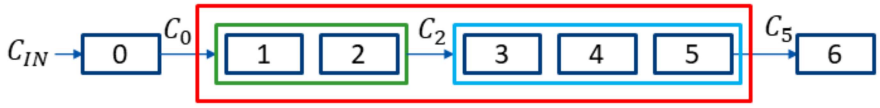
Con lo stesso procedimento è possibile dimostrare che un blocco può essere esteso aggiungendo il bit successivo:

$$P_{m:n+1} = G_{n+1:n+1} + G_{m:n}P_{n+1:n+1}$$
$$P_{m:n+1} = P_{m:n}P_{n+1:n+1}$$

Combinazione dei segnali P&G di blocco

- Esempio 3: supponiamo noti:
  - I segnali $P_{1:2}$ e $G_{1:2}$ relativi al blocco 1:2
  - I segnali $P_{3:5}$ e $G_{3:5}$ relativi al blocco 3:5 (contiguo a 1:2)
- Calcoliamo i segnali P e G di blocco $P_{1:5}$ e $G_{1:5}$ tali che, noto C_0 (riporto in ingresso a 1:2) scriviamo:

$$C_5 = G_{1:5} + P_{1:5}C_0$$



$G_{1:2} = G_2 + G_1P_2$ $\quad G_{3:5} = G_5 + G_4P_3 + G_3P_4P_5$

$P_{1:2} = P_2P_1$ $\quad P_{3:5} = P_5P_4P_3$

$$G_{1:5} = G_{3:5} + G_{1:2}P_{3:5}$$
$$P_{1:5} = P_{1:2}P_{3:5}$$

Riporto $C_2 = G_{1:2} + P_{1:2}C_0$

Riporto $C_5 = G_{3:5} + P_{3:5}C_2 = G_{3:5} + P_{3:5}(G_{1:2} + P_{1:2}C_0)$

$$C_5 = \underbrace{G_{3:5} + G_{1:2}P_{3:5}}_{G_5 + G_4P_3 + G_3P_4P_5 + G_2P_3P_4P_5 + G_1P_2P_3P_4P_5 = G_{1:5}} + \underbrace{P_{1:2}P_{3:5}}_{P_{3:5} = P_5P_4P_3P_2P_1 = P_{1:5}}C_0$$

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 61

Unione di blocchi adiacenti

Dato due blocchi adiacenti m:k e k+1:n è possibile unire i blocchi a partire dai rispettivi segnali P e G ($P_{m:k}$, $G_{n:k}$, $P_{k+1:n}$, $G_{k+1:n}$).
Il riporto di uscita dal bit n è:

$$C_n = G_{k+1:n} + P_{k+1:n}C_k$$

A sua volta il riporto C_k del bit k si calcola a partire da C_m:

$$C_k = G_{m:k} + P_{m:k}C_{m-1}$$

Unendo le due relazioni:

$$C_n = G_{k+1:n} + P_{k+1:n}(G_{m:k} + P_{m:k}C_{m-1}) = G_{k+1:n} + G_{m:k}P_{k+1:n} + P_{m:k}P_{k+1:n}C_{m-1}$$

Introducendo i segnali G e P del blocco m:n, otteniamo:

$$C_n = G_{m:n} + P_{m:n}C_{m-1}$$

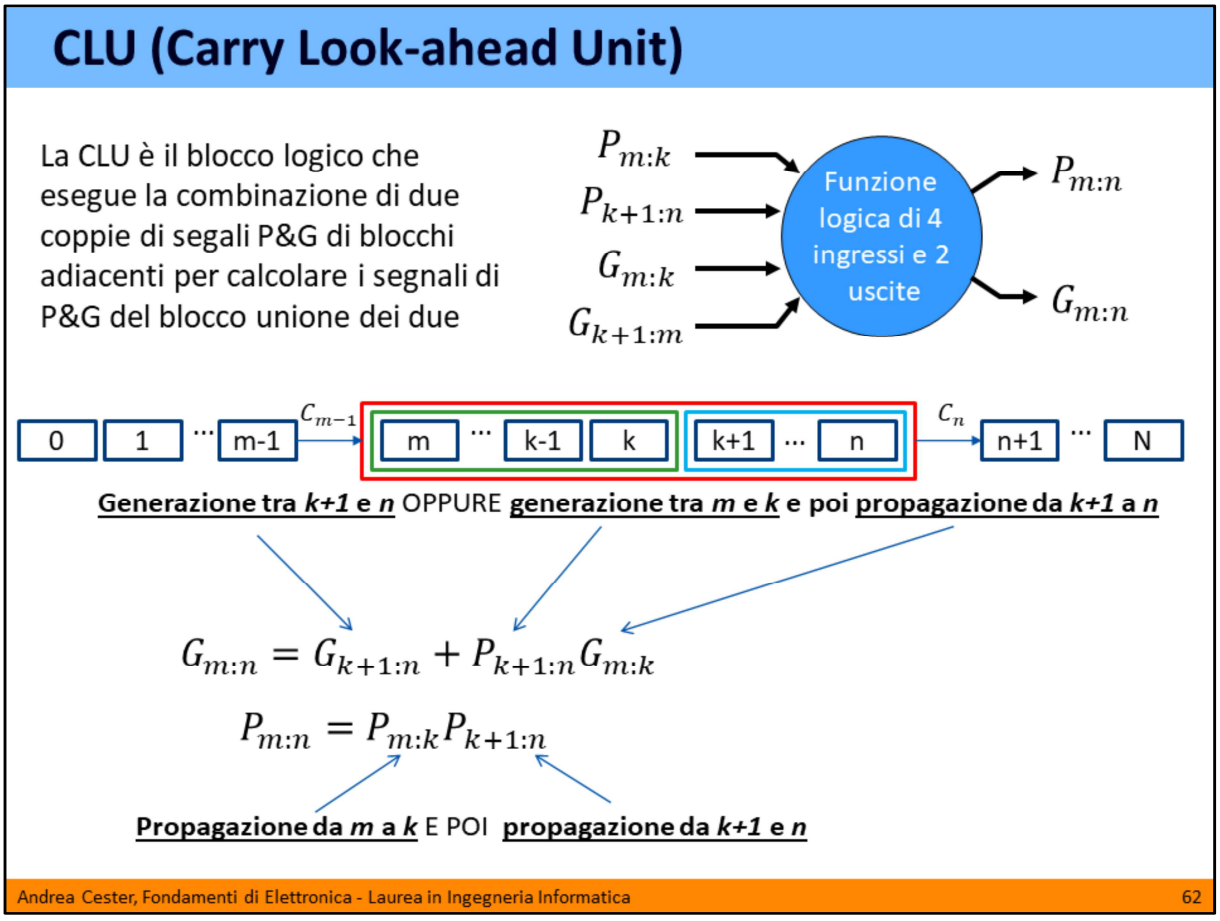
con:

$$G_{m:n} = G_{k+1:n} + G_{m:k}P_{k+1:n}$$
$$P_{m:n} = P_{m:k}P_{k+1:n}$$

I segnali P e G di blocco possono essere ottenuti dall'unione di due blocchi adiacenti.
Osserviamo che:

- Se m = k = n-1 otteniamo l'unione dei bit n-1 e n per formare il blocco n-1:n;
- Se m = k e n > k, otteniamo l'unione del bit m al blocco m+1:n;
- Se k > m e n = k+1, otteniamo l'unione del blocco m:n-1 con il bit n;
- Se k > m e n > k+1, otteniamo l'unione di due blocchi.

La coppia di relazioni scritte sopra rappresentano la forma generale che ci permette di accorpare bit o blocchi di bit adiacenti per formare blocchi più grandi.



Unità carry look-ahead (CLU, Carry Look-ahead Unit)

Tutto il ragionamento finora fatto si riassume nella seguente proposizione:

Consideriamo il blocco di bit dall'm-esimo all'n-esimo (m<n) e consideriamo un numero k compreso tra n e m. Il blocco m:k può essere spezzato in due sotto-blocchi adiacenti m:k e k+1:n. I segnali PROPAGATE e GENERATE del blocco m:n si ricavano in funzione dei segnali PROPAGATE e GENERATE dei due sotto-blocchi:

$$G_{m:n} = G_{k+1:n} + G_{m:k}P_{k+1:n}$$
$$P_{m:n} = P_{m:k}P_{k+1:n}$$

A livello logico-circuitale la coppia di segnali $G_{m:n}$ e $P_{m:n}$ si ricava come una funzione logica di 4 ingressi ($G_{k+1:n}$, $P_{k+1:n}$, $G_{m:k}$, $P_{m:k}$) e due uscite. Definiamo *Unità Carry Look-Ahead (CLU)* il circuito logico che realizza tale funzione.
Il ragionamento può essere facilmente esteso al caso in cui il blocco m:n venga suddiviso in 3 o più sotto-blocchi.
Ad esempio: consideriamo i 4 blocchi m:k, k+1:h, h+1:i, i+1:n

$$G_{m:n} = G_{i+1:n} + G_{h+1:i}P_{i+1:n} + G_{k+1:h}P_{h+1:i}P_{i+1:n} + G_{m:k}P_{k+1:h}P_{h+1:i}P_{i+1:n}$$
$$P_{m:n} = P_{m:k}P_{k+1:h}P_{h+1:i}P_{i+1:n}$$

Procedendo iterativamente in questo modo un blocco di molti bit può essere suddiviso in blocchi più piccoli, che a loro volta possono essere suddivisi in blocchi ancora più piccoli, fino ad arrivare a blocchi di 1 bit. Per i blocchi di 1 bit i segnali P e G coincidono con quelli dei singoli bit, che si ricavano dai segnali di ingresso:

$$G_{n:n} = G_n = A_nB_n$$
$$P_{n:n} = P_n = A_n \oplus B_n$$

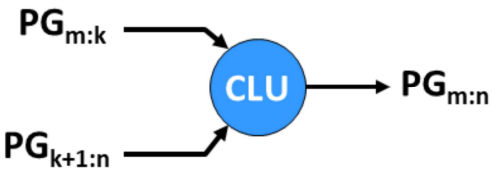
Notazione compatta per i segnali P e G di bit e blocco

Definiamo:

- PG_n la coppia di segnali logici (P_n, G_n). P_n e G_n sono i segnali primari di propagate e generate del singolo bit n
- $PG_{m:k}$ (k≥m) la coppia di segnale ($P_{m:k}$, $G_{m:k}$). $P_{m:k}$, $G_{m:k}$ sono i segnali di propagate e generate relativi al blocco di bit m:k

N.B. Se m=k, allora $PG_{m:m} = PG_m$ coincide con il vettore PG del bit m

Operatore CLU:



m ≤ k < n

$$PG_{m:n} = (P_{m:k}, G_{m:k})$$
$$PG_{k+1:n} = (P_{k+1:n}, G_{k+1:n})$$
$$PG_{m:n} = (P_{m:n}, G_{m:n})$$

$$G_{m:n} = G_{k+1:n} + P_{k+1:n}G_{m:k}$$
$$P_{m:n} = P_{m:k}P_{k+1:n}$$

Le funzioni logiche svolte dalla CLU sono le stesse indipendentemente dalla dimensioni dei blocchi o dal fatto che PG sia relativo a un bit o a un blocco di bit

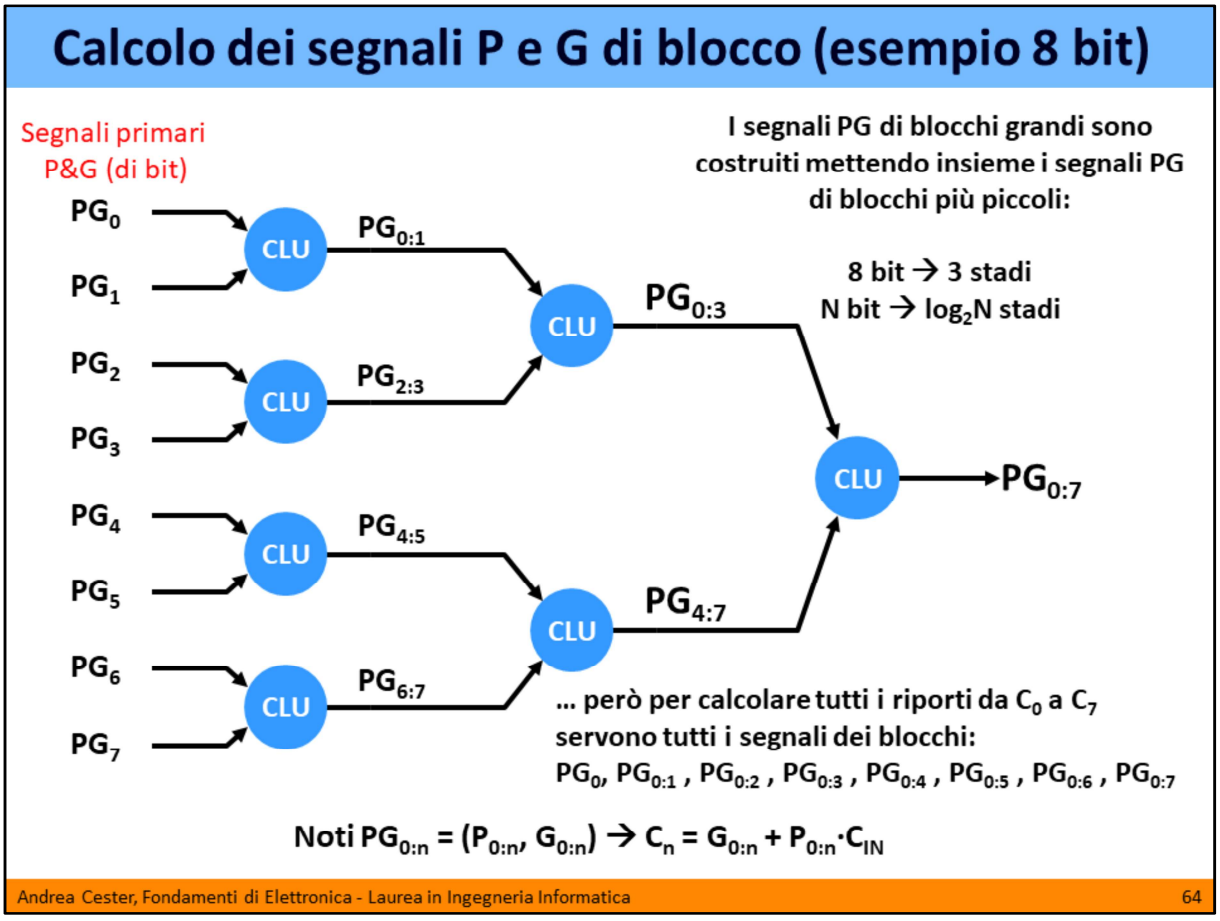
Unità carry look-ahead: rappresentazione compatta

Poiché i segnali P e G di blocco o di bit sono calcolati contemporaneamente e sono usati sempre in coppia, adottiamo una notazione più compatta indicando con:

- PG_n la coppia di segnali (P_n, G_n) relativa al singolo bit.
- $PG_{m:n}$ la coppia di segnali ($P_{m:n}$, $G_{m:n}$) relativa al blocco m:n. Ovviamente $PG_{n:n}$ coincide con PG_n.

L'analisi appena fatta ha dimostrato che l'unità CLU svolge sempre la stessa funzione logica indipendentemente dalle dimensioni dei blocchi di partenza e dalla posizione dei blocchi all'interno del vettore di bit (purché i blocchi siano adiacenti).

Nel seguito indicheremo l'operatore logico CLU mediante un cerchio che riceve in ingresso due coppie di segnali e produce in uscita una coppia di segnali.



Esempio: blocco da 8 bit suddiviso in blocchi di 2 bit usando due stadi di CLU

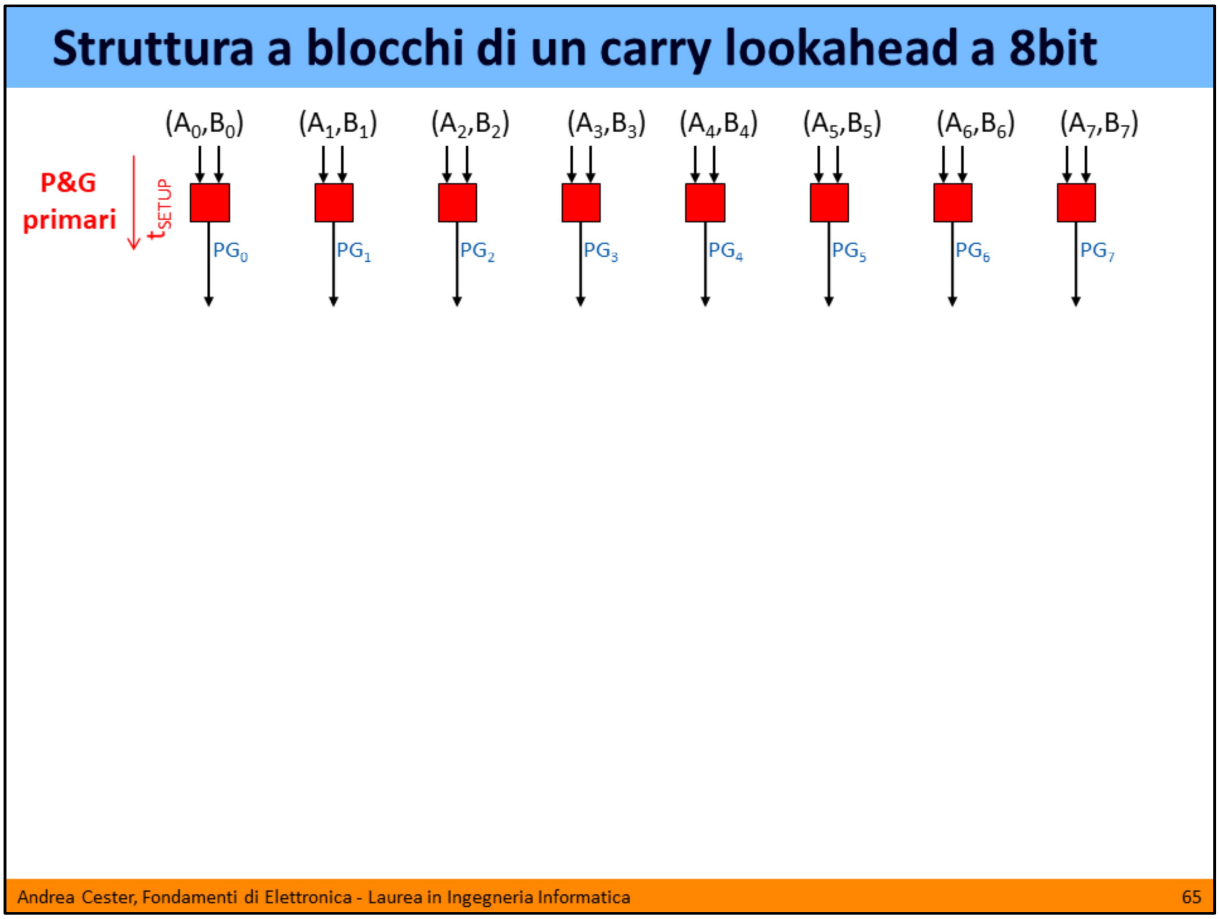
In questo esempio partiamo dai segnali PG_0, PG_0 , …, PG_7 calcolato dai soli ingressi A e B dal blocco di setup del sommatore.
I segnali PG di bit sono assemblati da 4 operatori CLU che lavorano in parallelo e producono i segnali $PG_{0:1}$, $PG_{2:3}$, $PG_{4:5}$ e $PG_{6:7}$ relativi a blocchi di 2 bit.
Il secondo stadio formato da 2 CLU che lavorano in parallelo tra loro assemblano i 4 gruppi di 2 bit in gruppi di 4 bit: $PG_{0:3}$ e $PG_{4:7}$.
L'ultimo stadio assembra i gruppi di 4 bit per ricavare il blocco da 8 bit:

$$\begin{bmatrix} G_{0:7} \\ P_{0:7} \end{bmatrix} = \begin{bmatrix} G_{4:7} + G_{0:3}P_{4:7} \\ P_{0:3}P_{4:7} \end{bmatrix}$$

Fatto ciò è possibile calcolare il riporto C_7:

$$C_7 = G_{0:7} + P_{0:7}C_{IN}$$

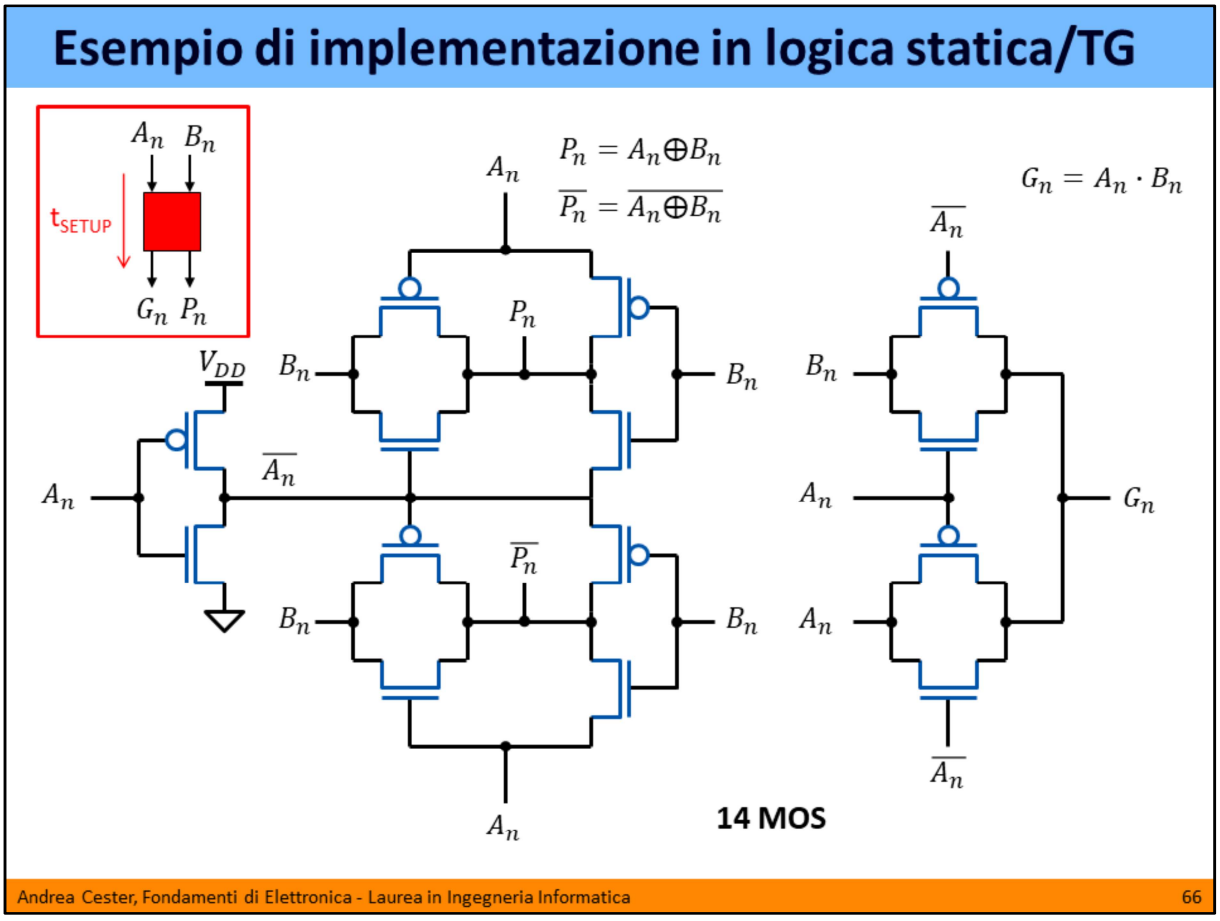
In generale per N bit sono richiesti logN stadi di CLU.
Bisogna fare attenzione che per calcolare i bit di somma S_0, S_1, … S_7 è necessario calcolare anche tutti i segnali $PG_{0:k}$ (k = 0, 1, …, 6).



Struttura a blocchi di un CLA binario a 8 bit: blocco di setup

Alla luce di tutte queste considerazione, analizziamo come è strutturato un sommatore carry look-ahead. Come per gli altre architetture di sommatore, mostreremo una possibile implementazione in logica statica complementare o a pass-T/TG a titolo di esempio per fare una stima del numero di MOS richiesto. Ove possibile, per semplicità, useremo gli stessi circuiti visti precedentemente.

Il primo stadio è come in tutte le altre architetture il blocco di setup che calcola i segnali P e G relativi a ciascun bit (ovvero blocchi di 1 bit).



Un esempio di calcolo di P e G è mostrato in figura ed è formato da tre porte logiche: XOR, XNOR e AND.

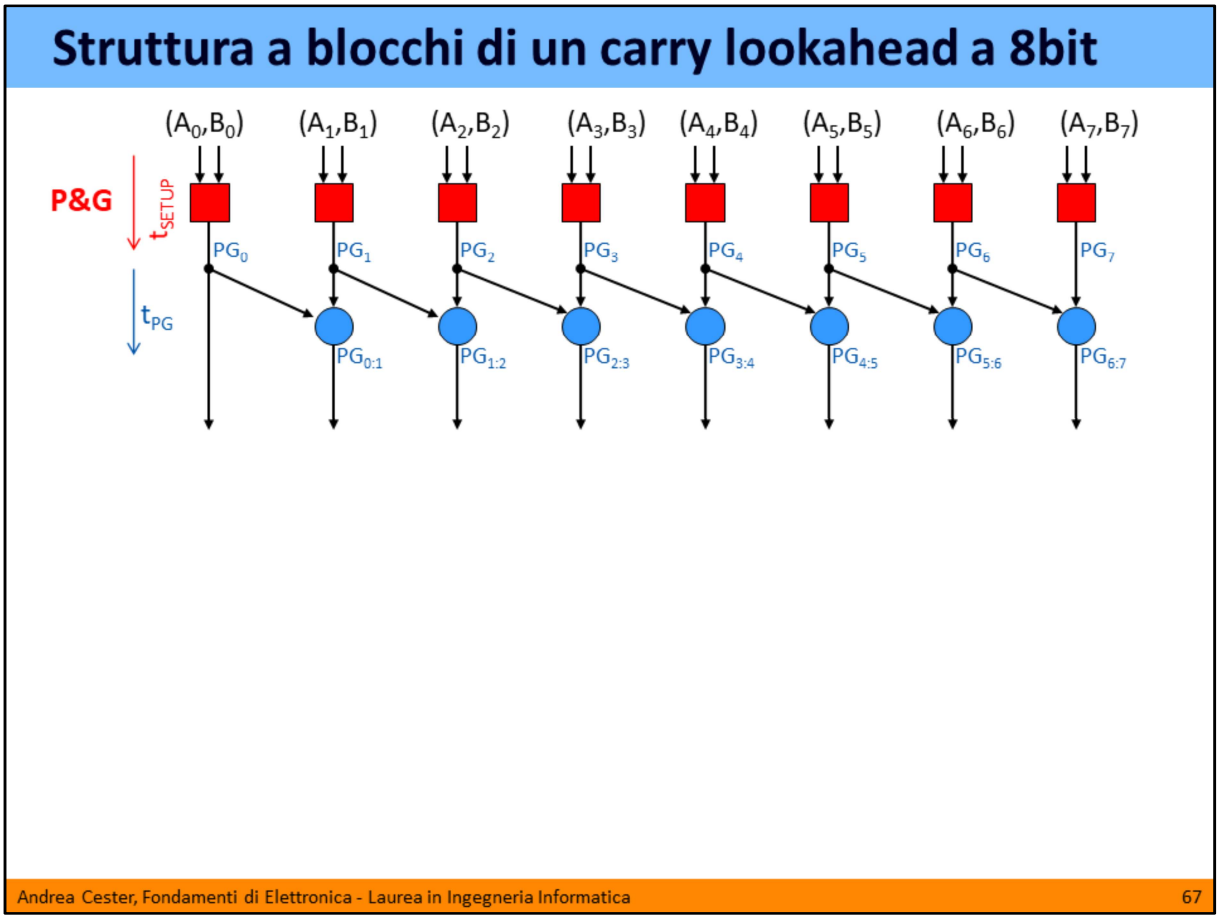
Alcune osservazioni:

- In questo stadio calcoliamo P in forma diretta e negata poiché ci servirà per generare la somma ($S_n = P_n \oplus C_{n-1}$)
- Il segnale $G_n = A_n B_n$ è realizzato mediante una coppia di TG senza invertitori poiché Gn sarà applicato ai gate del successivo stadio (in logica statica complementare, come vedremo). Non è indispensabile, quindi, che la porta AND sia rigenerativa. Usiamo A_n come segnale di controllo dei gate dei TG poiché è già disponibile in forma diretta e negata.

Il numero totale di MOS è 10 per la XOR/XNOR e 4 per la AND (14 MOS totali per blocco).

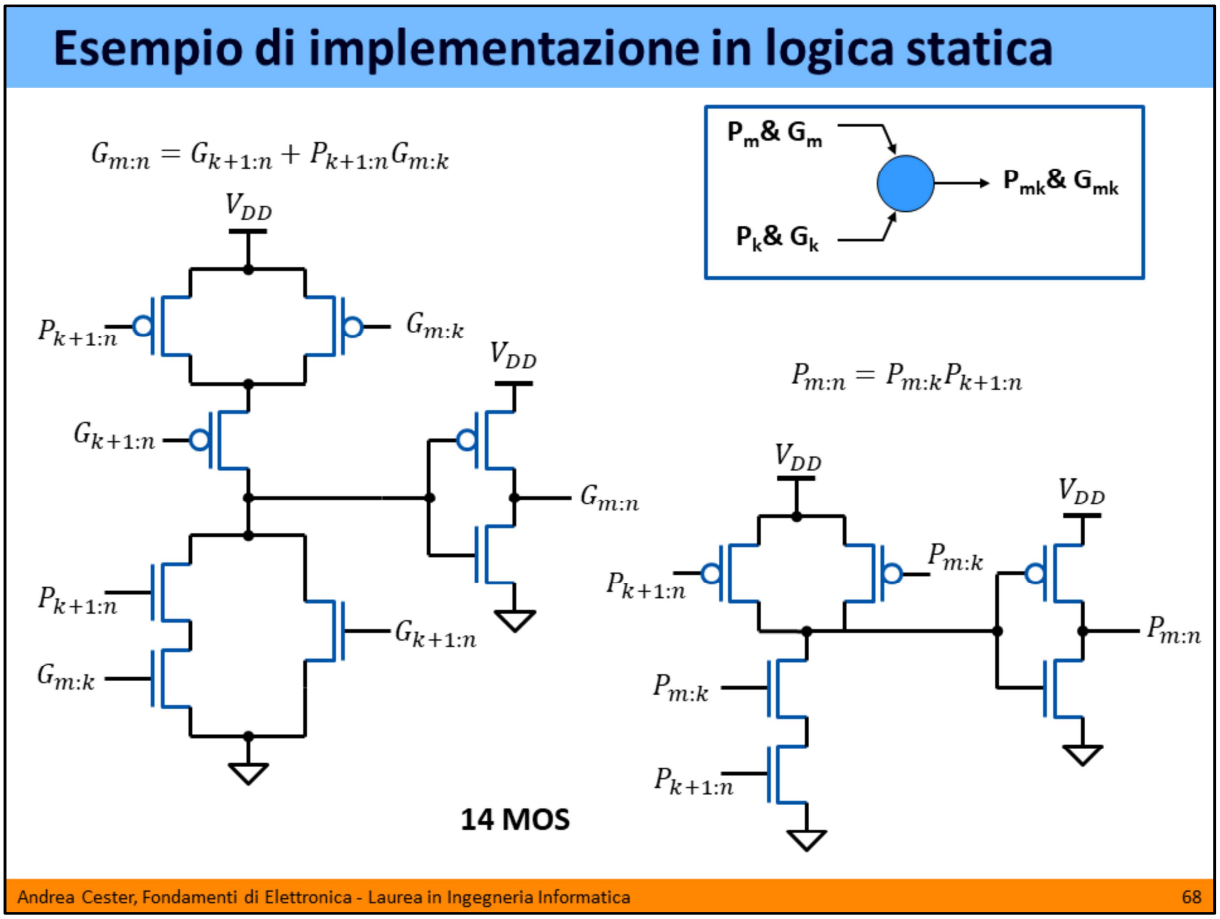
Sono necessari N blocchi (uno per bit).

Il numero totale di MOS nel blocco di setup del sommatore è 14N.

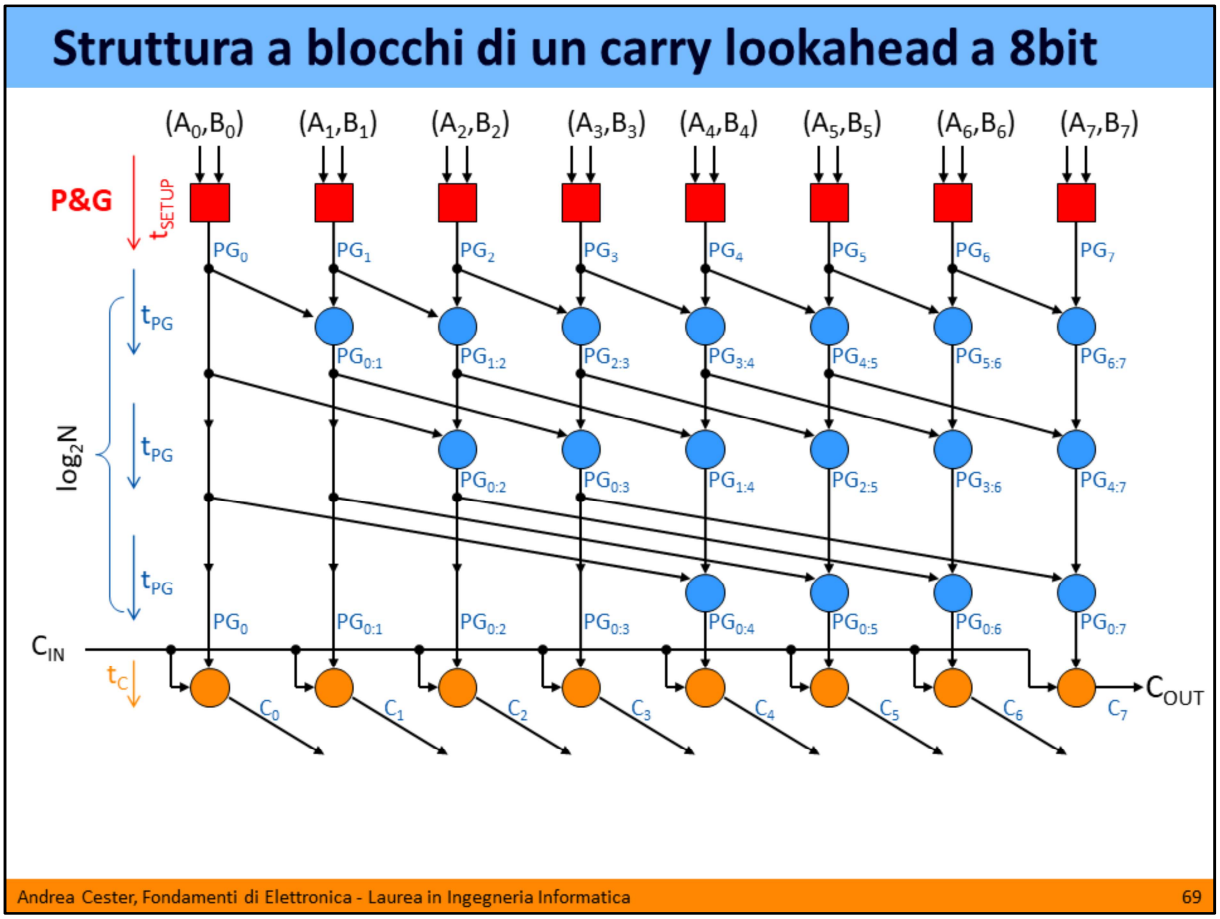


Struttura a blocchi di un CLA binario a 8 bit: unità CLU

In questo esempio vogliamo realizzare un CLA binario ovvero, raggruppando i bit a due a due e mettendo insieme i blocchi a due a due. Serviranno quindi CLU con due coppie di ingressi.
Il secondo stadio (primo stadio di CLU) mette insieme i singoli P&G per calcolare tutti i possibili blocchi di due bit: **0:1**, 1:2, 2:3, 3:4, 4:5, 5:6, 6:7 (sette in totale).
Per questo stadio servono in generale N – 1 blocchi di CLU.



Un blocco CLU è composto da due funzioni logiche: una da 2 ingressi e una da 3 ingressi.
Un esempio di implementazione è mostrato in figura usando logica statica complementare. In questo esempio sono richiesti 8 + 6 = 14 MOS.



Il terzo stadio (secondo stadio di CLU) mette insieme i blocchi da 1 o 2 bit per calcolare i segnali P&G relativi ai blocchi di 3 o 4 bit. In particolare:
- (4:5)+(6:7) → (4:7)
- (3:4)+(5:6) → (3:6)
- (2:3)+(4:5) → (2:5)
- (1:2)+(3:4) → (1:4)
- (0:1)+(2:3) → **(0:3)**
- (0)+(1:2) → **(0:2)**

In totale sono richieste N – 2 unità CLU.

Il quarto blocco assembla i blocchi di 1, 2, 3 o 4 bit per calcolare i blocchi da 8, 7, 6 o 5 bit:
- (0:3)+(4:7) → **(0:7)**
- (0:2)+(3:6) → **(0:6)**
- (0:1)+(2:5) → **(0:5)**
- (0)+(1:4) → **(0:4)**

In totale sono richieste N – 4 unità CLU

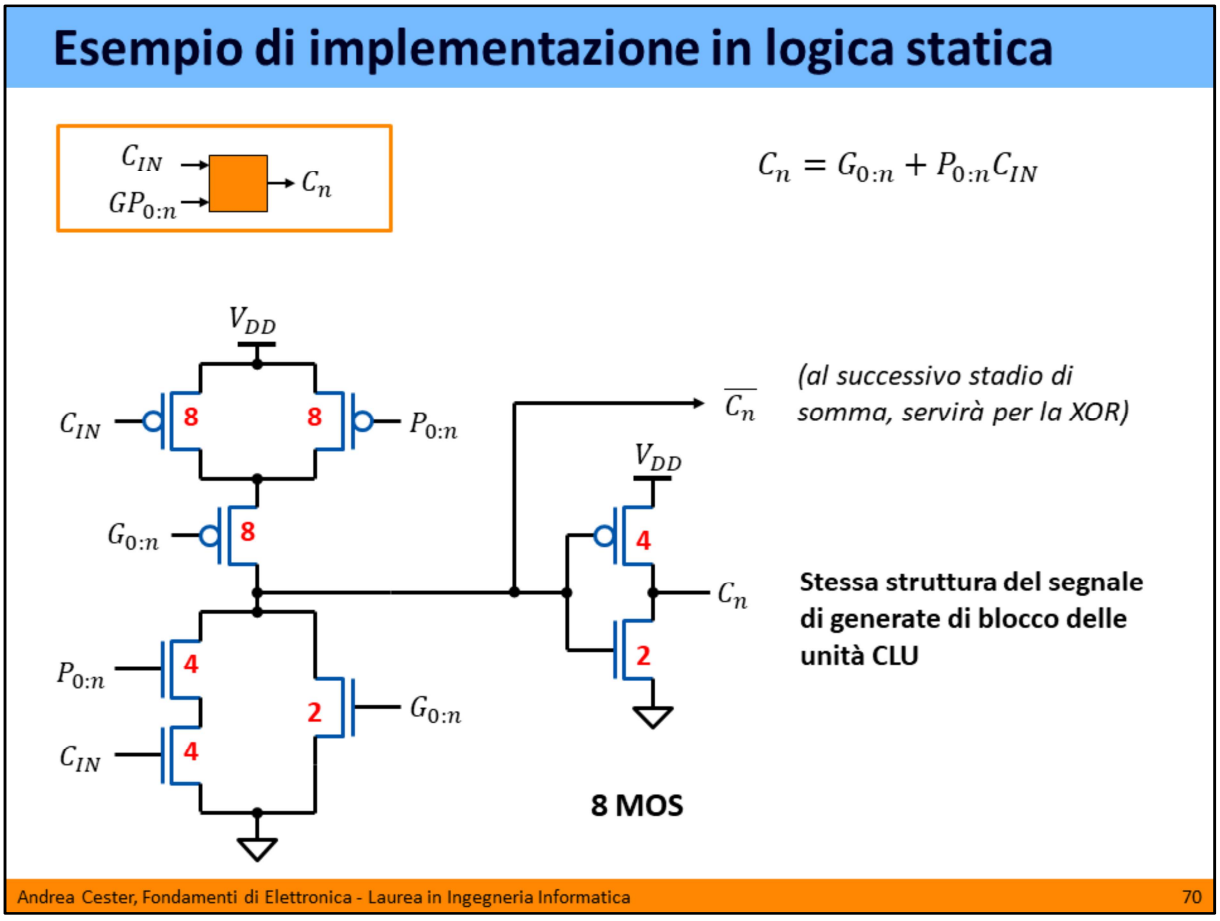
Dopo 3 (=$\log_2 N$) stadi di CLU abbiamo a disposizione i seguenti segnali P&G (0:n) con n = 0,1,…,7. Essi permettono il calcolo di tutti i riporti, a partire da C_{IN} (quinto stadio)

Nel complesso il numero di CLU è: (N – 1) + (N – 2) + (N – 4) + …

$$\sum_{i=1}^{\log N} (N - i) = N \cdot \log N - \frac{(1 + \log N)\log N}{2}$$

Il numero di MOS richiesti, con l'implementazione proposta, è dunque:

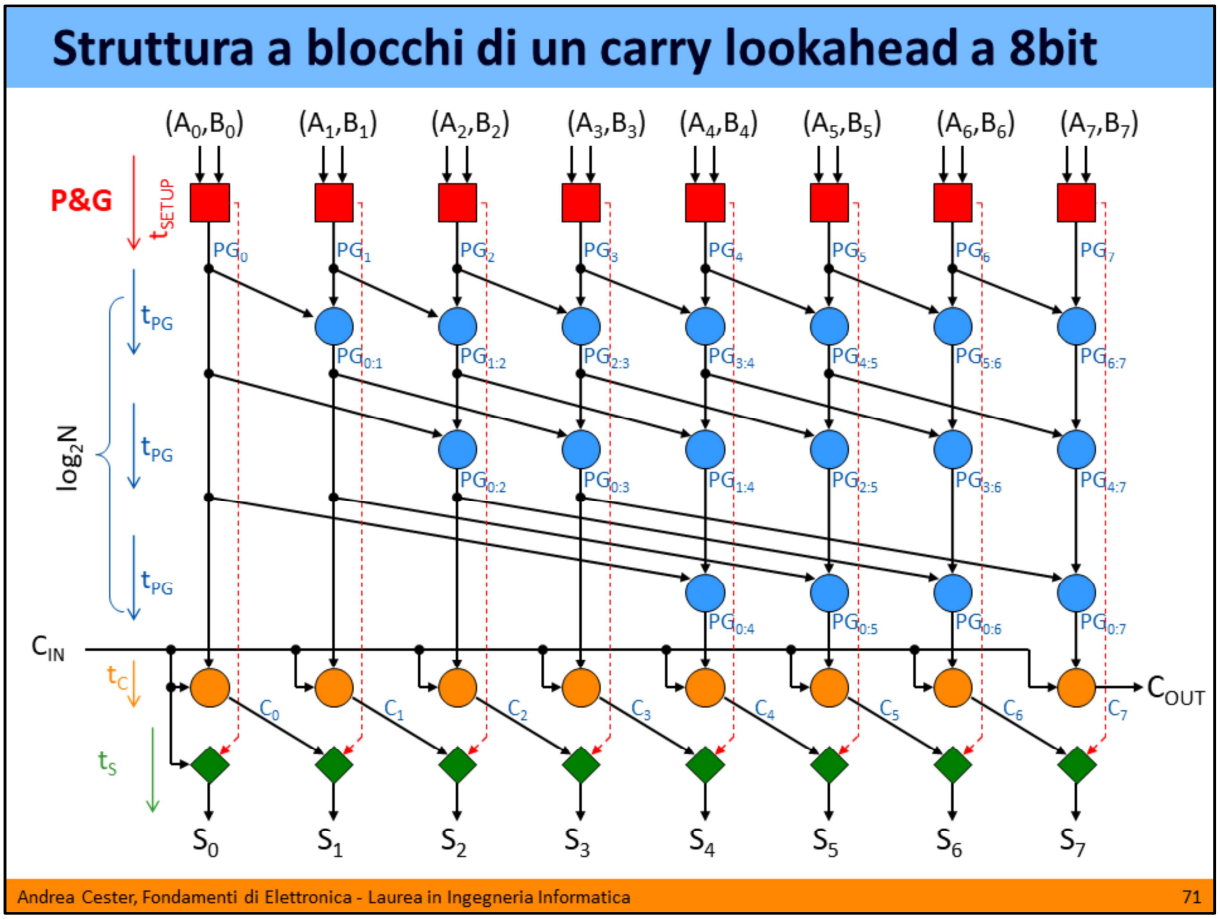
$$14N \cdot \log N - 7\,(1 + \log N)\log N$$



Per il calcolo del riporto possiamo usare una porta logica analoga a quella del segnale di generate di blocco:

$$C_n = G_{0:n} + P_{0:n}C_{IN}$$

Questa fornisce il riporto diretto e negato che ci servirà poi per la somma.
Il numero di MOS richiesti con questo tipo di implementazione è 8 per ciascun bit.
Essendo necessari N moduli per il calcolo del riporto, abbiamo in totale 8N MOS.

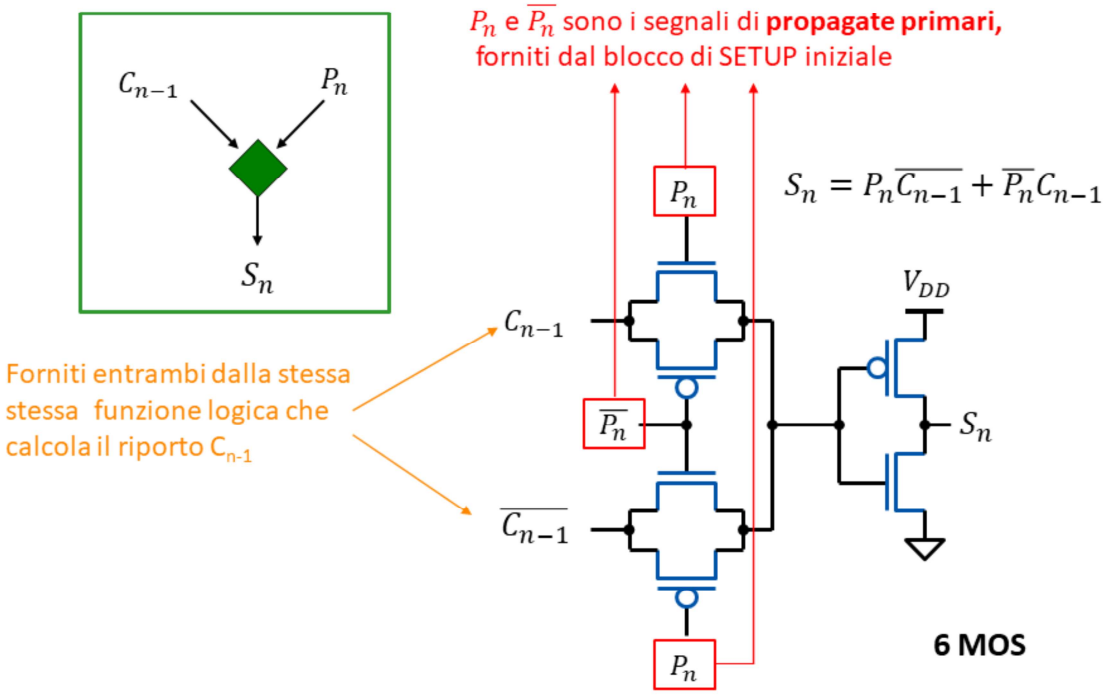


Struttura a blocchi di un CLA binario a 8 bit: somma

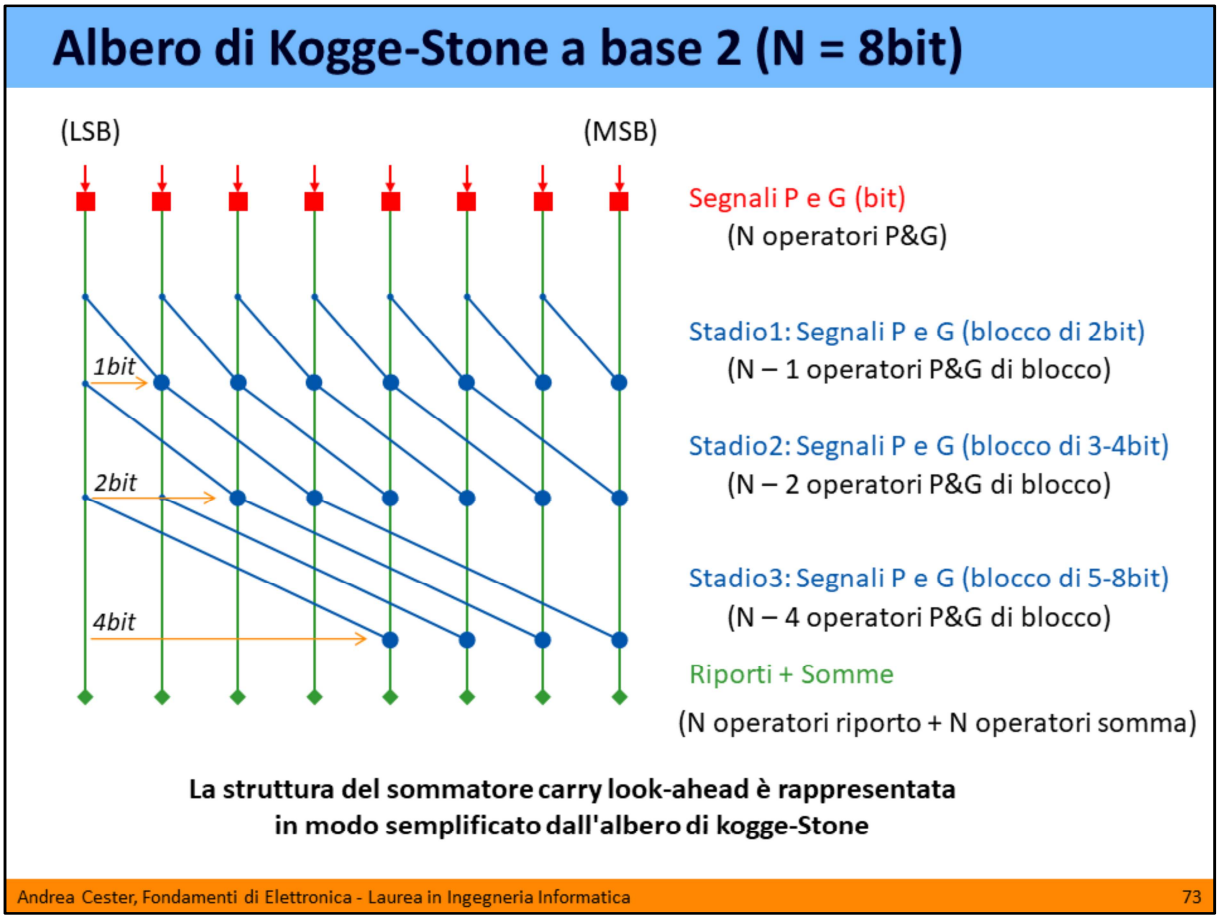
Infine, dai riporti parziali, il sesto stadio genera la somma.

È importante notare che il riporto di ingresso è usato SOLO nello stadio di riporto e di somma. In tutti gli stadi di CLU non viene mai usato. Questa è la principale differenza con tutte le altre architetture di somma, in cui il bit di riporto deve passare sequenzialmente (in modo più o meno ottimizzato) da uno stadio all'altro rallentando il calcolo. Il sommatore carry-lookahead *anticipa e calcola in parallelo* tutti i segnali ausiliari che permettono di calcolare immediatamente il riporto a partire da C_{IN}, eliminando la propagazione a cascata. Da questo deriva il nome *lookahead* ("*guardare avanti, anticipare*"). Il CLA è una delle architetture più veloci in assoluto, ampiamente utilizzato nei processori moderni e nei sistemi VLSI ad alte prestazioni.

Esempio di implementazione in logica statica



Ad esempio la somma può essere implementata a pass-T/TG con una porta XOR. In questo caso sono necessari N porte da 8 MOS ciascuna, tenendo conto che i segnali negati di P e C_{n-1} sono già disponibili dal blocco di riporto e di setup.

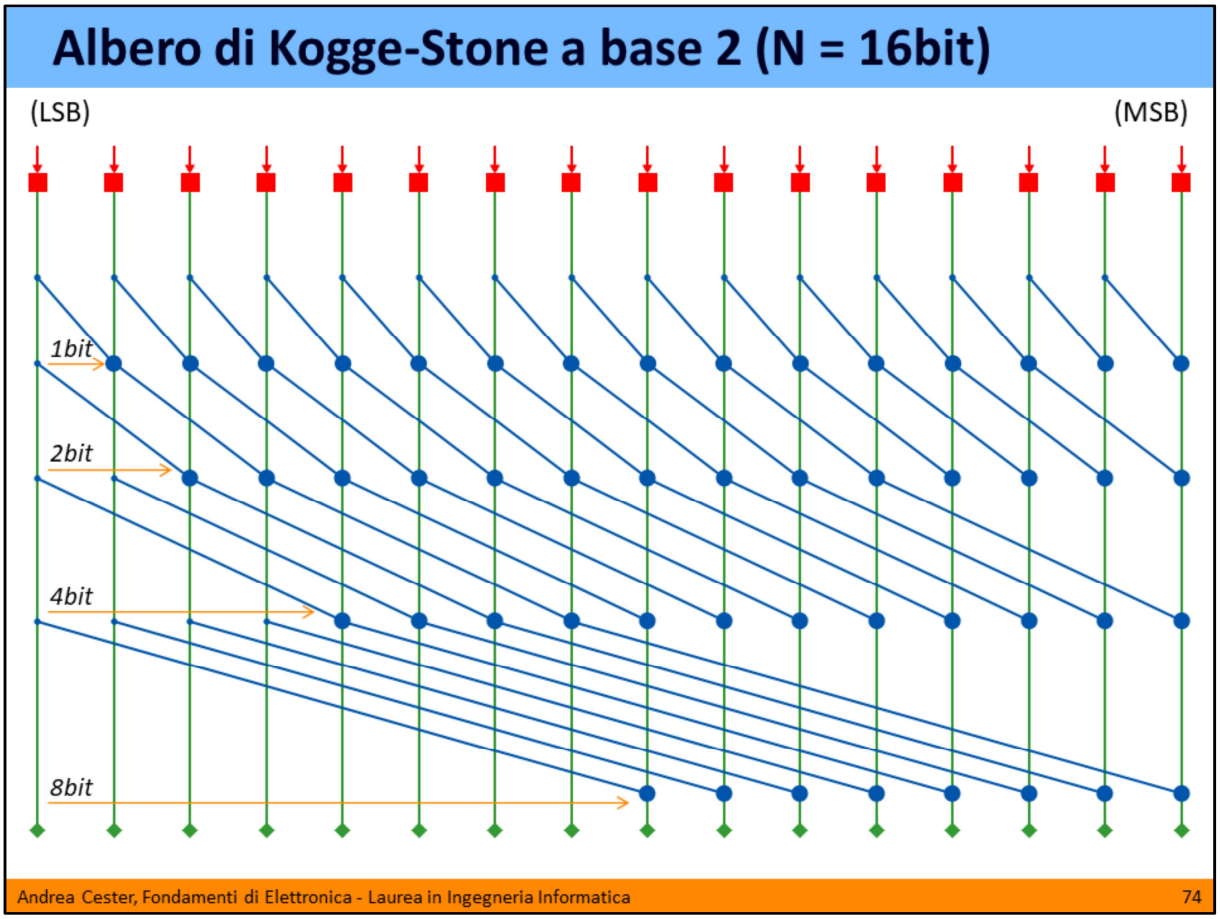


Rappresentazione del sommatore CLA mediante l'albero di Kogge-Stone

La struttura appena descritta viene chiamata albero di Kogge-Stone a base 2 ed è stata sviluppato da Peter M. Kogge e Harold S. Stone nel 1973, questo design ottimizza il calcolo del riporto riducendo al minimo il tempo di propagazione.

Tuttavia è richiesto molto hardware. Nel caso di un sommatore a N=8 bit essa impiega:

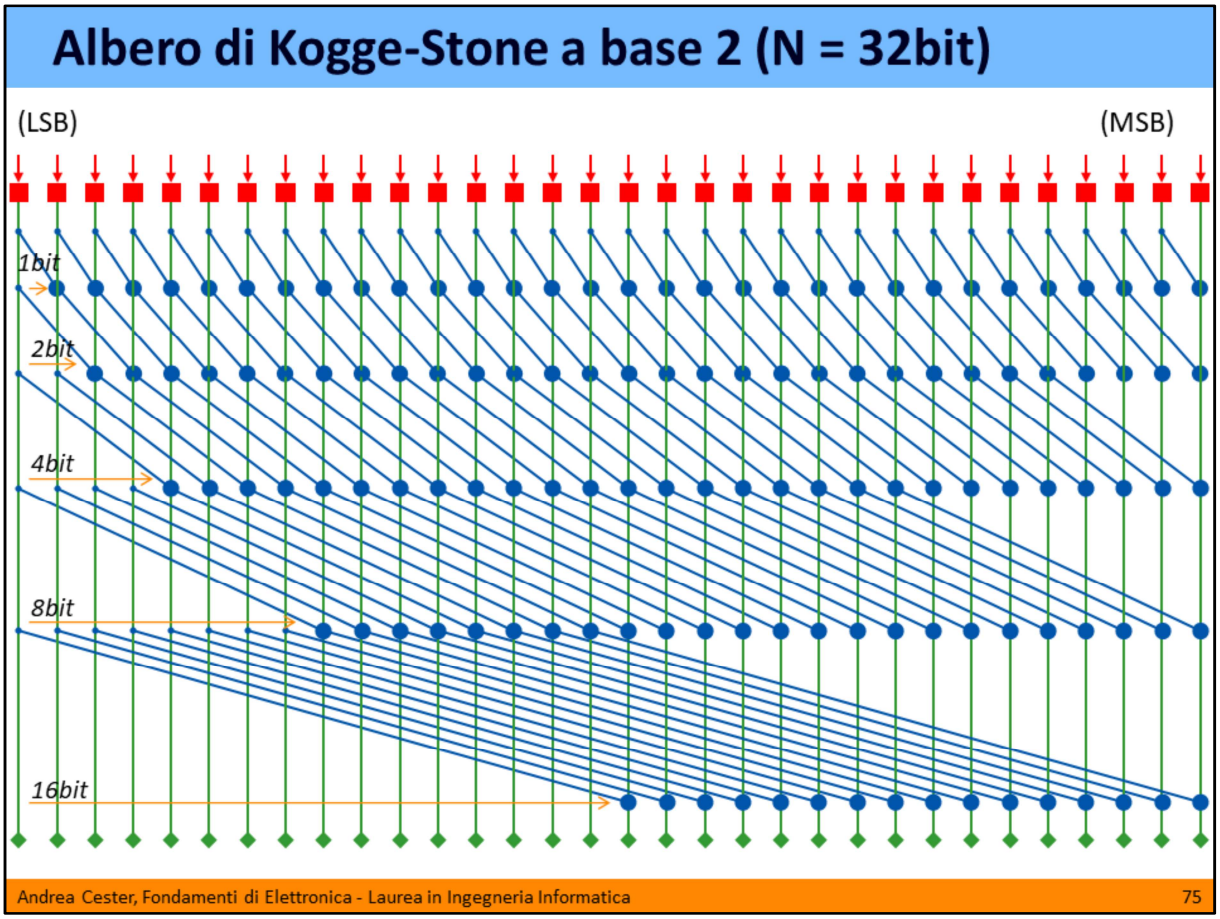
N=8 blocchi di operatori P&G dei singoli bit
N-1=7 blocchi di operatori P&G per blocchi di 2 bit
N-2=6 blocchi di operatori P&G per blocchi di 4 bit
N-4=4 blocchi di operatori P&G per blocchi di 8 bit
N=8 operatori per il calcolo del riporto
N=8 operatori per il calcolo della somma



La struttura può essere estesa ad un numero N arbitrario di bit. Ad esempio la figura mostra l'albero di kogge-stone a 16 bit
Rispetto al caso precedente esso ha uno stadio in più.
I quattro stadi di unità CLA hanno N-1, N-2, N-4, N-8 unità ciascuno.

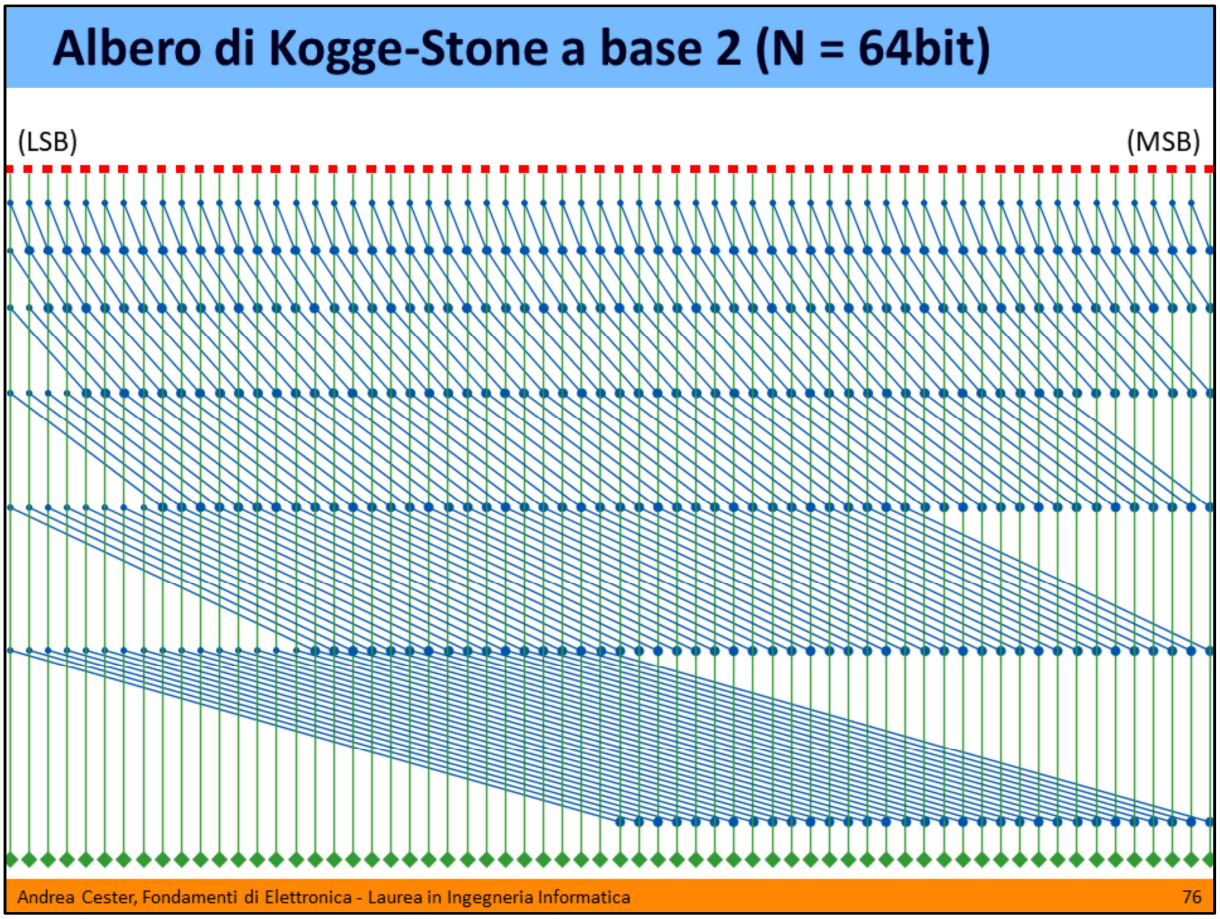
Nel caso di un sommatore a N=16 bit essa impiega:

N=16 blocchi di operatori P&G dei singoli bit
N-1=15 blocchi di operatori P&G per blocchi di 2 bit
N-2=14 blocchi di operatori P&G per blocchi di 4 bit
N-4=12 blocchi di operatori P&G per blocchi di 8 bit
N-8=8 blocchi di operatori P&G per blocchi di 16 bit
N=16 operatori per il calcolo del riporto
N=16 operatori per il calcolo della somma



Nel caso di un sommatore a N=32 bit essa impiega:

N=32 blocchi di operatori P&G dei singoli bit
N-1=31 blocchi di operatori P&G per blocchi di 2 bit
N-2=30 blocchi di operatori P&G per blocchi di 4 bit
N-4=28 blocchi di operatori P&G per blocchi di 8 bit
N-8=24 blocchi di operatori P&G per blocchi di 16 bit
N-16=16 blocchi di operatori P&G per blocchi di 32 bit
N=32 operatori per il calcolo del riporto
N=32 operatori per il calcolo della somma



Nel caso di un sommatore a N=64 bit essa impiega:

N=64 blocchi di operatori P&G dei singoli bit
N-1=63 blocchi di operatori P&G per blocchi di 2 bit
N-2=62 blocchi di operatori P&G per blocchi di 4 bit
N-4=60 blocchi di operatori P&G per blocchi di 8 bit
N-8=56 blocchi di operatori P&G per blocchi di 16 bit
N-16=48 blocchi di operatori P&G per blocchi di 32 bit
N-32=32 blocchi di operatori P&G per blocchi di 32 bit
N=64 operatori per il calcolo del riporto
N=64 operatori per il calcolo della somma

Sebbene l'albero binario sia il più semplice da capire e costruire, lo stesso ragionamento è estendile a un albero a base 4 (spesso usato al posto dell'albero binario). In questo caso i bit/gruppi sono assemblati quattro a quattro, riducendo il numero di stadi a $\log_4 N$. Tuttavia bisogna tenere conto che raggruppare quattro blocchi implica realizzare funzioni a 4-5 ingressi in ciascuna CLU, aumentandone i tempi di ritardo.
Bisogna quindi valutare se dimezzare il numero di stadi, raddoppiando gli ingressi nelle CLU sia più o meno convenieante.

Tempo di propagazione complessivo

RIPORTO: $T_{CARRY} = t_{SETUP} + \log_2 N \cdot t_{PG} + t_C$

SOMMA: $T_{SUM} = t_{SETUP} + \log_2 N \cdot t_{PG} + t_C + t_S$

La dipendenza da N avviene tramite t_{PG} (ma in modo logaritmico)
Raddoppiando il numero di bit il tempo di ritardo aumenta di $1xt_{PG}$

Assumendo per semplicità:
$t_{SETUP} = t_{PG} = t_C = t_S$

$$N = 8 \rightarrow T_{SUM} = t_{SETUP} + 3 \cdot t_{PG} + t_C + t_S = 6 \cdot T$$
$$N = 16 \rightarrow T_{SUM} = t_{SETUP} + 4 \cdot t_{PG} + t_C + t_S = 7 \cdot T$$
$$N = 32 \rightarrow T_{SUM} = t_{SETUP} + 5 \cdot t_{PG} + t_C + t_S = 8 \cdot T$$
$$N = 64 \rightarrow T_{SUM} = t_{SETUP} + 6 \cdot t_{PG} + t_C + t_S = 9 \cdot T$$

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 77

Tempi di ritardo di somma e riporto

L'architettura CLA permette il calcolo di somma e riporto con tempi che dipendono dal logaritmo di N:

$$T_{SUM} = t_{SETUP} + \log_2 N \cdot t_{PG} + t_C + t_S$$
$$T_{CARRY} = t_{SETUP} + \log_2 N \cdot t_{PG} + t_C$$

Numero di MOS per un sommatore a N bit

- Il numero di blocchi di setup sono N
- In base 2 il numero di stadi necessari al calcolo dei segnali PG di blocco è: **$\log_2 N$**
- il numero di CLU dello stadio k-esimo è **$N-2^{k-1}$**
- Il numero totale di CLU richiesti è: **$N \cdot \log_2 N - N + 1$**
- Il numero di operatori per il calcolo di somma e riporti finali sono N

$$Numero\ di\ MOS = 28N + 14(N \log_2 N - N + 1) = 14(N \log_2 N + N + 1)$$

		8 bit	16 bit	32bit	64 bit
Blocco	MOS/unità	*Unità*/**MOS**	*Unità*/**MOS**	*Unità*/**MOS**	*Unità*/**MOS**
Setup	14	*8* / **112**	*16* / **224**	*32* / **448**	*64* / **896**
CLU	14	*17* / **238**	*49* / **686**	*129* / **1806**	*321* / **4494**
Riporto	8	*8* / **64**	*16* / **128**	*32* / **256**	*64* / **512**
somma	6	*8* / **48**	*16* / **96**	*32* / **192**	*64* / **384**
	Totale MOS:	**462**	**1134**	**2702**	**6286**

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 78

Stima del numero di MOS

In generale, un sommatore CLA a N bit ha $\log_2 N$ stadi di CLU.
Il numero totale di CLU è:

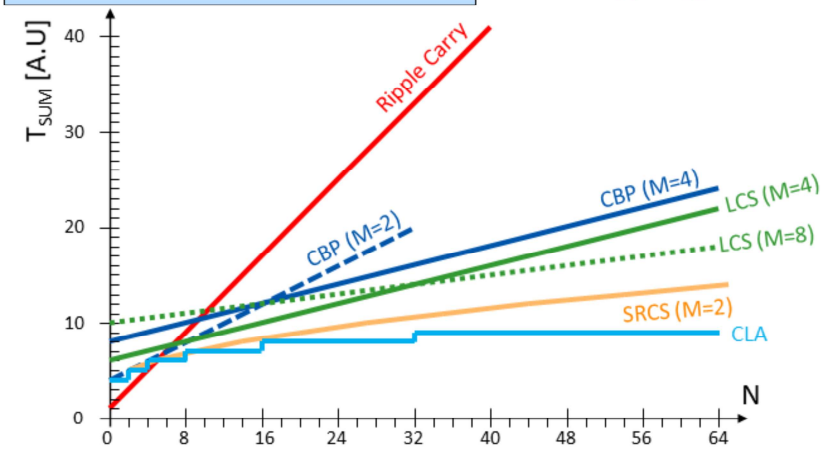
$$\sum_{i=1}^{\log_2 N} (N - 2^i) = \sum_{i=1}^{\log_2 N} N - \sum_{i=1}^{\log_2 N} 2^i = N \cdot \log_2 N - N + 1$$

Il numero di blocchi di calcolo dei segnali P e G singoli sono N.
Il numero di porte per il calcolo della somma e il riporto sono N.

Architetture a confronto

Supponiamo per semplicità $t_S = t_C = t_{MUX} = t_{SETUP} = T$

Ripple Carry	$T_{SUM} = t_{SETUP} + (N-1)t_C + t_S$	$= (N+1)T$
Carry-Bypass	$T_{SUM} = t_{SETUP} + (2M-1)t_C + (N/M-1)t_{MUX} + t_S$	$= 2MT + N(T/M)$
Lin. Carry. Sel.	$T_{SUM} = t_{SETUP} + Mt_C + (N/M)t_{MUX} + t_S$	$= T(M+2) + N(T/M)$
Sqr. Carry Sel.	$T_{SUM} = t_{SETUP} + Mt_C + \sqrt{2N}t_{MUX} + t_S$	$= T(M+2) + T\sqrt{2N}$
C. look-ahead	$T_{SUM} = t_{SETUP} + \log_2 N \cdot t_{PG} + t_C + t_S$	$= 3T + t_{PG} \cdot \log_2 N$



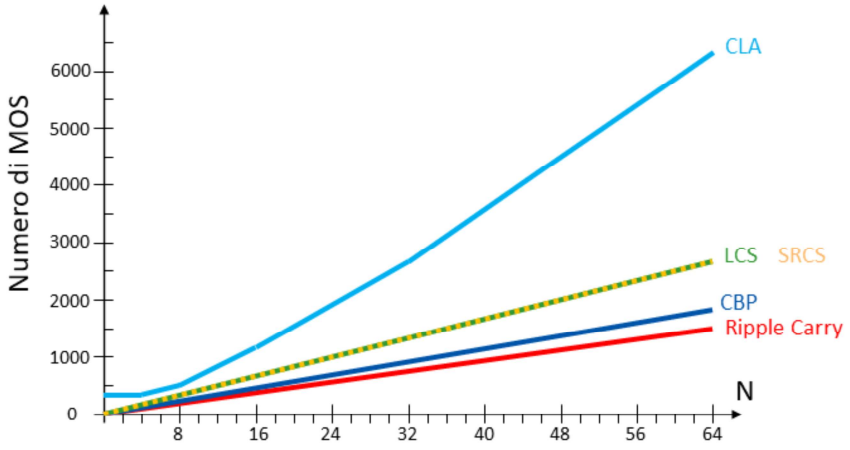
Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 79

Architetture a confronto: prestazioni

Completiamo il confronto delle prestazioni aggiungendo il sommatore CLA.
All'aumentare del numero di bit il divario di prestazioni è evidente.

Architetture a confronto: numero di MOS

Sommatore	8 bit	16 bit	32bit	64bit	N bit
Ripple carry	192	384	768	1536	24N
Carry by-pass	232	464	928	1856	29N
Linear Carry Select	336	672	1344	2688	42N
Square Root Carry Sel.	336	672	1344	2688	42N
Carry Look-ahead	462	1134	2702	6286	$14(N\log_2 N+N+1)$



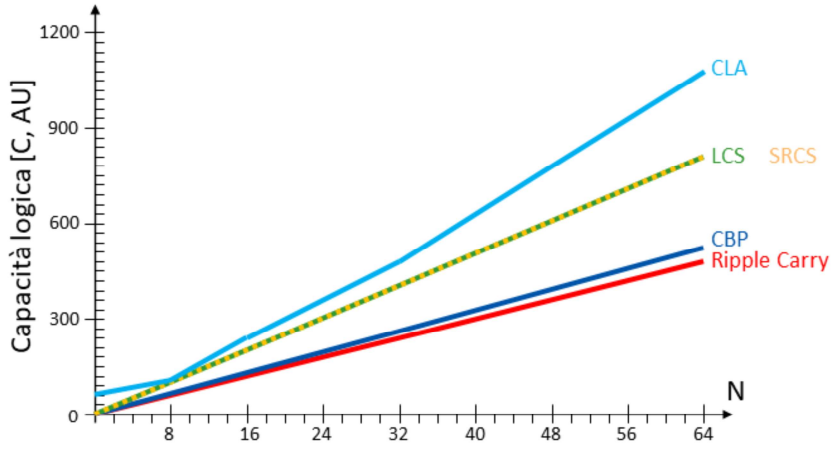
Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 80

Architetture a confronto: numero di MOS

L'elevata prestazione è pagata con un numero di MOS estremamente elevato che arriva fino a oltre il 400% rispetto al ripple-carry e al 250% in più rispetto a LCS/SRCS.

Architetture a confronto: consumo/capacità logica

Sommatore	8 bit	16 bit	32bit	64bit	N bit
Ripple carry	62C	124C	248C	486C	8N·C
Carry by-pass	67C	134C	268C	526C	8.625N·C
Linear Carry Select	102C	204C	408C	816C	13N·C
Square Root Carry Sel.	102C	204C	408C	816C	13N·C
Carry look-ahead	101C	223C	486C	1069C	-

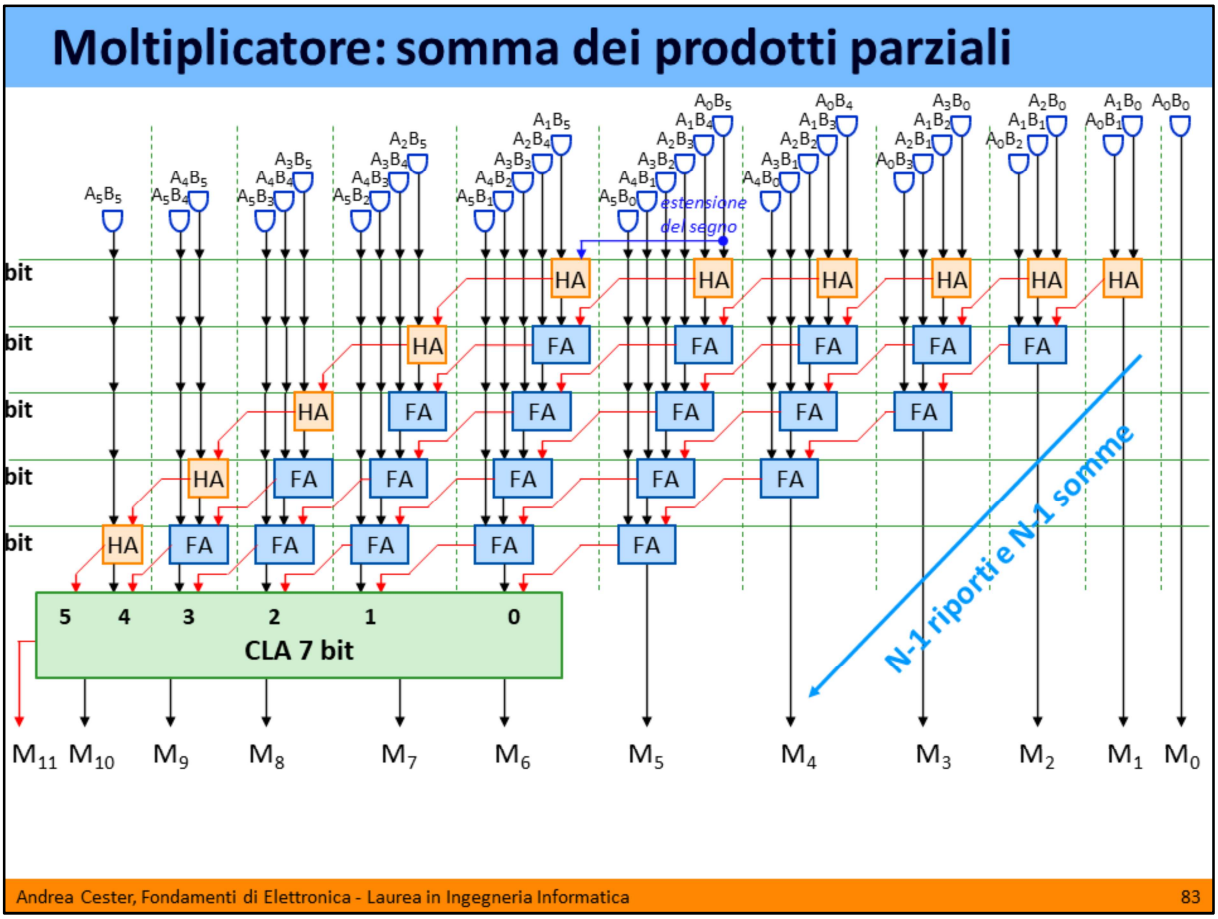


Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 81

Architetture a confronto: consumo dinamico / capacità logica

Anche il consumo aumenta, ma in modo minore rispetto al numero di MOS. Ciò è dovuto alla presenza di operazioni AND (blocchi di propagate) il cui fattore di attività è molto basso.

MOLTIPLICATORE OTTIMIZZATO



Moltiplicatore ottimizzato

Il sommatore CLA può essere impiegato come stadio finale anche nella moltiplicazione.
Ricordiamo che l'architettura del moltiplicatore comportava il calcolo di N-1 riporti/somme di prodotti parziali e uno stadio finale ripple-carry. Poiché gli ingressi dello stadio finale arrivano tutti allo stesso istante è possibile sostituire questo stadio con un sommatore ottimizzato.
In figura vediamo un esempio con interi di 6 bit (N=6).

Moltiplicatore: tempo di moltiplicazione

- Per il calcolo del prodotto è necessario aspettare:
  - Il tempo di propagazione attraverso le porte AND (array di prodotti bit a bit)
  - Il tempo di propagazione di (N – 1) riporti (o somme)
  - Il tempo di propagazione dell'ultimo stadio sommatore

$$T_{MULT} = t_{AND} + (N-1) \cdot \max(t_C, t_S) + T_{SUM}$$

Il sommatore dell'ultimo stadio può essere realizzato con architettura diversa dagli altri FA in modo da ottimizzare il tempo di propagazione (ad esempio carry-lookahead)

$$T_{MULT} = t_{AND} + (N-1) \cdot \max(t_C, t_S) + t_{SETUP} + t_{PG} \log_2 N + t_C' + t_S'$$

Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 84

Tempo di calcolo del moltiplicatore

Il tempo di calcolo del moltiplicatore richiede quindi:

- Il tempo di calcolo dei prodotti parziali t_{AND}
- Il tempo di N-1 somme di prodotti parziali: $(N-1)t_C$ o $(N-1)t_S$ a seconda se t_C è maggiore o minore di t_S. Nell'esempio di implementazione di FA adottata in queste slide t_C e t_S sono circa uguali.
- Il tempo di calcolo della somma finale: $t_{SETUP} + t_C + t_S + t_{PG}\log_2 N$

Mettendo tutto insieme e assumendo $t_S = t_C$:

$$T_{MULT} = t_{AND} + (N-1)t_C + t_{SETUP} + t_C + t_S + t_{PG}\log_2 N = (t_{AND} + t_{SETUP} + t_S) + Nt_C + t_{PG}\log_2 N$$

Il tempo di calcolo è ancora lineare, ma con pendenza dimezzata rispetto all'implementazione dello stadio finale con architettura ripple-carry.

Moltiplicatore: tempo di moltiplicazione

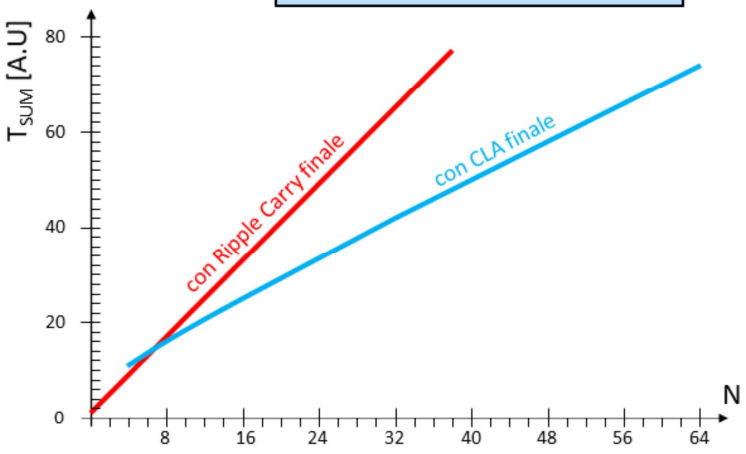
Con Ripple Carry $T_{MULT} = t_{AND} + (N-1) \cdot \max(t_C, t_S) + N \cdot t_C + t_S$

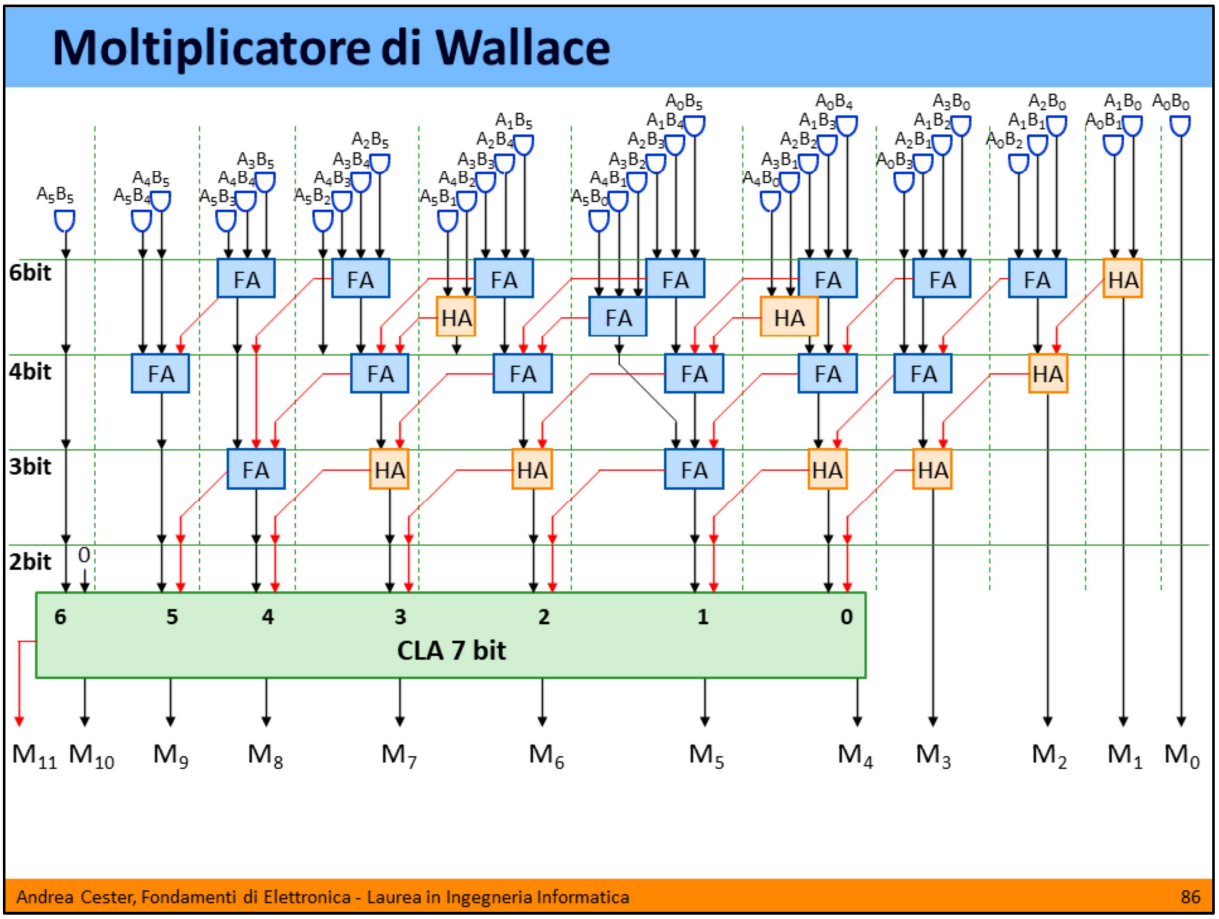
Con Carry look-ahead $T_{MULT} = t_{AND} + (N-1) \cdot \max(t_C, t_S) + t_{SETUP} + t_{PG} \log_2 N + t_C' + t_S'$

Assumendo che ciascun blocco elementare abbia tempo di ritardo T

Con Ripple Carry $T_{MULT} = (2N+1) \cdot T$

Con Carry look-ahead $T_{MULT} = (N + \log_2 N + 3) \cdot T$





Moltiplicatore di Wallace

Il moltiplicatore può essere ulteriormente ottimizzato usando un diverso approccio per l'accumulo dei prodotti parziali usando la struttura mostrata in figura (con N = 6 bit), che prende il nome di moltiplicatore di Wallace. Si tratta di è una struttura ad albero proposta da Chris Wallace nel 1964 per eseguire la moltiplicazione di numeri interi binari ad alta velocità.
Invece di sommare i prodotti parziali in modo sequenziale come nella struttura carry-save (che porta a un tempo di calcolo linearmente dipendente da N), il moltiplicatore di Wallace utilizza una struttura ad albero, aumentando il parallelismo. Esso è composto da 3 parti:

1) Generazione dei prodotti parziali.
Ogni bit di un operando è moltiplicato è moltiplicato per ciascun bit del secondo operando. La struttura è una matrice di porte AND esattamente uguale all'approccio carry-save.

2) Accumulo dei prodotti parziali con riduzione ad albero
La matrice dei prodotti parziali viene ridotta a soli due numeri: una riga di somma e una riga di riporto. Questo avviene nel modo seguente:

- I bit sono disposti in N righe ciascuna traslata di un bit a sinistra (come nel moltiplicatore carry-save). In tutto sono necessarie 2N-1 colonne.
- In ciascuna colonna i bit sono sommati a 3 a 3 mediante FA ove possibile o a coppie di due tramite HA. Ad esempio, in figura la colonna centrale ha 6 bit e sono raggruppati tre a tre: si usano due FA. La colonna alla sua sinistra ha 5 bit e sono sommati 3 con un FA e 2 con un HA. In entrambi FA/HA producono un bit di somma che viene trasferito al secondo stadio (stessa colonna) e un bit di riporto che è trasferito al secondo stadio (nella colonna a sinistra). Qualora in una colonna avanzi un solo bit, esso è trasferito direttamente al secondo stadio, come, ad esempio, i prodotti A_5B_5 o A_5B_2 nella. In questo modo tutte le terne o le coppie di bit possono essere sommati in parallelo (senza la necessita di conoscere il riporto.

Moltiplicatore: tempo di moltiplicazione

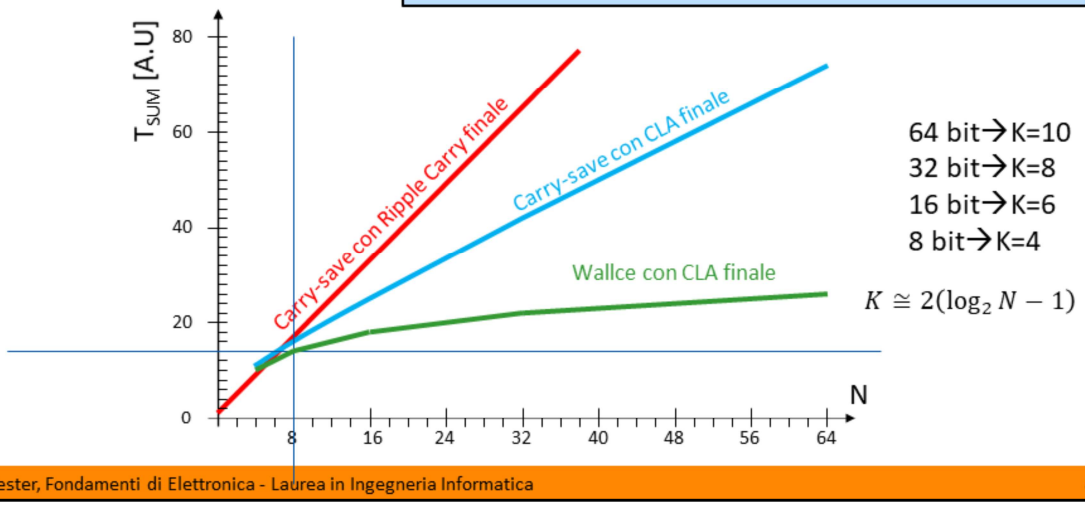
Carry Save $T_{MULT} = t_{AND} + (N-1)\cdot \max(t_C, t_S) + t_{SETUP} + t_{PG}\log_2 N + t_C' + t_S'$

Wallace $T_{MULT} = t_{AND} + K\cdot \max(t_{pFA}, t_{pHA}) + t_{SETUP} + t_{PG}\log_2 N + t_C' + t_S'$

Assumendo che ciascun blocco elementare abbia tempo di ritardo T e i FA (più complessi) 2T

Carry Save $T_{MULT} = (N + \log_2 N + 3)\cdot T$

Wallace $T_{MULT} = T(K\cdot 2T + \log_2 N + 4) \cong T(4\log_2 N + 2)$



Andrea Cester, Fondamenti di Elettronica - Laurea in Ingegneria Informatica 87

- Dal primo al secondo stadio, il numero di bit viene ridotto di un bit ogni 3. Notiamo infatti che la matrice di porte AND fornisce 6 righe. Il numero di uscite del primo stadio (ingressi del secondo) sono ridotti a 4. Nel secondo stadio e nei successivi, si procede nello stesso modo. Usando prioritariamente FA, si sommano i bit 3 a 3, riducendo di un ulteriore bit ogni 3 il numero di righe.
- Si procede iterativamente in questo modo fino a quando rimangono solo due righe. Nell'esempio mostrato (6 bit) il numero di righe viene ridotto da 6 a 4, 3, 2 in quattro stadi.

3) Somma finale.

I due numeri rimanenti vengono sommati tra loro usando un sommatore veloce come un carry-lookahead per ottenere il prodotto finale.

Tempo di calcolo

Il tempo di calcolo può essere espresso come:

$$T_{MULT} = t_{AND} + K\cdot \max(t_{pFA}, t_{pHA}) + T_{SUM}$$

T_{SUM} indica il tempo di calcolo dell'ultimo stadio. K è il numero di stadi necessari. Per stimare K in funzione di N, teniamo conto che ad ogni stadio viene eliminata una riga su tre. Quindi il numero di righe si riduce a 2/3 del precedente. Dopo K stadi il numero di righe è:

$$N\ (2/3)^K$$

e deve essere pari a:

$$N\left(\frac{2}{3}\right)^K = 2 \rightarrow \left(\frac{3}{2}\right)^K = \frac{N}{2} \rightarrow K = \frac{\log_2 \frac{N}{2}}{\log_2 \frac{3}{2}} \cong 2(\log_2 N - 1)$$

Ad esempio se N = 64 la riduzione del numero di righe avviene con la seguente progressione:

$$64 \rightarrow 43 \rightarrow 29 \rightarrow 20 \rightarrow 14 \rightarrow 10 \rightarrow 7 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \qquad (10\ stadi)$$

sommatore floating-point: stima dei MOS richiesti

Moltiplicatore doppia precisione (64 bit)	
Sommatori:	1x~~176~~ 784
	1x~~264~~ 784
Segno:	1x10
Traslatore:	1x532
Moltiplicatore:	
- AND + somme parz.:	82166
- Sommatore finale:	~~1272~~ 5208
TOT.	~~83148~~ MOS
	89484

sommatore doppia precisione (64 bit)	
MUX:	1x112
	2x532
Sottrattore (exp):	1x~~286~~ 784
Modulo:	1x~~264~~ 784
	1x~~1344~~ 5502
Traslatore:	2x1580
Somm./Sottr.:	1x~~1936~~ 5502
	1x~~380~~ 784
LZC:	744
Segno:	~50
TOT.	~~9340~~ MOS
	18486

Esercizi proposti

Problema 1

DATI: t_C = 0.2ns, t_S = 0.3ns, t_{SETUP} = 0.4ns, t_{MUX} = 0.2ns

1. Calcolare i tempi necessari a calcolare la somma e il riporto finale con i seguenti numeri a 20 bit (LSB a sinistra) usando l'architettura ripple-carry, carry-by-pass (con blocchi da 4 bit) e linear carry select (con blocchi da 4 bit):
   a) A = 1111 1111 1111 1111 1111 e B = 1111 1111 1111 1111 1111
   b) A = 1111 1111 1111 1111 1111 e B = 1000 0000 0000 0000 0000
   c) A = 1111 1111 1111 1111 1111 e B = 0000 0000 0000 0000 0000
   d) A = 1111 1111 1111 1111 1110 e B = 1000 0000 0000 0000 0000
   e) A = 1111 0001 1111 1111 1111 e B = 1000 1100 0000 0000 1000
2. Usare l'architettura square-root carry select per ottimizzare il tempo di ritardo. Come conviene raggruppare i bit? E quanto vale il tempo di ritardo nel caso peggiore?
3. Usare l'architettura carry-lookahead con t_{PG} = 0.2ns. Quanto vale il tempo di ritardo?

Soluzione

1a) RC: T_{CARRY} = 0.6ns; T_{SUM} = 0.9ns;
CBP e LCS: T_{CARRY} = 0.8ns; T_{SUM} = 1.1ns;
1b) RC: T_{CARRY} = 4.4ns; T_{SUM} = 4.5ns;
CBP: T_{CARRY} = 2.2ns; T_{SUM} = 2.9ns;
LCS: T_{CARRY} = 2.2ns; T_{SUM} = 2.5ns;
1c) RC: T_{CARRY} = 4.4ns; T_{SUM} = 4.5ns;
CBP: T_{CARRY} = 1.4ns; T_{SUM} = 2.1ns;
LCS: T_{CARRY} = 2.2ns; T_{SUM} = 2.5ns;
1d) RC: T_{CARRY} = 0.6ns; T_{SUM} = 4.5ns;
CBP: T_{CARRY} = 0.8ns; T_{SUM} = 2.9ns;
LCS: T_{CARRY} = 0.8ns; T_{SUM} = 2.5ns;
1d) RC: T_{CARRY} = 1.2ns; T_{SUM} = 2.7ns;
CBP: T_{CARRY} = 1.4ns; T_{SUM} = 2.1ns;
LCS: T_{CARRY} = 1.4ns; T_{SUM} = 1.9ns;
2) T_{CARRY} = 1.8ns; T_{SUM} = 2.1ns;
3) T_{CARRY} = 1.6ns; T_{SUM} = 1.9ns;

Esercizi proposti

Problema 2

Consideriamo un sommatore CLA a 32 bit in cui i blocchi hanno i seguenti tempi di ritardo:

- Segnali di propagate e generate primari: t_P = 0.15ns e t_G = 0.1ns.
- Segnali P&G di blocco: t_{PG} = 0.2ns
- Riporto; t_C = 0.25ns
- Somma t_S = 0.3ns

1. Calcolare i tempi necessari a calcolare la somma e il riporto finale.
2. Usiamo questo sommatore per realizzare un moltiplicatore in cui FA e HA hanno tempo di ritardo $t_C = t_S$ = 0.3ns e le porte AND hanno untempo di ritardo di 0.1ns. Calcolare il tempo necessario a eseguire la moltiplicazione.
3. Come cambia la risposta al punto 2 se al posto del sommatore CLA usiamo un sommatore ripple carry realizzato con gli stessi FA del moltiplicatore?

Soluzione

1) T_{CARRY} = 1.65ns; T_{SUM} = 1.95ns;
2) 2) T_{MULT} = 11.35ns;
3) 3) T_{MULT} = 19ns